Intelligent Reader Protocol Specification
Document version 1.96
Protocol version 1.56

Vendotek, Ltd.

9 October 2026

Contents

1 Introduction **10**
- 1.1 Getting started . . . 10
- 1.2 The protocol versioning . . . 10
- 1.3 The protocol compatibility . . . 10
- 1.4 The specification versioning . . . 10

2 Message overview **11**
- 2.1 Description . . . 11
- 2.2 Size . . . 11
- 2.3 HEADER . . . 11
  - 2.3.1 Description . . . 11
  - 2.3.2 Size . . . 11
- 2.4 LENGTH . . . 11
  - 2.4.1 Description . . . 11
  - 2.4.2 Size . . . 11
- 2.5 MSGID . . . 12
  - 2.5.1 Description . . . 12
  - 2.5.2 Size . . . 12
- 2.6 MODULE . . . 12
  - 2.6.1 Description . . . 12
  - 2.6.2 Size . . . 12
- 2.7 TYPE . . . 12
  - 2.7.1 Description . . . 12
  - 2.7.2 Size . . . 13
- 2.8 PAYLOAD . . . 13
  - 2.8.1 Description . . . 13
  - 2.8.2 Size . . . 13
- 2.9 CHECKSUM . . . 13
  - 2.9.1 Description . . . 13
  - 2.9.2 Size . . . 13

3 Protocol specification **14**
- 3.1 Message types . . . 14
  - 3.1.1 Command message . . . 15
  - 3.1.2 Successful response message . . . 16
  - 3.1.3 Failure response message . . . 17
  - 3.1.4 Pending response message . . . 18
  - 3.1.5 Notification message . . . 19
  - 3.1.6 Control message . . . 20

4 Messages specification **21**
- 4.1 Miscellaneous commands module . . . 22
  - 4.1.1 Set LEDs state . . . 23
  - 4.1.2 Read device specific information . . . 24
  - 4.1.3 Reboot device . . . 25
  - 4.1.4 Make a sound by the buzzer . . . 26
  - 4.1.5 Get device statistic . . . 27
  - 4.1.6 Echo . . . 29

4.1.7 Change COM-port baudrate . . . 30
4.1.8 Get possible COM-port baudrates . . . 31
4.1.9 Change LAN settings . . . 32
4.1.10 Set color for all RGB LEDs . . . 33
4.1.11 Set color for certain RGB LEDs . . . 34
4.1.12 Set RGB LEDs color by mask . . . 35
4.1.13 Get RGB LEDs information . . . 36
4.1.14 Set RGB LEDs brightness . . . 37
4.2 Contact cards module . . . 38
4.2.1 Power on . . . 39
4.2.2 Power off . . . 40
4.2.3 Transmit APDU . . . 41
4.3 Contactless Level 1 module . . . 42
4.3.1 Poll for token . . . 43
4.3.2 Run EMV Removal procedure . . . 46
4.3.3 Transceive data to contactless token . . . 47
4.3.4 Transceive ISO14443-4 command . . . 48
4.3.5 Power OFF reader's RF field . . . 49
4.3.6 Send the RATS command . . . 50
4.4 Contactless Level 2 module . . . 51
4.4.1 Perform transaction . . . 52
4.4.2 Get EMV kernel information . . . 59
4.5 Mifare Cards module . . . 60
4.5.1 Mifare Cards Authentication Parameters . . . 62
4.5.2 Mifare Classic/Plus memory map . . . 63
4.5.3 Mifare Classic Auto-Recovery Feature . . . 63
4.5.4 NXP SAM AV3 auto-reset . . . 63
4.5.5 Mifare Classic Authentication on Clear Key . . . 64
4.5.6 Mifare Classic Authentication on Key from SAMAV2 . . . 65
4.5.7 Mifare Classic Authentication on Internal Key . . . 66
4.5.8 Mifare Classic Read Data Blocks . . . 67
4.5.9 Mifare Classic Write Data Blocks . . . 68
4.5.10 Mifare Classic Get Counter Value . . . 69
4.5.11 Mifare Classic Set Counter Value . . . 70
4.5.12 Mifare Classic Modify Counter Value . . . 71
4.5.13 Mifare Classic Copy Counter Value . . . 72
4.5.14 Mifare Classic Commit Counter Value . . . 73
4.5.15 Mifare Classic Bulk operation . . . 74
4.5.16 Mifare Plus Authentication on Clear Key . . . 76
4.5.17 Mifare Plus Authentication on SAM AV2 . . . 77
4.5.18 Mifare Plus Authentication on Internal Key . . . 78
4.5.19 Mifare Plus Read Data Blocks . . . 79
4.5.20 Mifare Plus Write Data Blocks . . . 80
4.5.21 Mifare Plus Get Counter Value . . . 81
4.5.22 Mifare Plus Set Counter Value . . . 82
4.5.23 Mifare Plus Modify Counter Value . . . 83
4.5.24 Mifare Plus Copy Counter Value . . . 84
4.5.25 Mifare Plus Commit Counter Value . . . 85
4.5.26 Mifare Plus Bulk operation . . . 86
4.5.27 Mifare SAM AV2 Host Authentication . . . 88
4.5.28 Unlock Mifare SAM AV2 . . . 89
4.5.29 Change Key Entry in Mifare SAM AV2 . . . 90
4.5.30 Mifare Ultralight Read Pages . . . 92
4.5.31 Mifare Ultralight Write Pages . . . 93
4.5.32 Mifare Ultralight EV1 Get Counter Value . . . 94
4.5.33 Mifare Ultralight EV1 Increment Counter . . . 95
4.5.34 Mifare Ultralight EV1 Get Version . . . 96
4.5.35 Mifare Ultralight Bulk operation . . . 97
4.5.36 Mifare Ultralight-C 3DES Authentication on Clear Key . . . 99
4.5.37 Mifare Ultralight-C 3DES Authentication on Key from SAM AV2/AV3 . . . 100
4.5.38 Mifare Ultralight-C 3DES Authentication on Internal Key . . . 101

4.5.39 Mifare Ultralight EV1 Password-based Authentication on Clear Password . . . 102
4.5.40 Mifare Ultralight EV1 Password-based Authentication on Key from the SAM AV3 . . . 103
4.6 Mifare Extended commands module . . . 104
4.6.1 Mifare Classic Read Multiple Sectors . . . 105
4.6.2 Mifare Classic Write Multiple Sectors . . . 107
4.6.3 Read Product Version Info of Generic Mifare Card . . . 109
4.7 STMicroelectronics Cards module . . . 110
4.7.1 STM SRI512 / ST25TB Read Blocks . . . 111
4.7.2 STM SRI512 / ST25TB Write Blocks . . . 112
4.8 NXP NTAG Cards module . . . 113
4.8.1 NXP NTAG Select File . . . 114
4.8.2 NXP NTAG Authenticate Card On Clear Key . . . 115
4.8.3 NXP NTAG Authenticate Card On Internal Key . . . 116
4.8.4 NXP NTAG Read File . . . 117
4.8.5 NXP NTAG Write File . . . 118
4.9 PIN-pad module . . . 119
4.9.1 Inject a secret . . . 120
4.9.2 List injected secrets . . . 123
4.9.3 Encrypt data . . . 124
4.9.4 Compute HMAC . . . 126
4.9.5 Enter and encrypt PIN . . . 127
4.9.6 Compute MAC . . . 129
4.10 Service module . . . 131
4.10.1 Update Firmware . . . 132
4.10.2 Upload reader configuration . . . 138
4.10.3 Self test . . . 143
4.11 Graphic User Interface module . . . 144
4.11.1 Show user-defined screen . . . 145
4.11.2 Input screen dialog . . . 152
4.11.3 Menu screen dialog . . . 153
4.11.4 Draw bitmap . . . 155
4.11.5 Slideshow . . . 156
4.12 Touchscreen module . . . 157
4.12.1 Poll touchscreen for an event . . . 158
4.13 Complex module . . . 159
4.13.1 Poll for an event . . . 160
4.14 QR-code module . . . 162
4.14.1 Poll for QR-code . . . 163
4.14.2 Send command to the QR-scanner . . . 164
4.15 CIPURSE cards module . . . 165
4.15.1 Authorize HW SAM . . . 166
4.15.2 Select File . . . 167
4.15.3 Establish Secure Channel . . . 168
4.15.4 Read Binary Record . . . 169
4.15.5 Update Binary Record . . . 170
4.15.6 Read Linear Record . . . 172
4.15.7 Update Linear Record . . . 173
4.15.8 Append New Cyclic Record . . . 174
4.15.9 Read Value-Record . . . 175
4.15.10 Modify Value-Record . . . 176
4.16 Turnstile control module . . . 177
4.16.1 Allow one pass . . . 178
4.17 Troika transport card module . . . 179
4.17.1 Read logical ticket . . . 180
4.17.2 Validate ticket and write bitmap . . . 181
4.18 Speaker module . . . 182
4.18.1 Play audio file . . . 183
4.18.2 Speaker test . . . 184
4.19 Failure response . . . 185
4.20 Notification message . . . 187
4.20.1 Log message . . . 188

4.20.2 User message . . . 189

5 Best Practices 190
5.1 Hardware diagnostic . . . 191
5.1.1 Periodical diagnostic . . . 191
5.1.2 On demand diagnostic . . . 191
5.2 TCP-keepalive and re-connection . . . 192
5.3 Broken connection diagnostic for USB/RS232 port . . . 192
5.4 How to customize “AWAITING CONNECTION” screen . . . 192
5.4.1 Custom picture . . . 192
5.4.2 Build custom screen . . . 192
5.5 How to prevent display flickering . . . 192

6 Annex A Protobuf Scalar Value Types 193

7 Annex B Reader Configuration 194
7.1 Overview . . . 194
7.2 Customer pictures . . . 194
7.2.1 Convert GIF image to BMP sprite for the Slideshow command . . . 194
7.3 Customer MP3 audio files . . . 195
7.4 Contactless . . . 195
7.4.1 Built-in parameters . . . 195
7.5 Yoko QR-scanners . . . 196
7.5.1 E4000 default (built-in) configuration . . . 196
7.5.2 E10 default (built-in) configuration . . . 196
7.6 Hardware configuration . . . 197
7.6.1 RGB LEDs brightness . . . 197
7.7 Apple Enhanced Contactless Polling (ECP) configuration . . . 197

8 Annex C Built-in Fonts Metrics 198
8.1 Regular Font Symbols . . . 198
8.2 Large Font Symbols . . . 199
8.3 Monospace Font Symbols . . . 201

List of Tables

4.1 Miscellaneous . . . 22
4.2 Leds . . . 23
4.4 DeviceInfo . . . 24
4.6 Reboot . . . 25
4.8 MakeSound . . . 26
4.10 DeviceStatistic . . . 27
4.12 GetEcho . . . 29
4.13 Echo . . . 29
4.15 TryBaudrate . . . 30
4.17 PossibleBaudrates . . . 31
4.19 LanSettings . . . 32
4.20 ChangeLanSettings . . . 32
4.22 SetAllLeds . . . 33
4.24 SetLeds . . . 34
4.25 Led . . . 34
4.27 SetLedsByMask . . . 35
4.29 LedsInfo . . . 36
4.31 SetLedsBrightness . . . 37
4.33 ContactLevel1 . . . 38
4.34 PowerOnCard . . . 39
4.35 ContactCard . . . 39
4.37 PowerOffCard . . . 40
4.39 TransmitApdu . . . 41
4.40 ResponseApdu . . . 41
4.42 ContactlessLevel1 . . . 42
4.43 PollForToken . . . 43
4.46 Token . . . 44
4.48 TokenType . . . 45
4.49 EmvRemoval . . . 46
4.51 TransceiveBitArray . . . 47
4.52 BitArray . . . 47
4.54 Command . . . 48
4.55 Response . . . 48
4.57 PowerOffField . . . 49
4.59 AnswerToSelect . . . 50
4.61 ContactlessLevel2 . . . 51
4.62 PerformTransaction . . . 52
4.65 TransactionResult . . . 54
4.66 TransactionStatus . . . 54
4.67 PerformTransaction Errors . . . 54
4.71 TokenCarrierType . . . 57
4.72 KernelInfo . . . 59
4.73 Kernel . . . 59
4.75 Mifare . . . 61
4.76 KeyType . . . 62
4.77 KeyType . . . 62
4.78 SectorKey . . . 62
4.79 SectorKeyType . . . 62

4.80 SpecialKey . . . 62
4.81 ClearKey . . . 64
4.83 SamKey . . . 65
4.85 InternalKey . . . 66
4.87 ReadBlocks . . . 67
4.88 Blocks . . . 67
4.90 WriteBlocks . . . 68
4.92 GetCounter . . . 69
4.93 Counter . . . 69
4.95 SetCounter . . . 70
4.97 ModifyCounter . . . 71
4.99 CopyCounter . . . 72
4.101CommitCounter . . . 73
4.103BulkOperation . . . 74
4.104Command . . . 74
4.105BulkResult . . . 74
4.106CommandResult . . . 74
4.108ClearKey . . . 76
4.110SamKey . . . 77
4.112InternalKey . . . 78
4.114ReadBlocks . . . 79
4.115Blocks . . . 79
4.117WriteBlocks . . . 80
4.119GetCounter . . . 81
4.120Counter . . . 81
4.122SetCounter . . . 82
4.124ModifyCounter . . . 83
4.126CopyCounter . . . 84
4.128CommitCounter . . . 85
4.130BulkOperation . . . 86
4.131Command . . . 86
4.132BulkResult . . . 86
4.133CommandResult . . . 86
4.135AuthenticateHost . . . 88
4.137Unlock . . . 89
4.139ChangeKeyEntry . . . 90
4.140MifareKeyEntry . . . 90
4.141PlusKeyEntry . . . 90
4.142UltralightCKeyEntry . . . 90
4.144ReadPages . . . 92
4.145Pages . . . 92
4.147WritePages . . . 93
4.149GetCounter . . . 94
4.150CounterValue . . . 94
4.152IncrementCounter . . . 95
4.154GetVersion . . . 96
4.155Version . . . 96
4.157BulkOperation . . . 97
4.158Command . . . 97
4.159BulkResult . . . 97
4.160CommandResult . . . 97
4.162ClearKey . . . 99
4.164SamKey . . . 100
4.166InternalKey . . . 101
4.168ClearPassword . . . 102
4.169PasswordAcknowledge . . . 102
4.171SamPassword . . . 103
4.172PasswordAcknowledge . . . 103
4.174MifareExtended . . . 104
4.175ReadSectors . . . 105
4.176Sector . . . 105

4.177SectorsData . . . 105
4.178SectorData . . . 105
4.180WriteSectors . . . 107
4.181WriteSector . . . 107
4.183ProductVersionInfo . . . 109
4.185StmCard . . . 110
4.186ReadBlocks . . . 111
4.187Blocks . . . 111
4.189WriteBlocks . . . 112
4.191NtagCard . . . 113
4.192SelectFile . . . 114
4.194ClearKey . . . 115
4.196InternalKey . . . 116
4.198ReadFile . . . 117
4.199FileContent . . . 117
4.201WriteFile . . . 118
4.203PinPad . . . 119
4.204DeviceAttributes . . . 120
4.205InjectionData . . . 120
4.206Secret . . . 120
4.207SecretAttributes . . . 120
4.208InjectSecret . . . 121
4.209InjectResult . . . 121
4.212SecretsInfo . . . 123
4.213SecretInfo . . . 123
4.215CipherInput . . . 124
4.216CipherOutput . . . 124
4.217CBC . . . 124
4.218CipherPadding . . . 125
4.220HmacInput . . . 126
4.221HmacOutput . . . 126
4.223EnterOnlinePin . . . 127
4.224EntryResult . . . 127
4.225EntryStatus . . . 127
4.227GenerateMac . . . 129
4.228MacOutput . . . 129
4.230MacKeyDerivationMethod . . . 130
4.231Service . . . 131
4.233FirmwareManifest . . . 133
4.234FirmwareImage . . . 133
4.235Prepare . . . 133
4.236FirmwareImage . . . 134
4.238UpdateBlock . . . 135
4.240Apply . . . 136
4.243Prepare . . . 139
4.245UploadBlock . . . 140
4.247CheckConfiguration . . . 141
4.248Configuration . . . 141
4.249File . . . 141
4.251SelfTestResult . . . 143
4.252Hardware . . . 143
4.253HwComponentStatus . . . 143
4.255Gui . . . 144
4.256VerticalLayout . . . 145
4.257HorizontalLayout . . . 145
4.258Text . . . 145
4.259Picture . . . 145
4.260CustomerPicture . . . 145
4.261QrCode . . . 146
4.262GeneratedText . . . 146
4.264VerticalAlignment . . . 146

4.265HorizontalAlignment . . . 147
4.266Font . . . 147
4.267PictureId . . . 147
4.268ShowScreen . . . 150
4.270InputDialog . . . 152
4.271EnteredText . . . 152
4.273MenuDialog . . . 153
4.274SelectedItem . . . 153
4.276DrawBitmap . . . 155
4.278Slideshow . . . 156
4.280Touchscreen . . . 157
4.281PollTouchscreen . . . 158
4.282TouchscreenEvent . . . 158
4.284Complex . . . 159
4.285MifareCardAction . . . 160
4.286PollForEvent . . . 160
4.287Event . . . 160
4.289QrCode . . . 162
4.290PollForQrCode . . . 163
4.291EventType . . . 163
4.292Indication . . . 163
4.293QrCodeEvent . . . 163
4.295SendCommand . . . 164
4.296ScannerReply . . . 164
4.298Cipurse . . . 165
4.299AuthorizeHwSam . . . 166
4.301SelectFile . . . 167
4.303EstablishSecureChannel . . . 168
4.305ReadBinary . . . 169
4.306Binary . . . 169
4.308UpdateBinary . . . 170
4.310ReadRecord . . . 172
4.311Record . . . 172
4.313UpdateRecord . . . 173
4.315AppendRecord . . . 174
4.317ReadValue . . . 175
4.318Values . . . 175
4.320ModifyValue . . . 176
4.321NewValue . . . 176
4.323Turnstile . . . 177
4.324AllowOnePass . . . 178
4.326TroikaCard . . . 179
4.327ReadTicket . . . 180
4.328Ticket . . . 180
4.330WriteTicket . . . 181
4.331WriteResult . . . 181
4.333Speaker . . . 182
4.334PlayFile . . . 183
4.336SpeakerTest . . . 184
4.337SineWaveTest . . . 184
4.339FailureResponse . . . 185
4.340Error . . . 186
4.341Notification . . . 187
4.342LogMessage . . . 188
4.343Importance . . . 188
4.344UserMessage . . . 189
4.345MessageId . . . 189

6.1 Protobuf Scalar Value Types . . . 193

8.1 Regular Font . . . 199

8.2 Large Font 201

Chapter 1

Introduction

This document describes transport and application layers of the IntelliReader protocol.

1.1 Getting started

See the Getting Started Guide for the basic information about the protocol.

1.2 The protocol versioning

The protocol version is the `MAJOR.MINOR` part of the IntelliReader software version. For example, software version 1.23.3898 implements the protocol version 1.23.

The protocol version described in this document: 1.56

1.3 The protocol compatibility

New versions of the IntelliReader protocol are backward compatible with previous ones. For example, version 1.35 supports all commands/messages that were supported in the version 1.23.

In rare cases some commands can be marked as deprecated (obsolete). Such commands still supported in the newer protocol but **we strongly discourage you from use deprecated commands in new projects**.

1.4 The specification versioning

This document has its own versioning. It's version does not track the protocol versioning and does not directly depend from the protocol changes. The version change of this document tracks the change of the content of this document.

Chapter 2

Message overview

This chaper describes the structure of the messages used in the protocol.

2.1 Description

Message is bytes sequence in binary format, that consists of the following fields:

| HEADER | LENGTH | MSGID | MODULE | TYPE | PAYLOAD | CHECKSUM |

2.2 Size

Size of the message is variable, but at least 9 bytes for messages without *PAYLOAD* field and up to 4128 bytes.

2.3 HEADER

2.3.1 Description

This field allows easily find start of the message in the byte sequence.

Encoded as two ASCII chars: 'I' and 'R' (`4952` in hex).

2.3.2 Size

2 bytes.

2.4 LENGTH

2.4.1 Description

Defines the length of the included fields, except *HEADER*, *LENGTH* itself and *CHECKSUM*. Encoded as 16-bit BigEndian number.

Includes lengths of following fields:

- MSGID
- MODULE
- TYPE
- PAYLOAD

2.4.2 Size

2 bytes.

2.5 MSGID

2.5.1 Description

Continuously incremented message id, starts from 0 and wraps to 0 after 255. This field intended to be synchronization mark between *IR* and *Host*, that prevents processing messages out of order.

2.5.2 Size

1 byte.

2.6 MODULE

2.6.1 Description

This field describes *module* within *IR* software. It's meaning depends from the source of the message:

- if message issued by *IR*, then field describes *module* that issued this message
- if message issued by *Host*, then field describes *module* for which the message is intended

Possible values:

- **00h**: Special module type used in the `Notification message`
- **01h**: `Miscellaneous commands module`
- **02h**: `Contact cards module`
- **03h**: `Contactless Level 1 module`
- **04h**: `Contactless Level 2 (PayWave, Paypass, Mir) module`
- **05h**: `Mifare cards module`
- **06h**: `PIN-pad module`
- **07h**: `Service operations module`
- **08h**: `STMicroelectronics cards module`
- **09h**: `NXP NTAG cards module`
- **0Ah**: `Graphic User Interface (GUI) module`
- **0Bh**: `Touch-screen module`
- **0Ch**: `Module to combine requests to multiple interfaces`
- **0Dh**: `QR-code/Barcode reader module`
- **0Eh**: `CIPURSE cards module`
- **10h**: `Turnstile control module`
- **11h**: `Mifare Extended commands module`
- **12h**: `Troika transport card module`
- **13h**: `Speaker module`

Depending of loaded *IR* software some *module*s may be absent (unsupported).

2.6.2 Size

1 byte.

2.7 TYPE

2.7.1 Description

This field describes *type* of message. This field was added to simplify message processing allowing perform some intermediate processing at transport (instead of application) layer of protocol handling.

Ex. to reset timer used for response timeout when *Pending* response is arrived. Or reroute *Notification* message to logging entity within *Host* application while waiting for final response to *Command* message.

Possible values:

- **01h**: Command message
- **02h**: Successful (final) response message
- **03h**: Failure (final) response message
- **04h**: Pending (intermediate) response message

- **05h**: Notification message
- **06h**: Control message

2.7.2 Size

1 byte.

2.8 PAYLOAD

2.8.1 Description

This field contains application payload, that depends from *MODULE* and *TYPE* fields of current message.

Content of this field described by Google Protocol Buffers's *protocol* files (`*.proto` files) and encoded by their rules. *IR* uses version `2` of protobuf specification.

This field can be empty or contains single protobuf's *message*.

It worth mentioning that "decoder" of payload *must* know what is encoded in it. To distinct between various payloads that may be included in message's payload, OneOf feature is used.

2.8.2 Size

Variable (including 0), depends of protobuf's *message* that used for encoding.

2.9 CHECKSUM

2.9.1 Description

This field used to check integrity of the message.

This field encoded as XMODEM's CRC16 stored as 16-bit BigEndian number.

Calculated by whole message excluding itself.

2.9.2 Size

2 bytes.

Chapter 3

Protocol specification

3.1 Message types

This chapter describes usage and peculiar properties of various message types.

3.1.1 Command message

Command message always issued by *Host* and handled by *IR*.

3.1.1.1 Example

Host sends message that requests *IR* to enable the blue LED:

```
> 4952 # HEADER:  two ASCII bytes 'IR'
> 0007 # LENGTH:  16-bit number, encoded length of 7 bytes
> 01   # MSGID:   message id 01h
> 01   # MODULE:  module number 01h
> 01   # TYPE:    *Command* message
> 0A02 # PAYLOAD: Google Protobuf description intended for this payload:
> 0801 #
>      #  message Miscellaneous {
>      #     oneof misc_cmd {
>      #         // Sets new state of the LEDs
>      #         misc.leds.Leds set_leds_state = 1;
>      #     }
>      #  }
>      #
>      #  package misc.leds;
>      #  message Leds {
>      #     optional bool blue = 1;
>      #     optional bool yellow = 2;
>      #     optional bool green = 3;
>      #     optional bool red = 4;
>      #  }
>      #
>      # Human readable representation:
>      #   set_leds_state { blue: true }
> 09DF # CHECKSUM: 16-bit number of XMODEM CRC16
```

IR handles incoming message and responses with *Successful response* message:

```
< 4952 # HEADER:  two ASCII bytes 'IR'
< 0003 # LENGTH:  16-bit number, encoded length of 3 bytes
< 01   # MSGID:   message id 01h, it must match *Command* message
< 01   # MODULE:  module number 01h
< 02   # TYPE:    *Successful response* message
<      # PAYLOAD: absent because this *Command* message does not imply any response data
< B26B # CHECKSUM: 16-bit number of XMODEM CRC16
```

3.1.2 Successful response message

IR sends *Successful response* message as answer to *Command* message. *MODULE* and *MSGID* fields in this message must match those from *Command*. *PAYLOAD* field may contains associated response data or may be empty, it is defined by this protocol specification for each *Command* message.

3.1.2.1 Example

Host sends *Command* message with payload contains "poll for token" request:

```
> XX.. # *Command* message is omitted for brevity
```

Successful response message for "poll for token" request from *Contactless Level 1* module:

```
< 4952 # HEADER:  two ASCII bytes 'IR'
< 000B # LENGTH:  16-bit number, encoded length of 11 bytes
< 01   # MSGID:   message number 01h, it must match *Command* message
< 03   # MODULE:  module number 03h (Contactless Level1)
< 02   # TYPE:    *Successful response* message
< 0803 # PAYLOAD: Google Protobuf description intended for this payload:
< 1204 #
< 1234 #  package contactless.token;
< 5678 #  message Token {
>      #      required contactless.token_type.TokenType type = 1;
>      #      required bytes id = 2;
>      #      optional bytes answer_to_select = 3;
>      #  }
<      #
<      # Human readable representation:
<      #  type: MIFARE_CLASSIC_1K id: "\0224Vx"
< 635A # CHECKSUM: 16-bit number of XMODEM CRC16
```

3.1.3 Failure response message

All *Failure response*s use the same protobuf description for *PAYLOAD* field.

3.1.3.1 Example

Host sends *Command* message with payload contains "poll for token" request:

```
> XX.. # *Command* message is omitted for brevity
```

Failure response message for "poll for token" request from *Contactless Level1* module:

```
< 4952 # HEADER:  two ASCII bytes 'IR'
< 0005 # LENGTH:  16-bit number, encoded length of 5 bytes
< 01   # MSGID:   message number 01h, it must match *Command* message
< 03   # MODULE:  module number 03h (Contactless Level1)
< 03   # TYPE:    *Failure response* message
< 080D # PAYLOAD: Google Protobuf description intended for this payload:
<      #
<      #  package common.failure;
<      #  message FailureResponse {
<      #      required Error error = 1;
<      #      optional string error_string = 2;
<      #  }
<      #
<      # Human readable representation:
<      #  error: CARD_TIMEOUT
< AA1F # CHECKSUM: 16-bit number of XMODEM CRC16
```

3.1.4 Pending response message

Pending response message from *IR* indicates that:

- *Command* message successfully received by the *IR* and
- message now processing

IR sends such messages for commands that required undetermined time to process request, ex. “poll for token”.

In current version of the specification all *Pending response*s contains empty *PAYLOAD* field.

3.1.4.1 Example

Host sends *Command* message with payload contains “poll for token” request:

```
> XX.. # *Command* message is omitted for brevity
```

Pending response message for “poll for token” request from *Contactless Level1* module:

```
< 4952 # HEADER:  two ASCII bytes 'IR'
< 0003 # LENGTH:  16-bit number, encoded length of 3 bytes
< 02   # MSGID:   message number 02h, it must match *Command* message
< 03   # MODULE:  module number 03h (Contactless Level1)
< 04   # TYPE:    *Pending response* message
<      # PAYLOAD: empty
< ED9F # CHECKSUM: 16-bit number of XMODEM CRC16
```

3.1.5 Notification message

This message used by *IR* to notify *Host* about some actions performed internally in *IR*. It used to carry:

- log message, that may be collected by *Host*
- user interface message, that may be displayed to the end user

This message is intermediate, *IR* issues it between incoming *Command* message and final *Response* message. This message does not respects initial *Command*'s value of *MSGID* field nor value of *MODULE* field.

NOTE: Notification message always uses `00h` for `MSGID` & `MODULE` fields.

3.1.5.1 Example

Notification message:

```
< 4952 # HEADER:  two ASCII bytes 'IR'
< 0021 # LENGTH:  16-bit number, encoded length of 33 bytes
< 00   # MSGID:   message number unused and set always to 00h
< 00   # MODULE:  module number unused and set always to 00h
< 05   # TYPE:    *Notification* message
< 0A1C # PAYLOAD: Google Protobuf description intended for this payload:
< 0807 #
< 1218 #  package common.notification;
< 454D #  message Notification {
< 563A #      oneof notification {
< 2061 #          LogMessage log_message = 1;
< 6374 #      }
< 6976 #  }
< 6174 #
< 696E #  message LogMessage {
< 6720 #      enum Importance { /* ... */ }
< 6361 #
< 7272 #      required Importance level = 1;
< 6965 #      required string     msg   = 2;
< 720A #  }
<      #
<      # Human readable representation:
<      #  log_message { level: INFO msg: "EMV: activating carrier\n" }
< EE24 # CHECKSUM: 16-bit number of XMODEM CRC16
```

3.1.6 Control message

This message used by *Host* to control behavior of *IR*.

3.1.6.1 Cancellation control message

This type of *Control* message should be used to cancel currently pending *Command* from *Host*. This message should be sent after *Command* message and before final *Response* message, otherwise it will be ignored by *IR*.

Cancellation message have no dedicated module within *IR*, therefore it cancels any pending *Command* currently processing by the *IR*. It also may have any value of *MSGID* field.

3.1.6.2 Example

Cancellation message:

```
< 4952 # HEADER:  two ASCII bytes 'IR'
< 0003 # LENGTH:  16-bit number, encoded length of 3 bytes
< 00   # MSGID:   message number unused and set always to 00h
< 00   # MODULE:  module number unused and set always to 00h
< 06   # TYPE:    *Control* message
<      # PAYLOAD: empty
< F6EE # CHECKSUM: 16-bit number of XMODEM CRC16
```

Chapter 4

Messages specification

This chapters describes messages that can be used in *IR* protocol.

Note: Not all IntelliReader models support messages described below. If device does not support command it returns `Failure response` *with* `UNSUPPORTED_COMMAND` *error.*

4.1 Miscellaneous commands module

Each message should contains *one of* the following message types:

Field Name	Label	Type	Description
set leds state	optional	Leds	Sets new state of the LEDs
read device info	optional	ReadDeviceInfo	Reads out information about the reader
reboot device	optional	Reboot	Reboots device to one of operation modes
get device status	optional	GetDeviceStatus	Gets device status
make sound	optional	MakeSound	Makes sound on the buzzer
get device statistic	optional	GetDeviceStatistic	Gets various device statistic
get echo	optional	GetEcho	Replies with the same data
change baudrate	optional	ChangeBaudrate	DEPRECATED
change lan settings	optional	ChangeLanSettings	Changes LAN settings (IP-address, netmask & default gateway)
try baudrate	optional	TryBaudrate	Tries to change COM-port (UART/RS232) baudrate
confirm baudrate	optional	ConfirmBaudrate	Confirms COM-port (UART/RS232) baudrate
get possible baudrates	optional	GetPossibleBaudrates	Gets list of COM-port (UART/RS232) baudrates that are possible to use
set all rgb leds	optional	SetAllLeds	Sets color on all RGB LEDs
set rgb leds	optional	SetLeds	Sets color on certain RGB LEDs
set rgb leds by mask	optional	SetLedsByMask	Sets color on certain RGB LEDs by mask
get rgb leds info	optional	GetLedsInfo	Gets RGB LEDs information
set rgb leds brightness	optional	SetLedsBrightness	Sets max brightness of RGB LEDs

Table 4.1: Miscellaneous

Module ID: **01h**.

4.1.1 Set LEDs state

This message allow independently set each LED to the `ON/OFF` state. This message should contains only those LEDs which state should be changed. Other LEDs will preserve its state.

4.1.1.1 Request

Request contains `Leds` message:

Field Name	Label	Type	Description
blue	optional	bool	The Blue LED on the front panel
yellow	optional	bool	The Yellow LED on the front panel
green	optional	bool	The Green LED on the front panel
red	optional	bool	The Red LED on the front panel
lcd backlight	optional	bool	The Backlight of the LCD
emv symbol	optional	bool	The EMV symbol on the front panel

Table 4.2: Leds

The `emv_symbol` field is available since v. 1.50.

4.1.1.2 Response

No data carried with successful response.

4.1.1.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.1.1.4 Example

Set 'Blue' LED to ON state:

```
set_leds_state { blue: true }
```

4.1.2 Read device specific information

This command allows to read information specific to this device such as serial number and firmware version.

This command is used in the Initialization phase.

4.1.2.1 Request

Request contains `ReadDeviceInfo` empty message.

4.1.2.2 Response

IR answers with `DeviceInfo` message:

Field Name	Label	Type	Description
serial number	required	string	Serial Number of the device
intellireader version	required	string	Version of IntelliReader software
hardware model id	optional	uint32	Proprietary ID of the hardware model
hardware description	optional	string	Proprietary hardware description
external device info	optional	DeviceInfo	External IntelliReader device information

Table 4.4: DeviceInfo

`hardware_model_id` field presents in the response since v. 1.7.

`external_device_info` field presents in the response since v. 1.55 if an external IntelliReader device is connected to the reader.

4.1.2.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.1.3 Reboot device

Reboots device into the one of `Operational Modes`.

Before actual reboot device responses with empty `Successful Response`.

4.1.3.1 Request

Request contains `Reboot` message:

Field Name	Label	Type	Description
mode	optional	OperationMode	Operational mode that will used after reboot

Table 4.6: Reboot

4.1.3.2 Response

No data carried with successful response.

4.1.3.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.1.3.4 Example

Reboot to *Management mode*:

```
reboot_device { mode: MANAGEMENT_MODE }
```

4.1.4 Make a sound by the buzzer

Produce a sound (note) by the device's buzzer. It is possible to play simple melody by joining various `Note`s.

Note that `IR` returns response immediately and ready for the new command. Melody will be playing in the background.

4.1.4.1 Request

Request contains `MakeSound` message:

Field Name	Label	Type	Description
melody	repeated	Note	Melody consists of the notes

Table 4.8: MakeSound

4.1.4.2 Response

No data carried with successful response.

4.1.4.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.1.4.4 Example

Produce 'Bip-Bop' by the buzzer:

```
make_sound
{
    melody
    {
        frequency_hz: 1500 duration_ms: 100 silence_duration_ms: 150
    }
    melody
    {
        frequency_hz: 1000 duration_ms: 100
    }
}
```

4.1.5 Get device statistic

Get various statistic data collected by the device.

4.1.5.1 Request

Request contains `GetDeviceStatistic` empty message.

4.1.5.2 Response

IR answers with `DeviceStatistic` message:

Field Name	Label	Type	Description
lwip	optional	Statistic	lwIP TCP/IP stack statistic
events	repeated	Event	List of an events recorded by the reader in the internal ring buffer
ethernet	optional	Statistic	Built-in ethernet switch statistic
smartcard	optional	Statistic	Smartcard statistic

Table 4.10: DeviceStatistic

The `events` field

Available since v. 1.10.

IntelliReader has internal ring buffer for various events such as TCP connection is established, closed and so on. Because that buffer is limited controller should periodically reads out it by issue this command. We propose to send this command when TCP disconnection occurs and controller creates new connection.

Because reader has no built-in real time clock, controller should convert time in `event_time_offset` field to real time by adjusting it as follow: `real_time - event_time_offset`.

Reader clears internal ring buffer when gets this command.

The `ethernet` field

Available since v. 1.13.

Represent statistic got from MIB counters of the built-in Ethernet switch.

The `smartcard` field

Available since v. 1.52.

Provides usage statistic of Smartcard Slots.

4.1.5.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.1.5.4 Example

Example response with some statistic data:

```
lwip {
    link {
        xmit: 3 recv: 3 fw: 0 drop: 0 chkerr: 0 lenerr: 0
        memerr: 0 rterr: 0 proterr: 0 opterr: 0 err: 0
    }
    arp {
        xmit: 18 recv: 10 fw: 0 drop: 0 chkerr: 0 lenerr: 0
```

```
        memerr: 0 rterr: 0 proterr: 0 opterr: 0 err: 0
    }
    ip {
        xmit: 205 recv: 181 fw: 0 drop: 0 chkerr: 0 lenerr: 0
        memerr: 0 rterr: 0 proterr: 0 opterr: 0 err: 0
    }
    tcp {
        xmit: 161 recv: 164 fw: 0 drop: 0 chkerr: 0 lenerr: 0
        memerr: 0 rterr: 0 proterr: 0 opterr: 0 err: 0
    }
}
events {
    event_type: DEVICE_REBOOT
    event_time_offset: 819755
    event_info: "ESP_RST_POWERON"
}
events {
    event_type: IP_ADDRESS_IS_OBTAINED
    event_time_offset: 243350
    event_info: "10.42.0.10"
}
events {
    event_type: CONNECTION_ESTABLISHED
    event_time_offset: 239535
}
ethernet {
    port1 { tx_packets: 181 rx_packets: 197 rx_errors: 0 txrx_dropped: 0 }
    port2 { tx_packets: 0   rx_packets: 0   rx_errors: 0 txrx_dropped: 77 }
    port3 { tx_packets: 179 rx_packets: 223 rx_errors: 0 txrx_dropped: 0 }
}
```

4.1.6 Echo

Get echoed reply from the device. It is most simple way to check connection with the device.

4.1.6.1 Request

Request contains `GetEcho` message:

Field Name	Label	Type	Description
send pending	optional	bool	Reply with 'Pending' message before actual response
reply size	optional	uint32	Size of 'echo' field in the response (max. 4096)
data	optional	bytes	Any data

Table 4.12: GetEcho

4.1.6.2 Response

IR answers with `Echo` message:

Field Name	Label	Type	Description
echo	optional	bytes	Echoed from 'data' field of the request

Table 4.13: Echo

To change size of the `echo` field use `reply_size` field in the request. Note that `reply_size = 0` means that `echo` field will be absent at all. If `reply_size` more than `data` size then `echo` field will be padded by zeroes up to required length.

4.1.6.3 Auxiliary messages

Message	Comments
Pending	Return if `send_pending` field set to `true` in the request otherwise no
Notification	Never return
Cancellation	Ignore

4.1.7 Change COM-port baudrate

IntelliReader supports wide range of baudrates but particular hardware configuration can restrict possible baudrates.

To change baudrate in fault tolerant approach the baudrate changing procedure is splitted to the two phases:

1. Temporary change baudrate to new value and restart reader to temporary use new value
2. Confirm temporary changed baudrate so it becomes permanent

If inappropriate baudrate was selected then at the second phase reader is not able to successfully receive confirmation request. In such case you should reboot reader again via power line to restore previously used baudrate.

Available since version 1.35.

4.1.7.1 Temporary change request

Request contains `TryBaudrate` message:

Field Name	Label	Type	Description
baudrate	required	uint32	New baudrate value (bits per second)

Table 4.15: TryBaudrate

You may get possible baudrates using `Get possible COM-port baudrates` command.

4.1.7.2 Temporary change response

No data carried with successful response.

Reader automatically reboots and waits for commands at given temporary baudrate. The temporary baudrate will be active until reboot or next baudrate confirmation command, which makes it permanent.

4.1.7.3 Confirm request

Request contains `ConfirmBaudrate` message.

4.1.7.4 Confirm request response

No data carried with successful response.

Reader saves current baudrate as permanent. Now reader permanently use new baudrate.

4.1.7.5 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.1.7.6 Example

Try baudrate `921600`:

```
try_baudrate { baudrate: 921600 }
```

Wait 5-10 seconds until IntelliReader is booted up. Reconfigure your UART port to use new baudrate.

Confirm new baudrate:

```
confirm_baudrate { }
```

4.1.7.7 Deprecation note

`ChangeBaudrate` command is **deprecated** in favour of this command since v. 1.35.

4.1.8 Get possible COM-port baudrates

Lets to get list of baudrates that are possible to use on the current hardware model.

4.1.8.1 Request

Request contains empty `GetPossibleBaudrates` message.

4.1.8.2 Response

Successful response contains `PossibleBaudrates` message with list of possible baudrates:

Field Name	Label	Type	Description
baudrates	repeated	uint32	Possible baudrates in bits per second

Table 4.17: PossibleBaudrates

4.1.8.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.1.9 Change LAN settings

Change IntelliReader's LAN settings to the one of following mode:

Field Name	Label	Type	Description
dhcp	optional	Dhcp	Use DHCP to retrieve LAN settings
manual	optional	ManualSettings	Use manually prepared LAN settings

Table 4.19: LanSettings

The new setting will be used after a restart.

Please, be carefully when changing LAN settings. Reader may become unreachable if you set up wrong settings!

This command is available from protocol version 1.24

4.1.9.1 Request

Request contains `ChangeLanSettings` message:

Field Name	Label	Type	Description
lan settings	required	LanSettings	New LAN settings

Table 4.20: ChangeLanSettings

4.1.9.2 Response

No data carried with successful response.

4.1.9.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.1.9.4 Example

Change the LAN settings to use manually assigned IP-address:

```
change_lan_settings 'lan_settings {
    manual { local_address: "192.168.0.2" netmask: "255.255.255.0" gateway: "192.168.0.1" }
}'
```

4.1.10 Set color for all RGB LEDs

This command allows assign a color to all RGB LEDs.

4.1.10.1 Request

Request contains `SetAllLeds` message:

Field Name	Label	Type	Description
color	required	uint32	Color format: RED[23:16] GREEN[15:8] BLUE[7:0]

Table 4.22: SetAllLeds

4.1.10.2 Response

No data carried with successful response.

4.1.10.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.1.10.4 Example

Set all LEDs to OFF state:

```
set_all_rgb_leds { color: 0 }
```

Set all LEDs to white color:

```
set_all_rgb_leds { color: ffffff }
```

4.1.11 Set color for certain RGB LEDs

This command allows independently assign a color to each RGB LED by providing pairs of (number, color).

4.1.11.1 Request

Request contains `SetLeds` message:

Field Name	Label	Type	Description
leds	repeated	Led	List of the LEDs and their colors

Table 4.24: SetLeds

Field Name	Label	Type	Description
number	required	uint32	Number of the LED starting from 0
color	required	uint32	Color format: RED[23:16] GREEN[15:8] BLUE[7:0]

Table 4.25: Led

4.1.11.2 Response

No data carried with successful response.

4.1.11.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.1.11.4 Example

Set second (starting from zero), forth and 15th LED to red, green and blue colors accordingly:

```
set_rgb_leds {
    leds { number: 2 color: "ff0000" }
    leds { number: 4 color: "ff00" }
    leds { number: 15 color: "ff" }
}
```

4.1.12 Set RGB LEDs color by mask

This command allows independently assign a color to each RGB LED by providing mask of LEDs and colors.

4.1.12.1 Request

Request contains `SetLedsByMask` message:

Field Name	Label	Type	Description
leds mask	required	uint64	Each bit is set to 1 defines the number of LED that uses color data from the ‘colors‘ field
colors	repeated	uint32	Colors of LEDs that set to 1 in the mask. Color format: RED[23:16] GREEN[15:8] BLUE[7:0]

Table 4.27: SetLedsByMask

If length of the `colors` field less then bits in the ’leds_mask‘ then the last color will be used for remaining LEDs.

If the `colors` field is empty then black color (LED is off) is used.

4.1.12.2 Response

No data carried with successful response.

4.1.12.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.1.12.4 Example

Set all LEDs to OFF state:

```
set_rgb_leds_by_mask { leds_mask: FFFFFFFFFFFFFFFF }
```

Set all LEDs to white color:

```
set_rgb_leds_by_mask { leds_mask: FFFFFFFFFFFFFFFF colors: "ffffff" }
```

Set second (starting from zero), forth and 15th LED to red, green and blue colors accordingly:

```
set_rgb_leds_by_mask { leds_mask: 8014 colors: "ff0000" "ff00" "ff" }
```

4.1.13 Get RGB LEDs information

Some new reader models are equipped with RGB LEDs. This command returns information about them.

4.1.13.1 Request

Request contains `GetRgbLedsInfo` message.

4.1.13.2 Response

Successful response contains `LedsInfo` message:

Field Name	Label	Type	Description
leds count	required	uint32	Number of RGB LEDs

Table 4.29: LedsInfo

4.1.13.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.1.14 Set RGB LEDs brightness

This command allows to change the max brightness of the RGB LEDs till device reboot.

See also the `RGB LEDs brightness` in the Hardware configuration chapter.

Available from version 1.48.

4.1.14.1 Request

Request contains `SetLedsBrightness` message:

Field Name	Label	Type	Description
red	required	uint32	Max brightness of the red component in percentage (0-100)
green	required	uint32	Max brightness of the green component in percentage (0-100)
blue	required	uint32	Max brightness of the blue component in percentage (0-100)

Table 4.31: SetLedsBrightness

4.1.14.2 Response

No data carried with successful response.

4.1.14.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.1.14.4 Example

Set the max brightness to 50%:

```
set_rgb_leds_brightness { red: 50 green: 50 blue: 50 }
```

4.2 Contact cards module

Each message should contains *one of* the following message types:

Field Name	Label	Type	Description
power on card	optional	PowerOnCard	Power on Card (SAM)
power off card	optional	PowerOffCard	Power off Card (SAM)
transmit apdu	optional	TransmitApdu	Transmit Application PDU (APDU) to Card (SAM)

Table 4.33: ContactLevel1

Module ID: **02h**.

4.2.1 Power on

Power on contact SmartCard (SAM). If power was already enabled than disables power and enables it again (i.e. reset power).

Power on command resets command counter and increments resets in Smartcard statistic, obtainable by `Get device statistic`.

4.2.1.1 Request

Request contains `PowerOnCard` message:

Field Name	Label	Type	Description
slot	required	CardSlot	Card slot in the reader

Table 4.34: PowerOnCard

4.2.1.2 Response

Response contains `ContactCard` message:

Field Name	Label	Type	Description
historical bytes	required	bytes	Part of the Answer To Reset named as 'Historical Bytes'

Table 4.35: ContactCard

4.2.1.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.2.1.4 Example

Enable power on card in the 'SAM3_SLOT':

```
power_on_card { slot: SAM3_SLOT }
```

4.2.2 Power off

Power off SmartCard (SAM).

4.2.2.1 Request

Request contains `PowerOffCard` message:

Field Name	Label	Type	Description
slot	required	CardSlot	Card slot in the reader

Table 4.37: PowerOffCard

4.2.2.2 Response

No data carried with successful response.

4.2.2.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.2.2.4 Example

Power off SAM in 'SAM3_SLOT':

```
power_off_card { slot: SAM3_SLOT }
```

4.2.3 Transmit APDU

Transmit command APDU (Application Protocol Data Unit) to SmartCard (SAM) using ISO7816-4 protocol and receive the card's answer that consist of:

- optional body
- mandatory trailer (also known as SW bytes)

SmartCard (SAM) must be successfully powered by `Power on` command.

Transmit APDU command increments command counter in Smartcard statistic, obtainable by `Get device statistic`.

4.2.3.1 Request

Request contains `TransmitApdu` message:

Field Name	Label	Type	Description
slot	required	CardSlot	Card slot in the reader
command apdu	required	bytes	Contains mandatory header (4 bytes) and optional body of variable length

Table 4.39: TransmitApdu

4.2.3.2 Response

Response contains `ResponseApdu` message:

Field Name	Label	Type	Description
body	optional	bytes	Conditional body of variable length
trailer	required	uint32	Mandatory trailer of 2 bytes (SW1 SW2), example: 0x9000

Table 4.40: ResponseApdu

4.2.3.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.2.3.4 Example

Transmit `Get Version` command to card in `SAM3_SLOT` slot:

```
transmit_apdu { slot: SAM3_SLOT command_apdu: "\x80\x60\x00\x00\x00" }
```

4.3 Contactless Level 1 module

Each message should contains *one of* the following message types:

Field Name	Label	Type	Description
poll for token	optional	PollForToken	Polls for contactless token (card)
emv removal	optional	EmvRemoval	Runs EMV Removal procedure
tsv bit array	optional	TransceiveBitArray	Transceive BitArray to contactless token (card)
iso14443 4 command	optional	Command	Send ISO14443-4 command to token and receive the response
power off field	optional	PowerOffField	Power OFF reader's RF field
request for ats	optional	RequestForAnswerToSelect	Issue the RATS command to the card

Table 4.42: ContactlessLevel1

Module ID: **03h**.

4.3.1 Poll for token

Run EMV complaint polling loop within reader.

4.3.1.1 Request

Request contains `PollForToken` message:

Field Name	Label	Type	Description
timeout	optional	uint32	Timeout of poll for token to be present
prefer mifare	optional	bool	In case of dual contactless card (SmartMX, ISO14443A + Mifare) don't "switch" to ISO14443-4 level
poll stm sri512	optional	bool	Optional polling sequence for STM SRI512
polling mode	optional	PollingMode	Polling mode
light up led	optional	bool	Light up the polling LED before this command processing
poll iso15693	optional	bool	Optional polling sequence for ISO15693 compatible cards
poll ask cts	optional	bool	Optional polling sequence for ASK C.ticket CTS compatible cards
poll felica	optional	bool	Optional polling sequence for FeliCa compatible cards
polling led	optional	PollingLed	Optional polling LED mode

Table 4.43: PollForToken

`timeout` field defines whether reader waits for card before performing transaction:

- if `timeout` field is present then
- – if `timeout` equals to `0` then
- – ∗ reader returns previous result of `"Poll for token"` command
- – else
- – ∗ `timeout` defines time in *milliseconds* during which reader will be wait for token
- else
- – reader will be waits for token forever

If `light_up_led` set to `true`, then reader automatically turns the polling LED ON before execute the command.

That behaviour also applied to the `Perform transaction` and `Poll for an event` commands.

In case of hardware is equipped with more than four (4) RGB LEDs you can choose fancy RGB lighting for the polling by setting `polling_led` field to the `POLLING_LED_RGB_MODE_1` value.

Compatibility note: Since version 1.51 the `light_up_led` feature doesn't off the polling LED after the command execution.

4.3.1.2 Polling mode

There are following polling modes:

Polling mode	Description
`LOW_POWER_POLLING`	Use that mode if device is **equipped by LCD**. Consider that mode **for all devices** because it reduces power consumption & overall device heating.
`POLLING_WITH_PAUSES`	That mode **increases power consumption & device heating** compare to the previous mode but also increases card read distance.
`CONTINOUS_POLLING`	Consider that mode when there is **hard requirement** to poll for card without pauses.

4.3.1.3 Features availability

Some features are available only from the certain protocol version:

Feature	Available since version
`LOW_POWER_POLLING`	1.14
`light_up_led`	1.22
`poll_iso15693`	1.24
`poll_ask_cts`	1.27
`polling_led`	1.51

4.3.1.4 Response

Response contains `Token` message:

Field Name	Label	Type	Description
type	required	TokenType	Type of the token
id	required	bytes	Token's unique ID or serial number
answer to select	optional	bytes	Token response to ISO14443-4A Request ATS command
atqa	optional	bytes	Response to WUPA
sak	optional	bytes	Final response to ISO14443-3A SELECT - Select Acknowledge (SAK)

Table 4.46: Token

`answer_to_select` field always presents for `ISO_14443_4A`

It can be present for `SMART_MX_WITH_MIFARE_1K` and `SMART_MX_WITH_MIFARE_4K` if request was issued with `prefer_mifare = false`.

4.3.1.5 Auxiliary messages

Message	Comments
Pending Notification	Return One `PRESENT_ONE_CARD_ONLY` *User Notification Message* can be returned to each collision event occurred during `Poll for token` request. *Host* can display message to the cardholder or ignore that notification.
Cancellation	Allow

4.3.1.6 Example

Poll for token during `1000 ms`:

```
poll_for_token { timeout: 1000 }
```

Immediately return response for last card:

```
poll_for_token { timeout: 0 }
```

Infinitely poll for token (`timeout` field is absent):

```
poll_for_token { }
```

4.3.1.7 Type of the token

`Poll for token` command uses `Select acknowledge (SAK)` byte and algorithm from *"NXP's AN10834 Application Note Rev. 4.0 - 8 February 2017"* to distinguish various Mifare card types. For some types *Host* should use additional commands (such as `Request ATS`, `Deselect VC` and etc) to distinguish certain type of the token.

`Token Types`:

Value	Number	Description
UNKNOWN	0	Reader unable to determinate card type
ISO 14443 4A	1	Card compatible with ISO 14443 Layer 4A (used by EMVCo)
ISO 14443 4B	2	Card compatible with ISO 14443 Layer 4B (used by EMVCo)
MIFARE CLASSIC 1K	3	Mifare Classic 1K
MIFARE CLASSIC 2K	4	Mifare Classic 2K
MIFARE CLASSIC 4K	5	Mifare Classic 4K
MIFARE CLASSIC MINI	6	Mifare Mini
MIFARE PLUS X SL2 2K	7	Mifare Plus X 2K in SL2
MIFARE PLUS X SL2 4K	8	Mifare Plus X 4K in SL2
MIFARE UL OR ULC	9	Mifare Ultralight (or Ultralight C)
SMART MX WITH MIFARE 1K	10	Dual card: Smart MX (ISO14443A) + Mifare Classic 1K
SMART MX WITH MIFARE 4K	11	Dual card: Smart MX (ISO14443A) + Mifare Classic 4K
TAG N PLAY	12	
STM SRI512	13	STMicroelectronics SRI512 tag
GENERIC ISO15693	14	Generic ISO15693 compatible card
ICODE SLI	15	NXP ICODE SLI
ICODE SLIX	16	NXP ICODE SLIX
ICODE SLIX2	17	NXP ICODE SLIX2
ASK CTS512B	18	ASK C.ticket CTS512B
FELICA	19	Sony FeliCa

Table 4.48: TokenType

4.3.2 Run EMV Removal procedure

4.3.2.1 Request

Request contains `EmvRemoval` message:

Field Name	Label	Type	Description
timeout	optional	uint32	Timeout of token to be absent (no timeout = forever)
deselect	optional	bool	Whether to deselect the card before starting the procedure

Table 4.49: EmvRemoval

By the definition the EMV Removal procedure should be executed until card leaves the field. If you need to restrict maximum time of that operation you can:

- set `timeout` to desired time in `milliseconds` **or**
- cancel command by *Cancellation* request at any time

The electromagnetic (RF) field will be deactivated at the end of EMV Removal procedure.

The `deselect` field is available since version 1.52.

4.3.2.2 Response

No data carried with successful response.

4.3.2.3 Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Allow

4.3.2.4 Example

Request to perform EMV Removal procedure no more than `5` seconds:

```
emv_removal { timeout: 5000 }
```

4.3.3 Transceive data to contactless token

Send and receive data to/from contactless token using following options:

- CRC in both directions
- Parity bit in both directions

Contactless token must be “activated” by `"Poll for token"` command before use this command.

4.3.3.1 Request

Request contains `TransceiveBitArray` message:

Field Name	Label	Type	Description
bit array	required	BitArray	Array of bits to send
response timeout us	required	uint32	Time for waiting for answer in microseconds
tx crc	optional	bool	Generate and transmit CRC along with ‘bit array‘
rx crc	optional	bool	Check CRC in the response and then cut it off
parity	optional	bool	Generate/check parity bit in the TX/RX data

Table 4.51: TransceiveBitArray

4.3.3.2 Response

Response contains `BitArray` message:

Field Name	Label	Type	Description
data	required	bytes	Array of bits grouped by eight bits in one byte
count	required	uint32	Count of bits in ‘data‘ array

Table 4.52: BitArray

4.3.3.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.3.3.4 Example

Transceive EMV ‘Select PPSE’ I-Block to contactless token. *IR* adds parity and CRC bits automatically:

```
tsv_bit_array {
    bit_array {
        data: "\x02\x00\xa4\x04\x00\x0e\x32\x50\x41\x59\x2e\x53\x59\x53\x2e\x44\x44\x46\x30\x31\x00"
        count: 168
    }
    response_timeout_us: 50000
    tx_crc: true
    rx_crc: true
    parity: true
}
```

4.3.3.5 Array Of Bits

This commands operates by ‘Array Of Bits’ messages because there are cards that can accept and response with data that’s length is not multiply by eight bits (ex. Mifare Classic NAKs).

`count` field of `BitArray` defines data size in bits.

4.3.4 Transceive ISO14443-4 command

Send raw ISO14443-4 (also known as ISO Data Exchange Protocol (DEP)) command to the token and receive the response.

Contactless token must be "activated" by `"Poll for token"` command before use this command. `TokenType` must be `ISO_14443_4A` or `ISO_14443_4B`.

Controller do not need to fragment the payload (chaining), it will be automatically fragmented and defragmented if it exceeds FSD/FSC limits.

Response timeout defined by the token.

It worth noting that in the DEP protocol there is mechanism named "Frame waiting time extension". Thanks to this mechanism token is able to infinitely extend time of the answer. In such situation it is possible to cancel command by the `Cancel` control message request.

4.3.4.1 Request

Request contains `Command` message:

Field Name	Label	Type	Description
data	required	bytes	ISO14443-4 INF payload

Table 4.54: Command

4.3.4.2 Response

Response contains `Response` message:

Field Name	Label	Type	Description
data	required	bytes	ISO14443-4 INF payload

Table 4.55: Response

4.3.4.3 Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Allow

4.3.4.4 Example

Send "Select PPSE" command:

```
iso14443_4_command {
    data: "\x00\xa4\x04\x00\x0e\x32\x50\x41\x59\x2e\x53\x59\x53\x2e\x44\x44\x46\x30\x31\x00"
}
```

4.3.5 Power OFF reader's RF field

Power OFF electromagnetic field (RF field) that reader creates for contactless cards (PICC). Current status of the RF field does not influence on the command.

4.3.5.1 Request

Request contains `PowerOffField` message:

Field Name	Label	Type	Description
deselect	optional	bool	Whether to deselect the card before field is powered off

Table 4.57: PowerOffField

The `deselect` field is available since version 1.52.

4.3.5.2 Response

No data carried with successful response.

4.3.5.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.3.6 Send the RATS command

Send the RATS (Request For Answer To Select) command to "switch" the card from ISO14443-3A to ISO14443-4A layer. It is possible to use following steps for the dual cards (EMV + Mifare Classic):

1. Poll for token with `prefer_mifare = true`
2. If `token_type` set to `SMART_MX_WITH_MIFARE_1K` or `SMART_MX_WITH_MIFARE_4K` then
3. Try to authenticate the Mifare Classic token
4. If authentication is failed then
5. Send the RATS command and then
6. Perform transaction using `poll_for_token { timeout: 0 }`

This command is available from protocol version 1.1

4.3.6.1 Request

Request contains empty `RequestForAnswerToSelect` message.

4.3.6.2 Response

Response contains `AnswerToSelect` message:

Field Name	Label	Type	Description
data	required	bytes	Answer to Select data

Table 4.59: AnswerToSelect

4.3.6.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.4 Contactless Level 2 module

Each message should contains *one of* the following message types:

Field Name	Label	Type	Description
perform transaction	optional	PerformTransaction	Performs contactless transaction
get kernel info	optional	GetKernelInfo	Reads out information about embedded EMV kernels

Table 4.61: ContactlessLevel2

Module ID: **04h**.

4.4.1 Perform transaction

Wait for card (if requested) and perform the EMV transaction with the card.

When `IR` got request it sends `Pending` response message and starts processing contactless card. Maximum time of processing is not defined due to "Frame waiting time extension" mechanism of ISO14443-4 (for more information see Transceive ISO14443-4 command).

Before running this command make sure that the Initialization phase is done.

4.4.1.1 Request

Request contains `PerformTransaction` message:

Field Name	Label	Type	Description
poll for token	required	PollForToken	Parameters used in Poll For Token procedure
amount authorized	required	uint64	Whole Amount, Authorized (0x9F02) including fractional part ('cents') with maximum value '999999999999'
transaction date	required	bytes	Transaction Date (0x9A)
transaction type	required	bytes	Transaction Type (0x9C)
terminal country code	required	bytes	Terminal Country Code (0x9F1A)
transaction time	required	bytes	Transaction Time (0x9F21)
transaction currency code	required	bytes	Transaction Currency Code (0x5F2A)
mastercard amount	optional	uint64	DEPRECATED: use aid specific field
maestro amount	optional	uint64	DEPRECATED: use aid specific field
mir amount	optional	uint64	DEPRECATED: use aid specific field
troika amount	optional	uint64	DEPRECATED: use aid specific field
unionpay amount	optional	uint64	DEPRECATED: use aid specific field
aid specific	repeated	AidSpecificParams	Parameters desired for AID
merchant name and location	optional	bytes	Indicates the name and location of the merchant (0x9F4E)
merchant category code	optional	bytes	Classifies the type of business being done by the merchant (0x9F15)
terminal identification	optional	bytes	Designates the unique location of the Terminal (0x9F1C)
debug	optional	bool	Enable debug behavior
indication	optional	Indication	Indication (LEDs, buzzer) used during transaction

Table 4.62: PerformTransaction

`poll_for_token` field will be processed same as in `"Poll for token"` command. It is possible to run transaction right after successful `"Poll for token"` command by setting `poll_for_token.timeout = 0`.

Transaction Date (`0x9A`) field (`transaction_date`) contains local date and should be encoded according to EMV as follows: YYMMDD numeric 6, 3 bytes. Example: `301225h` for December 25th 2030. Note that the first day of the month as well as the first month of the year are `01`.

Transaction Type (`0x9C`) field (`transaction_type`) indicates the type of financial transaction, represented by the first two digits of the ISO 8583:1987 Processing Code. The actual values to be used for the Transaction Type data element are defined by the relevant payment system and should be encoded according to EMV as follows: numeric 2, 1 byte. Example: `00h`.

Terminal Country Code (`0x9F1A`) field (`terminal_country_code`) indicates the country of the terminal, represented according to ISO 3166, and should be encoded according to EMV as follows: numeric 3, 2 bytes. Example: `0643h` for Russia.

Transaction Time (`0x9F21`) field (`transaction_time`) contains local time that the transaction was authorised and should be encoded according to EMV as follows: HHMMSS numeric 6, 3 bytes. Example: `145534h` for 14:55:34.

Transaction Currency Code (`0x5F2A`) field (`transaction_currency_code`) indicates the currency code of the transaction according to ISO 4217 and should be encoded according to EMV as follows: numeric 3, 2 bytes. Example: `0643h` for Russian Ruble.

Merchant Name and Location (`0x9F4E`) field (`merchant_name_and_location`) indicates the name and location of the merchant. Example: `MosMetro. Kropotkinskaya` for Moscow Metro Kropotkinskaya station.

Merchant Category Code (`0x9F15`) field (`merchant_category_code`) classifies the type of business being done by the merchant, represented according to ISO 8583:1993 for Card Acceptor Business Code. Example: `4131h` for Bus Lines.

Following fields are available since particular protocol version:

Field	Avaliable since version
`token_carrier_type`	1.9
`encrypted_sensitive_data`	1.15
`indication`	1.37
`aid_specific`	1.41
`card_response_exceeds_threshold`	1.51

Following fields are supported but deprecated, **we strongly discourage you from use deprecated fields in new projects**:

Field	Deprecated since version
`mastercard_amount`	1.41
`maestro_amount`	1.41
`mir_amount`	1.41
`troika_amount`	1.41
`unionpay_amount`	1.41

AID specific parameters

Sometimes there it's necessary to use different transaction parameters depends on the card's Application ID (AID).

To achieve that use the `aid_specific` field.

Example:

- Requirement: use different amount and transaction type for Mastercard & Visa cards
- Implementation: add following messages to the `aid_specific` field:

```
AidSpecificParams {
    aids: ["\xA0\x00\x00\x00\x03"],
    amount_authorized: 123,
    transaction_type: "\x01",
}
AidSpecificParams {
    aids: ["\xA0\x00\x00\x00\x04"],
    amount_authorized: 456,
    transaction_type: "\x09",
}
```

If the card's AID starts with the value from the `aids` list then associated parameters will be used in the transaction.

4.4.1.2 Response

Response contains `TransactionResult` message:

Field Name	Label	Type	Description
status	required	TransactionStatus	Status of contactless transaction
token	required	Token	Level1 related information about card
emv tags	repeated	EmvTag	EMV tags related to transaction
last cmd	optional	bytes	Last command sent to the card

last sw bytes	optional	bytes	Status bytes (SW1 SW2) of last response of the card
error reason	optional	string	Describes reason due to unable perform transaction
token carrier type	optional	TokenCarrierType	Type of the token carrier
encrypted sensitive data	optional	bytes	SensitiveData message encrypted on the Controller's Public Key
pan hash	optional	bytes	Hashed value of the Application PAN
masked pan	optional	string	Masked value of the Application PAN
card response exceeds threshold	optional	bool	Whether card's response time exceeds the threshold defined by the configuration

Table 4.65: TransactionResult

Command can be finished with one the possible statuses:

Value	Number	Description
ONLINE AUTHORIZATION REQUIRED	0	Online authorization is required to finish transaction
OFFLINE APPROVED	1	Transaction is authorized offline
OFFLINE DECLINED	2	Transaction is declined offline
USE CONTACT INTERFACE	3	If it is possible use another (contact) interface
UNABLE PERFORM TRANSACTION	4	Unable to perform transaction, see error reason field
NON EMV CARD	5	Non-EMV (banking) card present to the reader
CARD EXPIRED	6	Transaction is declined offline due to card is expired
AUTHORIZATION ON CARDHOLDER DEVICE REQUIRED	7	Cardholder need to be authenticated on the their device

Table 4.66: TransactionStatus

The `pan_hash` and `masked_pan` fields are available since version 1.47.

NON_EMV_CARD status

Presented card is not EMV compatible. Most probably it is one of the transport (transit) cards.

UNABLE_PERFORM_TRANSACTION status

Unrecoverable error occurred during transaction. See `error_reason` field for human-readable description of the error. See `last_cmd` & `last_sw_bytes` fields for additional context data. Also you can investigate `emv_tags` fields to get the reason for the error.

Table 4.67: PerformTransaction Errors

Error	Description
PayPass: ... 48	Card supports only Magstripe mode, but reader supports only EMV mode
EntryPoint: 65	Reader is configured to deny zero amount, but Host sent zero amount in request
PayWave: ... 67	Offline Data Authentication (ODA) procedure has been failed
PayWave: ... 18	Card personalization error: redundant primitive tag is found

Other statuses

Other statuses mean that interaction with card was **successful**. `error_reason` field is absent in `TransactionResult` message. *Host* need takes actions according to certain status using EMV tags from `emv_tags` field.

Following tags will be returned in `emv_tags` fields if interaction with card was successful:

Tag Number	Tag Name
0x50	Application Label
0x57	Track 2 Equivalent Data
0x5A	Application Primary Account Number (PAN)
0x5F20	Cardholder Name
0x5F24	Application Expiration Date
0x5F28	Issuer Country Code
0x5F2A	Transaction Currency Code
0x5F34	Application Primary Account Number (PAN) Sequence Number
0x82	AIP
0x84	DF Name
0x8F	CA Public Key Index
0x95	TVR
0x96	Kernel Identifier-Terminal
0x9A	Transaction Date
0x9C	Transaction Type
0x9F02	Amount, Authorized (Numeric)
0x9F03	Amount, Other (Numeric)
0x9F06	Application Identifier (AID)
0x9F07	Application Usage Control
0x9F08	Application Version Number, Card
0x9F09	Application Version Number, Terminal
0x9F10	Issuer Application Data
0x9F12	Application Preferred Name
0x9F19	Token Requestor ID
0x9F1A	Terminal Country Code
0x9F1F	Track 1 Discretionary Data
0x9F21	Transaction Time
0x9F24	Payment Account Reference (PAR)
0x9F26	Application Cryptogram
0x9F27	Cryptogram Information Data
0x9F33	Terminal Capabilities, PayPass & UnionPay kernels only
0x9F34	Cardholder Verification Method (CVM) Results, Mir & PayPass kernels only
0x9F35	Terminal Type
0x9F36	Application Transaction Counter (ATC)
0x9F37	Unpredictable Number
0x9F4C	ICC Dynamic Number
0x9F66	TTQ, PayWave kernel only
0x9F6B	Track 2 Data
0x9F6C	CTQ, PayWave kernel only
0x9F6E	Third Party Data, PayPass, PayWave and JCB kernels only
0x9F70	CDA Result (CDAR), Mir kernel only
0x9F71	GDDOL Resulting Buffer, PURE kernel only
0x9F74	Data Elements Update Result, PURE kernel only
0x9F7C	Partner Discretionary Data, JCB only
0xBF0C	FCI Issuer Discretionary Data
0xC5	Contactless Cryptogram Information Data (CCID), PURE kernel only
0xC7	Terminal Transaction Processing Information (TTPI), PURE kernel only
0xDF3A	Card Feature Version Number, Discover kernel only
0xDF3B	Card Feature Descriptor, Discover kernel only
0xDF3D	Data Storage Directory, Discover kernel only
0xDF70	Application Info (AI), Mir kernel only
0xDF71	Transaction Processing Mode (TPM), Mir kernel only
0xDF74	Kernel Verification Results (KVR), Mir kernel only
0xDFA015	IntelliReader Proprietary tag
0xDFA50D	JCB Transaction Mode tag
0xDFA601	IntelliReader Proprietary tag
0xDFA602	Discover Container Records
0xDFA604	Discover Data Storage Write Results
0xDFA701	IntelliReader Proprietary tag

Tag Number	Tag Name
0xDFB101	Pure Kernel Capabilities
0xDFB107	Pure Kernel Offline CAM Selected
0xDFF001	Intellireader Proprietary tag: IntelliReader firmware version
0xDFF002	Intellireader Proprietary tag: `l2wave.json` checksum
0xDFF003	Intellireader Proprietary tag: `l2wave_ca.json` checksum
0xDFF004	Intellireader Proprietary tag: `l2wave_crl.json` checksum
0xDFF005	Intellireader Proprietary tag: `emvl2.json` checksum

4.4.1.3 Auxiliary messages

Message	Comments
Pending	Return
Notification	Same as `"Poll for token"` command.
Cancellation	Allow

4.4.1.4 Retries

It is quite often that communication between *IR* and token may be lost due to nature of the NFC technology. *IR* has detected token, but connection has been lost in the middle of transaction.

It may occur in the following cases:

- cardholder has placed card and immediately withdrawn it back
- card has been placed in some not suitable position
- buggy card that reboots during communication

In such cases *IR* restarts transaction: performs `poll_for_token` again and then repeats `PerformTransaction`.

IR repeats `PerformTransaction` five (5) times but no longer than 2 seconds. Number of attempts and repeat timeout cannot be configured and are subject to change without notice.

If retries have been failed then *IR* returns failure response with one of the `error`:

- `RETRY LIMIT IS OUT`
- `RETRY TIME IS OUT`

4.4.1.5 Sensitive Data Encryption

It is possible to encrypt cardholder's sensitive data that reader read from the card before that data will be transmitted to controller.

Reader encrypts data according to PKCS#1 v1.5.

To enable encryption controller need to:

- generate RSA 2048 bits key, that key should contain private and public parts of the key
- extract public part of the key in the PKCS#1 DER format
- put **public part** of the key as `emv.der` file into *the reader configuration*

To recover encrypted sensitive data:

- get `encrypted_sensitive_data` field from the `TransactionResult` message
- decrypt data of `encrypted_sensitive_data` field using controller's private key
- parse decrypted data using `SensitiveData` message

Following EMV tags are consist sensitive data:

Tag Number	Tag Name
0x57	Track 2 Equivalent Data
0x5A	Application Primary Account Number (PAN)
0x5F20	Cardholder Name
0x5F24	Application Expiration Date
0x5F30	Service Code

Tag Number	Tag Name
0x9F6B	Track 2 Data

4.4.1.6 PAN-hash computing

Reader can compute a PAN-hash value over the Application PAN value.

Supported algorithms:

- HMAC-SHA1
- HMAC-SHA256

The key with the `PAN_HASHING` usage should be injected into the Internal Secure Storage in advance.

Reader returns the computed PAN-hash in the `pan_hash` field and doesn't include the Sensitive Data into the `emv_tags` field but may include them into the `encrypted_sensitive_data` field.

4.4.1.7 Token Carrier Type

Reader is able to detect the following token carrier types by the information from the card:

Value	Number	Description
UNKNOWN	0	Reader unable to determinate token carrier type
PLASTIC CARD	1	Standard plastic card
MOBILE DEVICE	2	Mobile device with unknown wallet
GOOGLE PAY	3	Goolge Pay wallet
APPLE PAY	4	Apple Pay wallet
SAMSUNG PAY	5	Samsung Pay wallet
MIR PAY	6	MIR Pay wallet
HUAWEI PAY	7	Huawei Pay wallet
SBER PAY	8	Sber Pay wallet
TBANK PAY	9	TBank Pay wallet
GAZPROM PAY	10	Gazprom Pay wallet

Table 4.71: TokenCarrierType

`SBER_PAY` type is available since v. 1.31.

4.4.1.8 Expired tokens

According to `built-in contactless kernel settings` IntelliReader accepts both cards and tokens that are expired in March 2022 or later.

It's **recommended** that controller declines expired tokens by checking Application Expiration Date (`0x5F24`) or Track 2 Equivalent Data (`0x57`) tags.

4.4.1.9 Resistance to NFC Gate (Relay) attack

An NFC gate (relay) attack is a man-in-the-middle cyberattack where two devices wirelessly transmit commands/responses between a victim's card or phone and a reader, tricking the system into approving fraudulent transactions.

It is possible to detect such attacks by measuring the time between the request and the response.

This feature is not enabled by default because it can lead to false positives so it should be enabled explicitly in *the reader configuration*.

If the feature is enabled and the time between request and response exceeds the threshold defined in the reader configuration then `card_response_exceeds_threshold` field of the `TransactionResult` message will be set to `true`.

The field is absent if the feature is not enabled.

4.4.1.10 Example

Poll for token during 2 seconds and perform transaction with following parameters:

- Amount, Authorized: `1.00`
- Transaction Date: `130613`
- Transaction Time: `120000`
- Transaction Type: `00`
- Terminal Country Code: `643`
- Transaction Currency Code: `643`

```
perform_transaction
{
    poll_for_token { timeout: 2000 }
    amount_authorized: 100
    transaction_date: "\x13\x06\x13"
    transaction_time: "\x12\x00\x00"
    transaction_type: "\x00"
    terminal_country_code: "\x06\x43"
    transaction_currency_code: "\x06\x43"
}
```

4.4.2 Get EMV kernel information

This command allows to get EMV contactless kernel information.

Available since v. 1.53.

4.4.2.1 Request

Request contains `GetKernelInfo` empty message.

4.4.2.2 Response

IR answers with `KernelInfo` message:

Field Name	Label	Type	Description
kernels	repeated	Kernel	List of EMV kernels in the reader

Table 4.72: KernelInfo

Field Name	Label	Type	Description
name	required	string	Name of the EMV kernel
version	required	string	Version of the EMV kernel

Table 4.73: Kernel

4.4.2.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5 Mifare Cards module

Each message should contains *one of* the following message types:

Field Name	Label	Type	Description
mfr classic auth on clear key	optional	ClearKey	Authenticates Mifare Classic card's sector using clear key
mfr classic auth on sam key	optional	SamKey	Authenticates Mifare Classic card's sector using SAMAV2
mfr classic read blocks	optional	ReadBlocks	Reads data blocks from Mifare Classic card
mfr classic write blocks	optional	WriteBlocks	Writes data blocks to Mifare Classic card
mfr classic bulk operation	optional	BulkOperation	Combines several Mifare Classic commands into one command
mfr classic get counter	optional	GetCounter	Gets Mifare Classic counter block value
mfr classic set counter	optional	SetCounter	Sets Mifare Classic counter block value
mfr classic modify counter	optional	ModifyCounter	Modifies Mifare Classic counter block
mfr classic copy counter	optional	CopyCounter	Copies value of Mifare Classic counter from one block to another
mfr classic commit counter	optional	CommitCounter	Commits (writes) value from the internal data register to the block
mfr classic auth on internal key	optional	InternalKey	Authenticates Mifare Classic card's sector using key from internal secure storage
mfr plus auth on clear key	optional	ClearKey	Authenticates Mifare Plus card's sector using clear key
mfr plus read blocks	optional	ReadBlocks	Reads data blocks from Mifare Plus card
mfr plus write blocks	optional	WriteBlocks	Writes data blocks to Mifare Plus card
mfr plus bulk operation	optional	BulkOperation	Combines several Mifare Plus commands into one command
mfr plus get counter	optional	GetCounter	Gets Mifare Plus counter block value
mfr plus set counter	optional	SetCounter	Sets Mifare Plus counter block value
mfr plus modify counter	optional	ModifyCounter	Modifies Mifare Plus counter block
mfr plus copy counter	optional	CopyCounter	Copies value of Mifare Plus counter from one block to another
mfr plus commit counter	optional	CommitCounter	Commits (writes) value from the internal data register to the block
mfr plus auth on sam key	optional	SamKey	Authenticates Mifare Plus card's sector using NXP SAM AV2 module
mfr plus auth on internal key	optional	InternalKey	Authenticates Mifare Plus card's sector using key from internal secure storage
av2 authenticate host	optional	AuthenticateHost	Performs SAM AV2 Authenticate Host Command
av2 change keyentry	optional	ChangeKeyEntry	Creates/changes SAM AV2 KeyEntry
av2 unlock	optional	Unlock	Unlock SAM AV2 using mutual authentication
mfr ul read pages	optional	ReadPages	Read Mifare Ultralight pages
mfr ul write pages	optional	WritePages	Write pages to Mifare Ultralight
mfr ul get version	optional	GetVersion	Run Mifare Ultralight EV1 GET VERSION command
mfr ul get counter	optional	GetCounter	Read value of Mifare Ultralight EV1 built-in counter
mfr ul increment counter	optional	IncrementCounter	Increment Mifare Ultralight EV1 built-in counter
mfr ul bulk operation	optional	BulkOperation	Combines several Mifare Ultralight commands into one command
mfr ul auth on clear key	optional	ClearKey	Mifare Ultralight C 3DES Authentication using clear key
mfr ul auth on sam key	optional	SamKey	Mifare Ultralight C 3DES Authentication using SAM AV2
mfr ul auth clear password	optional	ClearPassword	Mifare Ultralight EV1 password based Authentication using clear key
mfr ul auth sam password	optional	SamPassword	Mifare Ultralight EV1 password based Authentication using NXP SAM AV3 module

mfr ul auth on internal key	optional	InternalKey	Mifare Ultralight C 3DES Authentication using key from internal secure storage

Table 4.75: Mifare

Module ID: **05h**.

4.5.1 Mifare Cards Authentication Parameters

Mifare Classic card can be authenticated by the Mifare sector key (A or B):

Value	Number	Description
TYPE A	0	The A key
TYPE B	1	The B key

Table 4.76: KeyType

Mifare Plus card can be authenticated by the `Sector` or `Special` key:

Field Name	Label	Type	Description
special key	optional	SpecialKey	Key to get access to non Mifare (special) blocks
sector key	optional	SectorKey	Key to get access to regular Mifare blocks

Table 4.77: KeyType

To authenticate by the `Sector` key controller should choose sector number and type of the Mifare sector key (A or B):

Field Name	Label	Type	Description
sector number	required	uint32	Number of Mifare Plus sector
sector key type	required	SectorKeyType	Type of Mifare Plus sector key

Table 4.78: SectorKey

Value	Number	Description
TYPE A	0	The A key
TYPE B	1	The B key

Table 4.79: SectorKeyType

To authenticate by the `Special` key controller should choose one of following keys:

Value	Number	Description
ORIGINALITY KEY	0	Originality Key
CARD MASTER KEY	1	Card Master Key
CARD CONFIG KEY	2	Card Configuration Key
LEVEL2 SWITCH KEY	3	Level 2 Switch Key
LEVEL3 SWITCH KEY	4	Level 3 Switch Key
SL1 CARD AUTH KEY	5	SL1 Card Authentication Key

Table 4.80: SpecialKey

4.5.2 Mifare Classic/Plus memory map

- Sectors up to `32` contain four (`4`) blocks in each sector
- Sectors after `32` contain sixteen (`16`) blocks in each sector
- Mifare Plus cards have special blocks

4.5.3 Mifare Classic Auto-Recovery Feature

This feature allows iterate multiple Mifare keys in the significantly faster manner.

In case of error Mifare Classic card usually does not answer to the following requests. It is special protection behaviour against the brute force attack.

But the reader implements a feature name “auto-recovery” that re-select Mifare Classic card after an error, so it is possible to quickly repeat an authentication with other key. The same feature is applicable for errors in the other Mifare Classic commands.

Example.

To check four (4) keys against one Mifare Classic card we need to execute four (4) “Mifare Classic Authentication” commands in the row. First successfully executed command gives us the right key.

4.5.4 NXP SAM AV3 auto-reset

There is a known issue with the NXP SAM AV3 module when it can be permanently locked after a certain number of commands.

To prevent that issue the reader implements the auto-reset feature that automatically resets the SAM AV3 module when the command counter exceeds **60000**. The auto-reset feature repeats Host Authentication and Unlock commands if they were done early.

You can prevent the auto-reset by resetting the SAM AV3 module by yourself before the command counter threshold is reached.

4.5.5 Mifare Classic Authentication on Clear Key

This command can be issued after successfully processed `Poll for token` command. If `clear_key` is not match card's key then the reader returns `Failure response` with `MFC_AUTHENTICATION_ERROR` error.

Also see Mifare Classic Auto-Recovery Feature.

4.5.5.1 Request

Request contains `ClearKey` message:

Field Name	Label	Type	Description
sector number	required	uint32	Number of Mifare Classic sector
key type	required	KeyType	Specifies which type of key to use for authentication
clear key	required	bytes	Value of the Key (6 bytes)

Table 4.81: ClearKey

4.5.5.2 Response

No data carried with successful response.

4.5.5.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.5.4 Example

Request Mifare Classic Authentication to sector `0` on Clear (Plain-text) key `A` with value `FFFFFFFFFFFF`:

```
mfr_classic_auth_on_clear_key {
    sector_number: 0
    key_type: TYPE_A
    clear_key: "\xff\xff\xff\xff\xff\xff"
}
```

4.5.6 Mifare Classic Authentication on Key from SAMAV2

This command can be issued after successfully processed `Poll for token` command.

If key within SAMAV2 is not match card's key then reader returns `Failure response` with `SAM_AV2_AUTHENTICATION_ERROR` error.

Also see Mifare Classic Auto-Recovery Feature.

4.5.6.1 Request

Request contains `SamKey` message:

Field Name	Label	Type	Description
sector number	required	uint32	Number of Mifare Classic sector
key type	required	KeyType	Specifies which type of key to use for authentication
av2 args	required	AuthenticationArguments	Specifies SAM AV2 arguments that will be used in authentication
need diversification	optional	bool	Defines whether Crypto1 key diversification is required

Table 4.83: SamKey

4.5.6.2 Response

No data carried with successful response.

4.5.6.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.6.4 Example

Request Mifare Classic Authentication with following arguments:

- `Mifare Classic` card:
- – the first (`1`) sector
- – the `A` key
- `Mifare SAM AV2` module in `SAM3` slot:
- – the third (`3`) key entry
- – the key version #0

```
mfr_classic_auth_on_sam_key {
    sector_number: 1
    key_type: TYPE_A
    av2_args {
        slot: SAM3_SLOT
        key_number: 3
    }
}
```

4.5.7 Mifare Classic Authentication on Internal Key

This command can be issued after successfully processed `Poll for token` command.

If key in the `Internal Storage` is not match card's key then the reader returns `Failure response` with `MFC_AUTHENTICATION_ERROR` error.

Also see Mifare Classic Auto-Recovery Feature.

Available from version 1.38.

4.5.7.1 Request

Request contains `InternalKey` message:

Field Name	Label	Type	Description
sector number	required	uint32	Number of Mifare Classic sector
key type	required	KeyType	Specifies which type of key to use for authentication
key index	required	uint32	Index of Mifare Classic key within internal storage
samav2	optional	TripleDESDiversification	Diversification method that uses Triple-DES key (the same algorithm as used in SAM AV2 diversification)

Table 4.85: InternalKey

4.5.7.2 Response

No data carried with successful response.

4.5.7.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.7.4 Example

Request Mifare Classic Authentication to sector `0` with key `A` using internal key at index `0`:

```
mfr_classic_auth_on_internal_key {
    sector_number: 0
    key_type: TYPE_A
    key_index: 0
}
```

4.5.8 Mifare Classic Read Data Blocks

This command allows to read multiple blocks per one command.

This command can be issued after successfully processed `Mifare Classic Authentication`: `On Clear Key` or `On SAMAV2 Key`.

4.5.8.1 Request

Request contains `ReadBlocks` message:

Field Name	Label	Type	Description
start block	required	uint32	Number of the first block within sector (possible values 0-3)
block count	optional	uint32	Count of blocks to be read

Table 4.87: ReadBlocks

`start_block` field defines number of the block **within** sector previously authenticated by `On Clear Key` or `On SAMAV2 Key` commands. Values allowed for `start_block` field are `0..3` for 1K cards and `0..15` for 4K cards.

4.5.8.2 Response

Response contains `Blocks` message:

Field Name	Label	Type	Description
data	required	bytes	Block(s) data

Table 4.88: Blocks

Size of `data` field `:=` `block_count` * 16 bytes.

4.5.8.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.8.4 Example

Request to read three (`3`) Mifare Classic blocks starting off zero (`0`) block of sector:

```
mfr_classic_read_blocks { start_block: 0 block_count: 3 }
```

4.5.9 Mifare Classic Write Data Blocks

This command allows to write multiple blocks per one command.

This command can be issued after successfully processed `Mifare Classic Authentication`: `On Clear Key` or `On SAMAV2 Key`.

4.5.9.1 Request

Request contains `WriteBlocks` message:

Field Name	Label	Type	Description
start block	required	uint32	Number of the first block within sector (possible values 0-3)
data	required	bytes	Data to be written into Block(s)

Table 4.90: WriteBlocks

`start_block` field defines number of the block **within** sector previously authenticated by `On Clear Key` or `On SAMAV2 Key` commands. Values allowed for `start_block` field are `0..3` for 1K cards and `0..15` for 4K cards.

Size of `data` field **must be multiple of 16 bytes**. Counts of blocks to be written defined as `sizeof( data ) / 16`.

4.5.9.2 Response

No data carried with successful response.

4.5.9.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.9.4 Example

Request to write two (2) Mifare Classic blocks starting off first (1) block of sector with values `00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F` and `10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F`:

```
mfr_classic_write_blocks {
    start_block: 1
    data: "\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f
    \x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f"
}
```

4.5.10 Mifare Classic Get Counter Value

Returns value of counter block as a 32-bit signed integer. Returns error if block type is not counter.

This command can be issued after successfully processed `Mifare Classic Authentication`: `On Clear Key` or `On SAMAV2 Key`.

4.5.10.1 Request

Request contains `GetCounter` message:

Field Name	Label	Type	Description
src block	required	uint32	Source block within sector

Table 4.92: GetCounter

`src_block` field defines number of the block **within** sector previously authenticated by `On Clear Key` or `On SAMAV2 Key` commands. Values allowed for `src_block` field are `0..2` for 1K cards and `0..14` for 4K cards.

4.5.10.2 Response

Response contains `Counter` message:

Field Name	Label	Type	Description
value	required	int32	Value of counter

Table 4.93: Counter

4.5.10.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.10.4 Example

Get counter value from the first (`1`) block of sector:

```
mfr_classic_get_counter { src_block: 1 }
```

4.5.11 Mifare Classic Set Counter Value

Set value of counter block to required value. If block type is not counter then initialize it accordingly.

This command can be issued after successfully processed `Mifare Classic Authentication`: `On Clear Key` or `On SAMAV2 Key`.

4.5.11.1 Request

Request contains `SetCounter` message:

Field Name	Label	Type	Description
dst block	required	uint32	Destination block within sector
value	required	int32	Value of counter

Table 4.95: SetCounter

`dst_block` field defines number of the block **within** sector previously authenticated by `On Clear Key` or `On SAMAV2 Key` commands. Values allowed for `dst_block` field are `0..2` for 1K cards and `0..14` for 4K cards.

4.5.11.2 Response

No data carried with successful response.

4.5.11.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.11.4 Example

Set first (`1`) block of sector to value `0`:

```
mfr_classic_set_counter { dst_block: 1 value: 0 }
```

4.5.12 Mifare Classic Modify Counter Value

Modifies (increments or decrements) value of the counter block. Returns error if block type is not counter.

This command can be issued after successfully processed `Mifare Classic Authentication`: `On Clear Key` or `On SAMAV2 Key`.

4.5.12.1 Request

Request contains `ModifyCounter` message:

Field Name	Label	Type	Description
src block	required	uint32	Source block within sector
operand	required	int32	Modification operand If value is positive then increment otherwise decrement original value
dst block	optional	uint32	Destination block within sector If absent, store the result in the internal data register

Table 4.97: ModifyCounter

`src_block` and `dst_block` fields define number of the block **within** sector previously authenticated by `On Clear Key` or `On SAMAV2 Key` commands. Values allowed for `src_block` and `dst_block` field are `0..2` for 1K cards and `0..14` for 4K cards.

Sign of `operand` field defines type of operation:

- negative `operand` value => decrement operation
- positive `operand` value => increment operation

`dst_block` is optional field. If `dst_block` is absent then result of operation will be stored within internal register in the Mifare Classic card. Value of the internal register can be written to the block by `CommitCounter` command.

4.5.12.2 Response

No data carried with successful response.

4.5.12.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.12.4 Example

Increment value of first (`1`) block of sector by `1` and store result to the first (`1`) block:

```
mfr_classic_modify_counter { src_block: 1 operand: 1 dst_block: 1 }
```

4.5.13 Mifare Classic Copy Counter Value

Copies value from counter block to another block or card's internal register. Returns error if block type is not counter.

This command can be issued after successfully processed `Mifare Classic Authentication`: `On Clear Key` or `On SAMAV2 Key`.

4.5.13.1 Request

Request contains `CopyCounter` message:

Field Name	Label	Type	Description
src block	required	uint32	Source block within sector
dst block	optional	uint32	Destination block within sector If absent, copy to the internal data register

Table 4.99: CopyCounter

`src_block` and `dst_block` fields define number of the block **within** sector previously authenticated by `On Clear Key` or `On SAMAV2 Key` commands. Values allowed for `src_block` and `dst_block` field are `0..2` for 1K cards and `0..14` for 4K cards.

`dst_block` is optional field. If `dst_block` is absent then `src_block` value will be stored within internal register in the Mifare Classic card. Value of the internal register can be written to the block by `CommitCounter` command.

4.5.13.2 Response

No data carried with successful response.

4.5.13.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.13.4 Example

Copies the first (`1`) block of sector to the second (`2`) block of the same sector:

```
mfr_classic_copy_counter { src_block: 1 dst_block: 2 }
```

4.5.14 Mifare Classic Commit Counter Value

Writes value from card's internal register to the counter block. Returns error if block type is not counter.

This command can be issued after successfully processed `Mifare Classic Authentication`: `On Clear Key` or `On SAMAV2 Key`.

4.5.14.1 Request

Request contains `CommitCounter` message:

Field Name	Label	Type	Description
dst block	required	uint32	Destination block within sector

Table 4.101: CommitCounter

`dst_block` field defines number of the block **within** sector previously authenticated by `On Clear Key` or `On SAMAV2 Key` commands. Values allowed for `dst_block` field are `0..2` for 1K cards and `0..14` for 4K cards.

4.5.14.2 Response

No data carried with successful response.

4.5.14.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.14.4 Example

Writes value from card's internal register to the second (2) block of the sector:

```
mfr_classic_commit_counter { dst_block: 2 }
```

4.5.15 Mifare Classic Bulk operation

This command allows to combine several commands to be processed as single request to the reader, that usually processed as separated requests.

The reader internally processes combined commands on one-by-one approach. If error is occurred then reader stops processing, clears all results from already processed commands and returns this error in `Failure response`.

In case of error the `mifare_error_at` field of the `Failure response` contains a sector & block number where the error is occured.

Format of the commands and their responses the same as in separated commands.

4.5.15.1 Request

Request contains `BulkOperation` message:

Field Name	Label	Type	Description
operations	repeated	Command	Sequence of the Mifare Classic commands

Table 4.103: BulkOperation

The reader processing following `Command`s one-by-one in order specified in `operations` field:

Field Name	Label	Type	Description
auth on clear key	optional	ClearKey	Authenticates to Mifare Classic sector using clear key
auth on sam key	optional	SamKey	Authenticates to Mifare Classic sector using NXP SAM AV2 module
read blocks	optional	ReadBlocks	Read blocks from Mifare Classic card
write blocks	optional	WriteBlocks	Write blocks to Mifare Classic card
get counter	optional	GetCounter	Gets Mifare Classic counter block value
set counter	optional	SetCounter	Sets Mifare Classic counter block value
modify counter	optional	ModifyCounter	Modifies Mifare Classic counter block
copy counter	optional	CopyCounter	Copies value of Mifare Classic counter from one block to another
commit counter	optional	CommitCounter	Commits (writes) value from the internal data register to the block
auth on internal key	optional	InternalKey	Authenticates to Mifare Classic sector using internal key

Table 4.104: Command

4.5.15.2 Response

Response contains `BulkResult` message:

Field Name	Label	Type	Description
results	repeated	CommandResult	Sequence of the results of Mifare Classic commands

Table 4.105: BulkResult

The reader collects results in `results` field in order specified in `operations` field of the *Request*:

Field Name	Label	Type	Description
read blocks	optional	Blocks	Response to ReadBlocks command
get counter	optional	Counter	Response to GetCounter command

Table 4.106: CommandResult

The reader does not collect results for commands that have *empty* response on success, such as `Write Data Blocks`.

4.5.15.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.15.4 Example

Combine commands in the following order:

1. Mifare Classic Authentication to sector `0` on Clear (Plain-text) key `A` with value `FFFFFFFFFFFF`
2. Write first (`1`) Mifare Classic block of the sector with value `00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F`
3. Read first (`1`) Mifare Classic block of the sector

```
mfr_classic_bulk_operation {
    operations {
        auth_on_clear_key {
            sector_number: 0
            key_type: TYPE_A
            clear_key: "\xff\xff\xff\xff\xff\xff"
        }
    }
    operations {
        write_blocks {
            start_block: 1
            data: "\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f"
        }
    }
    operations {
        read_blocks {
            start_block: 1
            block_count: 1
        }
    }
}
```

4.5.16 Mifare Plus Authentication on Clear Key

This command can be issued after successfully processed `Poll for token` command. If `clear_key` is not match card's key then reader returns `Failure response` with `MFP_AUTHENTICATION_ERROR` error.

4.5.16.1 Request

Request contains `ClearKey` message:

Field Name	Label	Type	Description
key type	required	KeyType	Specifies which type of key to use for authentication
clear key	required	bytes	Value of the Key (16 bytes)
security level	optional	SecurityLevel	Specifies which security level to use for authentication
diversification input	optional	bytes	AES128 key diversification input data if required

Table 4.108: ClearKey

4.5.16.2 Response

No data carried with successful response.

4.5.16.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.16.4 Example

Request Mifare Plus Authentication to sector `0` on Clear (Plain-text) key `A` with value `FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF`:

```
mfr_plus_auth_on_clear_key {
    key_type {
        sector_key {
            sector_number: 0
            sector_key_type: TYPE_A
        }
    }
    clear_key: "\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff"
}
```

4.5.17 Mifare Plus Authentication on SAM AV2

This command can be issued after successfully processed `Poll for token` command. If key in the SAM AV2 is not match card's key then reader returns `Failure response` with `MFP_AUTHENTICATION_ERROR` error.

4.5.17.1 Request

Request contains `SamKey` message:

Field Name	Label	Type	Description
key type	required	KeyType	Specifies which type of key to use for authentication
av2 args	required	AuthenticationArguments	Specifies SAM AV2 arguments that will be used in authentication
security level	optional	SecurityLevel	Specifies which security level to use for authentication
diversification input	optional	bytes	AES128 key diversification input data if required

Table 4.110: SamKey

4.5.17.2 Response

No data carried with successful response.

4.5.17.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.17.4 Example

Request Mifare Plus Authentication to sector `0` on SAM AV2 key entry #25 in `SAM3_SLOT` slot:

```
mfr_plus_auth_on_sam_key {
    key_type {
        sector_key {
            sector_number: 0
            sector_key_type: TYPE_A
        }
    }
    av2_args {
        slot: SAM3_SLOT
        key_number: 25
    }
}
```

4.5.18 Mifare Plus Authentication on Internal Key

This command can be issued after successfully processed `Poll for token` command.

If key from the `Internal Storage` is not match card's key then reader returns `Failure response` with `MFP_AUTHENTICATION_ERROR` error.

4.5.18.1 Request

Request contains `InternalKey` message:

Field Name	Label	Type	Description
key type	required	KeyType	Specifies which type of key to use for authentication
key index	required	uint32	Index of the key within internal storage
security level	optional	SecurityLevel	Specifies which security level to use for authentication
diversification input	optional	bytes	AES128 key diversification input data if required

Table 4.112: InternalKey

Available from version 1.38.

4.5.18.2 Response

No data carried with successful response.

4.5.18.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.18.4 Example

Request Mifare Plus Authentication to sector `0` on key `A` using key from internal storage at index `2`:

```
mfr_plus_auth_on_internal_key {
    key_type {
        sector_key {
            sector_number: 0
            sector_key_type: TYPE_A
        }
    }
    key_index: 2
}
```

4.5.19 Mifare Plus Read Data Blocks

This command allows to read multiple blocks (up to 16) per one command.

This command can be issued after successfully processed `Mifare Plus Authentication`:

- `On Clear Key`
- `On SAM AV2`

4.5.19.1 Request

Request contains `ReadBlocks` message:

Field Name	Label	Type	Description
start block	required	uint32	Number of the first block within sector
block count	optional	uint32	Count of blocks to be read (up to 16)

Table 4.114: ReadBlocks

`start_block` field defines number of the block **within** sector previously authenticated by `On Clear Key` command. Values allowed for `start_block` field are `0..3` for sectors up to `32` and `0..15` for sectors after `32`.

4.5.19.2 Response

Response contains `Blocks` message:

Field Name	Label	Type	Description
data	required	bytes	Block(s) data

Table 4.115: Blocks

Size of `data` field `:=` `block_count` * 16 bytes.

4.5.19.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.19.4 Example

Request to read three (`3`) Mifare Plus blocks starting off zero (`0`) block of sector:

```
mfr_plus_read_blocks { start_block: 0 block_count: 3 }
```

4.5.20 Mifare Plus Write Data Blocks

This command allows to write multiple blocks (up to 3) per one command.

This command can be issued after successfully processed `Mifare Plus Authentication`:

- `On Clear Key`
- `On SAM AV2`

4.5.20.1 Request

Request contains `WriteBlocks` message:

Field Name	Label	Type	Description
start block	required	uint32	Number of the first block within sector
data	required	bytes	Data to be written into Block(s)

Table 4.117: WriteBlocks

`start_block` field defines number of the block **within** sector previously authenticated by `On Clear Key` command. Values allowed for `start_block` field are `0..3` for sectors up to `32` and `0..15` for sectors after `32`.

Size of `data` field **must be multiple of 16 bytes**. Counts of blocks to be written defined as `sizeof( data ) / 16`.

4.5.20.2 Response

No data carried with successful response.

4.5.20.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.20.4 Example

See the `Mifare Classic Write Data Blocks` command for the `WriteBlocks` definition. Request to write two (2) Mifare Plus blocks starting off first (1) block of sector with values `00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F` and `10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F`:

```
mfr_plus_write_blocks {
    start_block: 1
    data: "\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f
    \x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f"
}
```

4.5.21 Mifare Plus Get Counter Value

Returns value of counter block as a 32-bit signed integer. Returns error if block type is not counter.

This command can be issued after successfully processed `Mifare Plus Authentication`:

- `On Clear Key`
- `On SAM AV2`

4.5.21.1 Request

Request contains `GetCounter` message:

Field Name	Label	Type	Description
src block	required	uint32	Source block within sector

Table 4.119: GetCounter

`src_block` field defines number of the block **within** sector previously authenticated by `On Clear Key` command. Values allowed for `src_block` field are `0..2` for sectors up to `32` and `0..14` for sectors after `32`.

4.5.21.2 Response

Response contains `Counter` message:

Field Name	Label	Type	Description
value	required	int32	Value of counter

Table 4.120: Counter

4.5.21.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.21.4 Example

Get counter value from the first (`1`) block of sector:

```
mfr_plus_get_counter { src_block: 1 }
```

4.5.22 Mifare Plus Set Counter Value

Set value of counter block to required value. If block type is not counter then initialize it accordingly.

This command can be issued after successfully processed `Mifare Plus Authentication`:

- `On Clear Key`
- `On SAM AV2`

4.5.22.1 Request

Request contains `SetCounter` message:

Field Name	Label	Type	Description
dst block	required	uint32	Destination block within sector
value	required	int32	Value of counter

Table 4.122: SetCounter

`dst_block` field defines number of the block **within** sector previously authenticated by `On Clear Key` command. Values allowed for `dst_block` field are `0..2` for sectors up to `32` and `0..14` for sectors after `32`.

4.5.22.2 Response

No data carried with successful response.

4.5.22.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.22.4 Example

Set first (`1`) block of sector to value `0`:

```
mfr_plus_set_counter { dst_block: 1 value: 0 }
```

4.5.23 Mifare Plus Modify Counter Value

Modifies (increments or decrements) value of the counter block. Returns error if block type is not counter.

This command can be issued after successfully processed `Mifare Plus Authentication`:

- `On Clear Key`
- `On SAM AV2`

4.5.23.1 Request

Request contains `ModifyCounter` message:

Field Name	Label	Type	Description
src block	required	uint32	Source block within sector
operand	required	int32	Modification operand If value is positive then increment otherwise decrement original value
dst block	optional	uint32	Destination block within sector If absent, store the result in the internal data register

Table 4.124: ModifyCounter

`src_block` and `dst_block` fields define number of the block **within** sector previously authenticated by `On Clear Key` command. Values allowed for `src_block` and `dst_block` field are `0..2` for sectors up to `32` and `0..14` for sectors after `32`.

Sign of `operand` field defines type of operation:

- negative `operand` value => decrement operation
- positive `operand` value => increment operation

`dst_block` is optional field. If `dst_block` is absent then result of operation will be stored within internal register in the Mifare Plus card. Value of the internal register can be written to the block by `CommitCounter` command.

4.5.23.2 Response

No data carried with successful response.

4.5.23.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.23.4 Example

Increment value of first (`1`) block of sector by `1` and store result to the first (`1`) block:

```
mfr_plus_modify_counter { src_block: 1 operand: 1 dst_block: 1 }
```

4.5.24 Mifare Plus Copy Counter Value

Copies value from counter block to another block or card's internal register. Returns error if block type is not counter.

This command can be issued after successfully processed `Mifare Plus Authentication`:

- `On Clear Key`
- `On SAM AV2`

4.5.24.1 Request

Request contains `CopyCounter` message:

Field Name	Label	Type	Description
src block	required	uint32	Source block within sector
dst block	optional	uint32	Destination block within sector If absent, copy to the internal data register

Table 4.126: CopyCounter

`src_block` and `dst_block` fields define number of the block **within** sector previously authenticated by `On Clear Key` command. Values allowed for `src_block` and `dst_block` field are `0..2` for sectors up to `32` and `0..14` for sectors after `32`.

`dst_block` is optional field. If `dst_block` is absent then `src_block` value will be stored within internal register in the Mifare Plus card. Value of the internal register can be written to the block by `CommitCounter` command.

4.5.24.2 Response

No data carried with successful response.

4.5.24.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.24.4 Example

Copies the first (`1`) block of sector to the second (`2`) block of the same sector:

```
mfr_plus_copy_counter { src_block: 1 dst_block: 2 }
```

4.5.25 Mifare Plus Commit Counter Value

Writes value from card's internal register to the counter block. Returns error if block type is not counter.

This command can be issued after successfully processed `Mifare Plus Authentication`:

- `On Clear Key`
- `On SAM AV2`

4.5.25.1 Request

Request contains `CommitCounter` message:

Field Name	Label	Type	Description
dst block	required	uint32	Destination block within sector

Table 4.128: CommitCounter

`dst_block` field defines number of the block **within** sector previously authenticated by `On Clear Key` command. Values allowed for `dst_block` field are `0..2` for sectors up to `32` and `0..14` for sectors after `32`.

4.5.25.2 Response

No data carried with successful response.

4.5.25.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.25.4 Example

Writes value from card's internal register to the second (`2`) block of the sector:

```
mfr_plus_commit_counter { dst_block: 2 }
```

4.5.26 Mifare Plus Bulk operation

This command allows to combine several commands to be processed as single request to the reader, that usually processed as separated requests.

The reader internally processes combined commands on one-by-one approach. If error is occurred then reader stops processing, clears all results from already processed commands and returns this error in `Failure response`.

Format of the commands and their responses the same as in separated commands.

4.5.26.1 Request

Request contains `BulkOperation` message:

Field Name	Label	Type	Description
operations	repeated	Command	Sequence of the Mifare Plus commands

Table 4.130: BulkOperation

The reader processing following `Command`s one-by-one in order specified in `operations` field:

Field Name	Label	Type	Description
auth on clear key	optional	ClearKey	Authenticates to Mifare Plus card using clear key
read blocks	optional	ReadBlocks	Read blocks from Mifare Plus card
write blocks	optional	WriteBlocks	Write blocks to Mifare Plus card
get counter	optional	GetCounter	Gets Mifare Plus counter block value
set counter	optional	SetCounter	Sets Mifare Plus counter block value
modify counter	optional	ModifyCounter	Modifies Mifare Plus counter block
copy counter	optional	CopyCounter	Copies value of Mifare Plus counter from one block to another
commit counter	optional	CommitCounter	Commits (writes) value from the internal data register to the block
auth on sam key	optional	SamKey	Authenticates to Mifare Plus card using key in SAM AV2
auth on internal key	optional	InternalKey	Authenticates to Mifare Plus card using key from internal secure storage

Table 4.131: Command

4.5.26.2 Response

Response contains `BulkResult` message:

Field Name	Label	Type	Description
results	repeated	CommandResult	Sequence of the results of Mifare Plus commands

Table 4.132: BulkResult

The reader collects results in `results` field in order specified in `operations` field of the *Request*:

Field Name	Label	Type	Description
read blocks	optional	Blocks	Response to ReadBlocks command
get counter	optional	Counter	Response to GetCounter command

Table 4.133: CommandResult

The reader does not collect results for commands that have *empty* response on success, such as `Write Data Blocks`.

4.5.26.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.26.4 Example

Combine commands in the following order:

1. Mifare Plus Authentication to sector `0` on Clear (Plain-text) key `A` with value `FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF`
2. Write first (`1`) Mifare Plus block of the sector with value `00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F`
3. Read first (`1`) Mifare Plus block of the sector

```
mfr_plus_bulk_operation {
    operations {
        auth_on_clear_key {
            key_type {
                sector_key {
                    sector_number: 0
                    sector_key_type: TYPE_A
                }
            }
            clear_key: "\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff"
        }
    }
    operations {
        write_blocks {
        start_block: 1
        data: "\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f"
        }
    }
    operations {
        read_blocks {
            start_block: 1
            block_count: 1
        }
    }
}
```

4.5.27 Mifare SAM AV2 Host Authentication

Mifare SAM AV2 Host Authentication usually used to unlock SAM AV2 module for certain crypto operations and to protect channel between IntelliReader and SAM AV2 itself by session keys.

For more information see “MIFARE SAM AV2 Product data sheet” from NXP.

Before execute this command SAM AV2 must be successfully powered by the `Power on` command.

4.5.27.1 Request

Request contains `AuthenticateHost` message:

Field Name	Label	Type	Description
args	required	AuthenticationArguments	Specifies SAM AV2 arguments that will be used in authentication
key	optional	bytes	AES128 key to be used in authentication

Table 4.135: AuthenticateHost

If the `key` field is not provided then *default* key value (all zeroes) is used.

4.5.27.2 Response

No data carried with successful response.

4.5.27.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.27.4 Example

Perform “Host Authentication” on SAM AV2 in `SAM3_SLOT`. Use *default* Master Key that placed in the key entry #0:

```
av2_authenticate_host { args { slot: SAM3_SLOT key_number: 0 } }
```

4.5.28 Unlock Mifare SAM AV2

Unlock Mifare SAM AV2 using mutual authentication.

For more information see “MIFARE SAM AV2 Product data sheet” from NXP.

Before execute this command SAM AV2 must be successfully powered by the `Power on` command.

4.5.28.1 Request

Request contains `Unlock` message:

Field Name	Label	Type	Description
args	required	AuthenticationArguments	Specifies SAM AV2 arguments that will be used in authentication
key	optional	bytes	AES128 key to be used in authentication

Table 4.137: Unlock

If the `key` field is not provided then *default* key value (all zeroes) is used.

4.5.28.2 Response

No data carried with successful response.

4.5.28.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.28.4 Example

Unlock SAM AV2 in `SAM2_SLOT`:

```
av2_unlock {
    args {
        slot: SAM2_SLOT
        key_number: 1
    }
    key: "\xe5\x71\x80\x0c\x72\x30\x99\x60\x35\x04\x23\x9a\x26\xc4\xb6\x41"
}
```

4.5.29 Change Key Entry in Mifare SAM AV2

This command allows to change (or create) the `KeyEntry` in Mifare SAM AV2.

`Mifare SAM AV2 Host Authentication` is required to be done before issue this command.

4.5.29.1 Request

Request contains `ChangeKeyEntry` message:

Field Name	Label	Type	Description
slot	required	CardSlot	Card slot in the reader
channel	optional	Channel	Number of SAM AV2 Logical Channel
keyentry number	required	uint32	Number of the key entry (0-127)
key version	required	uint32	Version of the key within the key entry (0-127)
key position	optional	KeyPosition	Position of the key within the key entry
mifare	optional	MifareKeyEntry	Key Entry for Mifare Classic authentication
plus	optional	PlusKeyEntry	Key Entry for Mifare Plus Authtication
ultralight c	optional	UltralightCKeyEntry	Key Entry for Mifare Ultralight C Authtication

Table 4.139: ChangeKeyEntry

Currently this command allows to change the following Key Entries:

Field Name	Label	Type	Description
key a	required	MifareKey	MIFARE key A
key b	required	MifareKey	MIFARE key B

Table 4.140: MifareKeyEntry

Field Name	Label	Type	Description
key	required	bytes	AES128 key (16 bytes)

Table 4.141: PlusKeyEntry

Field Name	Label	Type	Description
key	required	bytes	2TDEA key (16 bytes)

Table 4.142: UltralightCKeyEntry

`KeyEntry` #0 is Master Key. Currently it is not possible to change it by the command.

4.5.29.2 Response

No data carried with successful response.

4.5.29.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.29.4 Example

Change `KeyEntry` #10 to the MIFARE keys type and set both A/B keys to the `[0xff; 6]` value:

```
av2_change_keyentry {
    slot: SAM3_SLOT
    keyentry_number: 10
    key_version: 0
    mifare {
        key_a { key: "\xff\xff\xff\xff\xff\xff" }
        key_b { key: "\xff\xff\xff\xff\xff\xff" }
    }
}
```

4.5.30 Mifare Ultralight Read Pages

This command reads multiple pages out of the Mifare Ultralight card. For the original Ultralight (and "C" version) it is possible to read up to 4 pages at once. For the Ultralight EV1 it is possible to read all available pages.

Consider "Mifare Ultralight Bulk operation" for read more than 4 pages at single command from original Mifare Ultralight card.

4.5.30.1 Request

Request contains `ReadPages` message:

Field Name	Label	Type	Description
start address	required	uint32	Start page address
page count	required	uint32	Page count to read

Table 4.144: ReadPages

4.5.30.2 Response

Response contains `Blocks` message:

Field Name	Label	Type	Description
data	required	bytes	Pages data

Table 4.145: Pages

4.5.30.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.30.4 Example

Read four (4) pages starting from address `04h`:

```
mfr_ul_read_pages { start_address: 4 page_count: 4 }
```

4.5.31 Mifare Ultralight Write Pages

This command writes multiple pages to the Mifare Ultralight card. It is possible to write whole memory by the single command.

4.5.31.1 Request

Request contains `WritePages` message:

Field Name	Label	Type	Description
start address	required	uint32	First page address
data	required	bytes	Data to be written

Table 4.147: WritePages

If `data` field is not multiple of 4 bytes then it will be padded automatically by zeroes.

4.5.31.2 Response

No data carried with successful response.

4.5.31.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.31.4 Example

Write eight (8) pages (32 bytes) starting from address `04h`:

```
mfr_ul_write_pages {
    start_address: 4
    data: "\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f
    \x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f"
}
```

4.5.32 Mifare Ultralight EV1 Get Counter Value

This command supported only by the Mifare Ultralight EV1 card. The command returns value of the one of the EV1 counters as a 32-bit unsigned integer.

4.5.32.1 Request

Request contains `GetCounter` message:

Field Name	Label	Type	Description
counter number	required	CounterNumber	Counter number

Table 4.149: GetCounter

4.5.32.2 Response

Response contains `CounterValue` message:

Field Name	Label	Type	Description
value	required	uint32	Counter value

Table 4.150: CounterValue

According to the EV1 specification only the 24 least significant bits of the `value` field are meaningful.

4.5.32.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.33 Mifare Ultralight EV1 Increment Counter

This command supported only by the Mifare Ultralight EV1 card. The command increments value of the one of the EV1 counters stored in the EV1 card.

4.5.33.1 Request

Request contains `IncrementCounter` message:

Field Name	Label	Type	Description
counter number	required	CounterNumber	Counter number
operand	required	uint32	Increment value

Table 4.152: IncrementCounter

According to the EV1 specification only the 24 least significant bits of the `operand` field are meaningful.

4.5.33.2 Response

No data carried with successful response.

4.5.33.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.34 Mifare Ultralight EV1 Get Version

This command supported only by the Mifare Ultralight EV1 card. The command returns “Product version information” from the EV1 card “as is”, i.e. without modifications.

4.5.34.1 Request

Request contains `GetVersion` message:

Field Name	Label	Type	Description

Table 4.154: GetVersion

4.5.34.2 Response

Response contains `CounterValue` message:

Field Name	Label	Type	Description
raw version data	required	bytes	Response to the ’GET VERSION’ command as is

Table 4.155: Version

4.5.34.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.35 Mifare Ultralight Bulk operation

This command allows to combine several commands to be processed as single request to the reader, that usually processed as separated requests.

The reader internally processes combined commands on one-by-one approach. If error is occurred then reader stops processing, clears all results from already processed commands and returns this error in `Failure response`.

Format of the commands and their responses the same as in separated commands.

4.5.35.1 Request

Request contains `BulkOperation` message:

Field Name	Label	Type	Description
operations	repeated	Command	Sequence of the Mifare Ultralight commands

Table 4.157: BulkOperation

The reader processing following `Command`s one-by-one in order specified in `operations` field:

Field Name	Label	Type	Description
read pages	optional	ReadPages	Read Mifare Ultralight pages
write pages	optional	WritePages	Write pages to Mifare Ultralight
get counter	optional	GetCounter	Read value of Mifare Ultralight EV1 built-in counter
increment counter	optional	IncrementCounter	Increment Mifare Ultralight EV1 built-in counter
auth on clear key	optional	ClearKey	Mifare Ultralight C 3DES Authentication using clear key
auth on sam key	optional	SamKey	Mifare Ultralight C 3DES Authentication using SAM AV2
auth clear password	optional	ClearPassword	Mifare Ultralight EV1 password based Authentication using clear key
auth sam password	optional	SamPassword	Mifare Ultralight EV1 password based Authentication using NXP SAM AV3 module
auth on internal key	optional	InternalKey	Mifare Ultralight C 3DES Authentication using key from internal secure storage

Table 4.158: Command

4.5.35.2 Response

Response contains `BulkResult` message:

Field Name	Label	Type	Description
results	repeated	CommandResult	Sequence of the results of Mifare Ultralight commands

Table 4.159: BulkResult

The reader collects results in `results` field in order specified in `operations` field of the *Request*:

Field Name	Label	Type	Description
pages	optional	Pages	Response to ReadPages command
counter value	optional	CounterValue	Response to GetCounter command

Table 4.160: CommandResult

The reader does not collect results for commands that have *empty* response on success, such as `Write Pages`.

4.5.35.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.35.4 Example

Perform the following operations in the specified order:

1. write eight (8) pages (32 bytes) starting from address 04h
2. read four (4) pages starting from address 04h
3. read four (4) pages starting from address 08h

```
mfr_ul_bulk_operation {
    operations {
        write_pages {
            start_address: 4
            data: "\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f
            \x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f"
        }
    }
    operations {
        read_pages {
            start_address: 4
            page_count: 4
        }
    }
    operations {
        read_pages {
            start_address: 8
            page_count: 4
        }
    }
}
```

4.5.36 Mifare Ultralight-C 3DES Authentication on Clear Key

This command supported only by the Mifare Ultralight-C card.

After successful authentication Read/Write commands may be performed to memory areas, which are only readable and/or writeable after authentication.

4.5.36.1 Request

Request contains `ClearKey` message:

Field Name	Label	Type	Description
clear key	required	bytes	Value of the Key (16 bytes)

Table 4.162: ClearKey

4.5.36.2 Response

No data carried with successful response.

4.5.36.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.36.4 Example

Perform authentication using example key:

```
mfr_ul_auth_on_clear_key { clear_key: "IEMKAERB!NACUOYF" }
```

4.5.37 Mifare Ultralight-C 3DES Authentication on Key from SAM AV2/AV3

This command supported only by the Mifare Ultralight-C card.

After successful authentication Read/Write commands may be performed to memory areas, which are only readable and/or writeable after authentication.

4.5.37.1 Request

Request contains `SamKey` message:

Field Name	Label	Type	Description
av2 args	required	AuthenticationArguments	Specifies SAM AV2 arguments that will be used in authentication
diversification input	optional	bytes	Input value to the Key Diversification Algorithm if required

Table 4.164: SamKey

The Key Diversification Algorithm is optional and described in the SAM AV2/AV3 documentation.

4.5.37.2 Response

No data carried with successful response.

4.5.37.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.37.4 Example

Perform authentication using key at `KeyEntry` #27 on the SAM at `SAM3_SLOT`:

```
mfr_ul_auth_on_sam_key { av2_args { slot: SAM3_SLOT key_number: 27 } }
```

4.5.38 Mifare Ultralight-C 3DES Authentication on Internal Key

This command supported only by the Mifare Ultralight-C card and gets key from the `Internal Storage`.

After successful authentication Read/Write commands may be performed to memory areas, which are only readable and/or writeable after authentication.

Available from version 1.38.

4.5.38.1 Request

Request contains `InternalKey` message:

Field Name	Label	Type	Description
key index	required	uint32	Index of the key within internal storage

Table 4.166: InternalKey

4.5.38.2 Response

No data carried with successful response.

4.5.38.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.38.4 Example

Perform authentication using key from internal storage at index 4:

```
mfr_ul_auth_on_internal_key { key_index: 4 }
```

4.5.39 Mifare Ultralight EV1 Password-based Authentication on Clear Password

This command supported only by the Mifare Ultralight EV1 card.

After successful authentication Read/Write commands may be performed to memory areas, which are only readable and/or writeable after authentication.

4.5.39.1 Request

Request contains `ClearPassword` message:

Field Name	Label	Type	Description
password	required	bytes	Value of the Password (4 bytes)

Table 4.168: ClearPassword

4.5.39.2 Response

Response contains `PasswordAcknowledge` message:

Field Name	Label	Type	Description
password ack	required	bytes	Password acknowledged value (2 bytes)

Table 4.169: PasswordAcknowledge

4.5.39.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.39.4 Example

Perform authentication using factory password:

```
mfr_ul_auth_clear_password { password: "\xff\xff\xff\xff" }
```

4.5.40 Mifare Ultralight EV1 Password-based Authentication on Key from the SAM AV3

This command supported only by the Mifare Ultralight EV1 card and NXP SAM **AV3**.

After successful authentication Read/Write commands may be performed to memory areas, which are only readable and/or writeable after authentication.

4.5.40.1 Request

Request contains `SamPassword` message:

Field Name	Label	Type	Description
av3 args	required	AuthenticationArguments	Specifies SAM AV3 arguments that will be used in authentication
diversification input	optional	bytes	Input value to the Key Diversification Algorithm if required

Table 4.171: SamPassword

4.5.40.2 Response

Response contains `PasswordAcknowledge` message:

Field Name	Label	Type	Description
password ack	required	bytes	Password acknowledged value (2 bytes)

Table 4.172: PasswordAcknowledge

Password acknowledge value in the response is already successfully checked by the SAM.

4.5.40.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.5.40.4 Example

Perform authentication using key #28 at the `SAM3_SLOT`:

```
mfr_ul_auth_sam_password { av3_args { slot: SAM3_SLOT key_number: 28 } }
```

4.6 Mifare Extended commands module

Each message should contains *one of* the following message types:

Field Name	Label	Type	Description
mfr classic read sectors	optional	ReadSectors	Authenticates & reads multiple sectors from Mifare Classic card
mfr classic write sectors	optional	WriteSectors	Authenticates & writes multiple sectors on Mifare Classic card
mfr get version	optional	GetVersion	Read Product Version Information from generic Mifare card

Table 4.174: MifareExtended

Module ID: **11h**.

Available from version 1.26.

4.6.1 Mifare Classic Read Multiple Sectors

This command allows to authenticate and read multiple sectors at once.

The following rules are apply to the command:

- Reader does three (3) attempts to authentication & read an each sector
- Reader returns an error if no sectors have beed read at all
- If card does not answer then reader just return the `CARD_TIMEOUT` error

This command read the whole sector including trailing block if required. For sectors 32 and futher the reader reads out up to 16 blocks.

4.6.1.1 Request

Request contains `ReadSectors` message:

Field Name	Label	Type	Description
sectors	repeated	Sector	Sectors to be read
read trailer block	optional	bool	Whether to read trailer block

Table 4.175: ReadSectors

Each `Sector` can be authenticated by the Clear Key or by the Sam Key:

Field Name	Label	Type	Description
clear key	optional	ClearKey	Authenticates to Mifare Classic sector using clear key
sam key	optional	SamKey	Authenticates to Mifare Classic sector using NXP SAM AV2 module
internal key	optional	InternalKey	Authenticates to Mifare Classic sector using internal key

Table 4.176: Sector

4.6.1.2 Response

Response contains `SectorsData` message:

Field Name	Label	Type	Description
sectors data	repeated	SectorData	Successfully read sectors

Table 4.177: SectorsData

`SectorsData` contains only sectors that have been successfully read:

Field Name	Label	Type	Description
sector number	required	uint32	Number of Mifare Classic sector
blocks	required	Blocks	Sector's blocks data

Table 4.178: SectorData

4.6.1.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.6.1.4 Example

Request to authenticate and read three (3) sectors:

```
mfr_classic_read_sectors {
    sectors {
        sam_key {
            sector_number: 7
            key_type: TYPE_A
            av2_args { slot: SAM3_SLOT key_number: 3 }
        }
    }
    sectors {
        sam_key {
            sector_number: 8
            key_type: TYPE_A
            av2_args { slot: SAM3_SLOT key_number: 1 }
        }
    }
    sectors {
        clear_key {
            sector_number: 9
            key_type: TYPE_A
            clear_key: "\xff\xff\xff\xff\xff\xff" }
    }
}
```

4.6.2 Mifare Classic Write Multiple Sectors

This command allows to authenticate and write multiple sectors at once.

The following rules are apply to the command:

- Reader does three (3) attempts to authentication & write an each sector
- If reader is unable to authenticate or write the sector then it immediately returns an error
- If card does not answer then reader just return the `CARD_TIMEOUT` error

In case of error the `mifare_error_at` field of the `Failure response` contains a sector & block number where the error is occured.

4.6.2.1 Request

Request contains `WriteSectors` message:

Field Name	Label	Type	Description
sectors	repeated	WriteSector	Sectors to be write

Table 4.180: WriteSectors

`WriteSectors` contains list of the sector that will be written:

Field Name	Label	Type	Description
sector	required	Sector	Sector to be write
blocks	optional	WriteBlocks	Sector blocks to be write
counters	repeated	ModifyCounter	Sector counters to be modified

Table 4.181: WriteSector

See the `Mifare Classic Write Data Blocks` command for the `WriteBlocks` definition.

Since v. 1.49 it is possible to modify counter blocks.

See the `Mifare Classic Modify Counter Value` command for the `ModifyCounter` definition.

Counter modification works only if all following conditions are met:

- the 'dst_block' field is specified in the command
- this command is preceded by the `Get Counter Value` or the `Set Counter Value` commands for the same block
- the `Get/Set Counter Value` commands is issued after the last `Poll for token` command

4.6.2.2 Response

No data carried with successful response. Successful response means that all sectors are written.

4.6.2.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.6.2.4 Example

Request to authenticate and write three (3) sectors:

```
mfr_classic_write_sectors {
    sectors {
        sector {
            sam_key {
                sector_number: 7
```

```
                key_type: TYPE_B
                av2_args { slot: SAM3_SLOT key_number: 4 }
            }
        }
        blocks {
            start_block: 0
            data: "\x11\x11\x11\x11\x11\x11\x11\x11\x11\x11\x11\x11\x11\x11\x11\x11"
        }
    }
    sectors {
        sector {
            sam_key {
                sector_number: 8
                key_type: TYPE_B
                av2_args { slot: SAM3_SLOT key_number: 2 }
            }
        }
        blocks {
            start_block: 0
            data: "\x22\x22\x22\x22\x22\x22\x22\x22\x22\x22\x22\x22\x22\x22\x22\x22"
        }
    }
    sectors {
        sector {
            clear_key {
                sector_number: 9
                key_type: TYPE_A
                clear_key: "\xff\xff\xff\xff\xff\xff"
            }
        }
        blocks {
            start_block: 0
            data: "\x33\x33\x33\x33\x33\x33\x33\x33\x33\x33\x33\x33\x33\x33\x33\x33"
        }
    }
}
```

4.6.3 Read Product Version Info of Generic Mifare Card

This command uses proprietary Mifare command `GET_VERSION` to read out "Product Version Information" from a Mifare card.

A card should be "activated" in advance by the `Poll for token` command.

At the moment the following Mifare products are support the `GET_VERSION` command:

- Mifare Ultralight EV1
- Mifare NTAG 424
- Mifare Plus EV2
- Mifare DESFire EV1

Successors of these products should also support the command.

4.6.3.1 Request

Request contains empty `GetVersion` message.

4.6.3.2 Response

Response contains `ProductVersionInfo` message:

Field Name	Label	Type	Description
info	required	bytes	Product Version Information

Table 4.183: ProductVersionInfo

The `info` field contains the Product Version Information "as is". Please refer to the particular Mifare specification for details about that information.

4.6.3.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.7 STMicroelectronics Cards module

Each message should contains *one of* the following message types:

Field Name	Label	Type	Description
stm sri512 read blocks	optional	ReadBlocks	Reads multiple blocks from STM SRI 512 card
stm sri512 write blocks	optional	WriteBlocks	Writes multiple blocks to STM SRI 512 card

Table 4.185: StmCard

Module ID: **08h**.

4.7.1 STM SRI512 / ST25TB Read Blocks

This command reads requested count of blocks from the STM token and should be issued after successfully processed `Poll for token` command.

4.7.1.1 Request

Request contains `ReadBlocks` message:

Field Name	Label	Type	Description
start block	required	uint32	Number of the first block
block count	optional	uint32	Count of blocks to be read

Table 4.186: ReadBlocks

Values allowed for `start_block` field are `0..15` and `255`.

4.7.1.2 Response

Response contains `Blocks` message:

Field Name	Label	Type	Description
data	required	bytes	Block(s) data

Table 4.187: Blocks

Size of `data` field `:=` `block_count` * 4 bytes.

4.7.1.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.7.1.4 Example

Read three (`3`) blocks starting from seven (`7`):

```
stm_sri512_read_blocks { start_block: 7 block_count: 3 }
```

4.7.2 STM SRI512 / ST25TB Write Blocks

This command allows to write multiple blocks per one command and should be issued after successfully processed `Poll for token` command.

4.7.2.1 Request

Request contains `WriteBlocks` message:

Field Name	Label	Type	Description
start block	required	uint32	Number of the first block
data	required	bytes	Data to be written into Block(s)

Table 4.189: WriteBlocks

Values allowed for `start_block` field are `0..15` and `255`.

Size of `data` field **must be multiple of 4 bytes**. Counts of blocks to be written defined as `sizeof( data ) / 4`.

4.7.2.2 Response

No data carried with successful response.

Successful response does not guarantee that data have been written because STM card do not provide any response to the reader. So `Read Blocks` command should be used to check that actual data written on the card.

4.7.2.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.7.2.4 Example

Writes four (4) blocks starting from seven (7) with value `00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F`:

```
stm_sri512_write_blocks {
    start_block: 7
    data: "\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f"
}
```

4.8 NXP NTAG Cards module

This module contains commands to work with NXP NTAG 4xx cards.

Each message should contains *one of* the following message types:

Field Name	Label	Type	Description
ntag select file	optional	SelectFile	Selects file on the card's filesystem
ntag auth on clear key	optional	ClearKey	Authenticates card by the plaintext key
ntag read file	optional	ReadFile	Reads out file from the card
ntag write file	optional	WriteFile	Writes down raw data to the file on the card
ntag auth on internal key	optional	InternalKey	Authenticates card by the key from internal secure storage

Table 4.191: NtagCard

Module ID: **09h**.

4.8.1 NXP NTAG Select File

This command selects file on filesystem according to ISO/IEC 7816-4. `Poll for token` command should successfully performed in advance.

Command allows to select one of the files:

- PICC Level (ISO MF Name = D2760000850100h)
- Application Level (ISO DF Name = D2760000850101h)

NTAG functionality (except `Read_Sig` command) is available only on Application Level.

4.8.1.1 Request

Request contains `SelectFile` message:

Field Name	Label	Type	Description
file	optional	IsoFile	ISO file to be selected

Table 4.192: SelectFile

4.8.1.2 Response

No data carried with successful response.

4.8.1.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.8.1.4 Example

Select default file (Application Level):

```
ntag_select_file { }
```

4.8.2 NXP NTAG Authenticate Card On Clear Key

This command authenticates NTAG card using plaintext AES128 key. `NTAG Select File` command should be performed in advance.

4.8.2.1 Request

Request contains `ClearKey` message:

Field Name	Label	Type	Description
key on card	required	uint32	Number of the Key in NTAG card
clear key	required	bytes	Value of the Key (16 bytes)

Table 4.194: ClearKey

`key_on_card` field allows values 0..4.

4.8.2.2 Response

No data carried with successful response.

4.8.2.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.8.2.4 Example

Authenticate card using the second (2) key with value 00000000000000000000000000000000:

```
ntag_auth_on_clear_key {
    key_on_card: 2
    clear_key: "\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00"
}
```

4.8.3 NXP NTAG Authenticate Card On Internal Key

This command authenticates NTAG card using key from the `Internal Storage`. `NTAG Select File` command should be performed in advance.

Available from version 1.38.

4.8.3.1 Request

Request contains `InternalKey` message:

Field Name	Label	Type	Description
key on card	required	uint32	Number of the Key in NTAG card
key index	required	uint32	Index of the key within internal storage

Table 4.196: InternalKey

`key_on_card` field allows values 0..4.

4.8.3.2 Response

No data carried with successful response.

4.8.3.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.8.3.4 Example

Authenticate card using the second (`2`) key on card and key from internal storage at index `5`:

```
ntag_auth_on_internal_key {
    key_on_card: 2
    key_index: 5
}
```

4.8.4 NXP NTAG Read File

This command reads whole file at offset 0 (zero) until end and return its content. Depending on file's settings successful authentication may be required.

4.8.4.1 Request

Request contains `ReadFile` message:

Field Name	Label	Type	Description
file	required	File	File to be read
protection	optional	Protection	Protection used during transmission

Table 4.198: ReadFile

4.8.4.2 Response

Response contains `FileContent` message:

Field Name	Label	Type	Description
data	required	bytes	Raw data

Table 4.199: FileContent

4.8.4.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.8.4.4 Example

Read `NDEF_MESSAGE` file without any protection (`PLAINTEXT` mode):

```
ntag_read_file { file: NDEF_MESSAGE protection: PLAINTEXT }
```

4.8.5 NXP NTAG Write File

This command writes raw data to the file at offset 0 (zero) on the card's filesystem. Depending on file's settings successful authentication may be required.

4.8.5.1 Request

Request contains `WriteFile` message:

Field Name	Label	Type	Description
file	required	File	File to be write
data	required	bytes	Raw data
protection	optional	Protection	Protection used during transmission

Table 4.201: WriteFile

Size of `data` field must be equal or less than maximum size of file on the card's filesystem.

4.8.5.2 Response

No data carried with successful response.

4.8.5.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.8.5.4 Example

Write `[0x00..0xff]` array to `NDEF_MESSAGE` file under full protection mode:

```
ntag_write_file {
    file: NDEF_MESSAGE
    data: "
    \x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f
    \x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f
    \x20\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f
    \x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f
    \x40\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f
    \x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f
    \x60\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f
    \x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f
    \x80\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f
    \x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f
    \xa0\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf
    \xb0\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf
    \xc0\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf
    \xd0\xd1\xd2\xd3\xd4\xd5\xd6\xd7\xd8\xd9\xda\xdb\xdc\xdd\xde\xdf
    \xe0\xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea\xeb\xec\xed\xee\xef
    \xf0\xf1\xf2\xf3\xf4\xf5\xf6\xf7\xf8\xf9\xfa\xfb\xfc\xfd\xfe\xff
    "
    protection: ENCRYPTED_WITH_MAC
}
```

4.9 PIN-pad module

This module accepts requests for PIN-pad and secure storage related operations, some of which are supported only certain hardware model:

Field Name	Label	Type	Description
online pin	optional	EnterOnlinePin	Asks cardholder to enter PIN and returns encrypted PIN-block
prepare injection	optional	PrepareInjection	Prepare pinpad to inject a secret
inject secret	optional	InjectSecret	Inject a secret into secure pinpad storage
list secrets	optional	ListSecrets	Return list of injected secrets
cipher	optional	CipherInput	Encrypt/decrypt arbitrary data using key from internal storage
hmac	optional	HmacInput	Compute HMAC hash for arbitrary data using key from internal storage
generate mac	optional	GenerateMac	Generate MAC for arbitrary data using key from internal storage

Table 4.203: PinPad

Module ID: **06h**.

Available from version 1.38.

4.9.1 Inject a secret

Inject a secret procedure consists of three steps:

4.9.1.1 Step 1. Get PIN-pad's attributes required for injection

1. Send `PrepareInjection` message
2. Get `DeviceAttributes` message

Field Name	Label	Type	Description
unpredictable number	required	bytes	8 random bytes generated by a device
certificates	required	bytes	List of PEM-encoded X.509 certificates: Unique (per device) certificate, Manufacture CA certificate

Table 4.204: DeviceAttributes

3. Validate the device's certificate
4. Extract the device's public key from the certificate

To validate the device's certificate use following chain of trust:

1. Use `product_ca.pem` as Root CA certificate
2. Use Manufacture CA certificate from the `certificates` field as first certificate in the chain of trust
3. Use Unique device's certificate from the `certificates` field as last certificate in the chain of trust

4.9.1.2 Step 2. Prepare an encrypted secret

1. Fill `InjectionData` message:

Field Name	Label	Type	Description
index	required	uint32	Index in the secure storage associated with a secret
unpredictable number	required	bytes	The 'unpredictable number' field from DeviceAttributes
secret	required	Secret	Secret to be injected

Table 4.205: InjectionData

Field Name	Label	Type	Description
value	required	bytes	Value of the secret
attributes	required	SecretAttributes	Attributes assigned to the secret

Table 4.206: Secret

Field Name	Label	Type	Description
usage	required	Usage	Target usage allowed for the secret
algorithm	required	Algorithm	Algorithm that allowed to use with the secret
tag	required	uint32	A tag with arbitrary value associated with the secret value
fixed key	optional	FixedKey	Fixed key method
master session	optional	MasterSession	Master/session key method
dukpt	optional	Dukpt	Derived unique key per transaction (DUKPT) method

Table 4.207: SecretAttributes

2. Encode the message into the Protocol Buffers binary representation
3. Encrypt the encoded data using device's public key extracted in the Step 1

To encrypt data use PKCS #1 RSAES-OAEP v2.1 encryption with SHA-1 as a hash function.

The `tag` field can be used for KCV value or as version of a secret.

4.9.1.3 Step 3. Inject the encrypted secret

1. Send `InjectSecret` message with encrypted injection data from the Step 2:

Field Name	Label	Type	Description
encrypted injection data	required	bytes	Protobuf encoded InjectionData message encrypted with device's public key

Table 4.208: InjectSecret

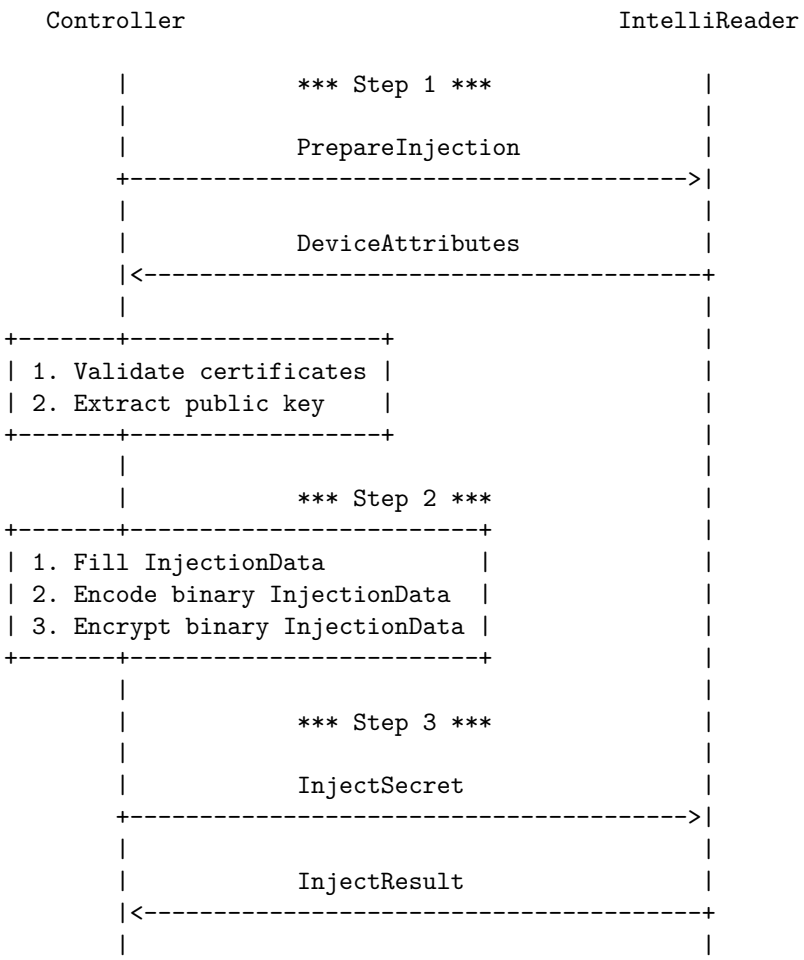
2. Get `InjectResult` message:

Field Name	Label	Type	Description
inject status	required	InjectStatus	Injection status

Table 4.209: InjectResult

3. Check that `inject_status` field is `INJECT_SUCCESS`

4.9.1.4 Inject a secret message flow



4.9.1.5 Mifare card authentication using injected secret

To use a secret as a key in the mifare authentication inject a secret according to the tables:

- For direct authentication choose the `PICC_AUTHENTICATION` usage:

Mifare card family	Algorithm	Length (bytes)
Miface Classic	CRYPTO1	6
Mifare Plus	AES	16
Mifare Ultralight C	TRIPLE_DES	16
Mifare NTAG	AES	16

- To use secret as a diversification key choose the `PICC_KEY_DIVERSIFICATION` usage:

Mifare card family	Algorithm	Length (bytes)
Two keys are required for Mifare Classic	CRYPTO1	6
	TRIPLE_DES	16
Mifare Plus	AES	16

The index of secret can be arbitrary.

Controller should maintain list of injected secret indexes and use appropriate index in the mifare authentication commands that use PIN-pad's secure storage.

4.9.2 List injected secrets

This commands allows to get list of secrets injected to PIN-pad's secure storage without reveal the secret's value.

Request

Request contains `ListSecrets` message.

Response

Response contains `SecretsInfo` message:

Field Name	Label	Type	Description
secret info	repeated	SecretInfo	List of secrets injected to the secure storage

Table 4.212: SecretsInfo

Field Name	Label	Type	Description
index	required	uint32	Index in the secure storage associated with a secret
secret attributes	required	SecretAttributes	Attributes assigned to the secret

Table 4.213: SecretInfo

Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.9.3 Encrypt data

This commands allows to encrypt and decrypt arbitrary data on secret key from the PIN-pad's secure storage.

Available from version 1.39.

DUKPT is available from version 1.45.

Request

Request contains `CipherInput` message:

Field Name	Label	Type	Description
operation	required	CipherOperation	Cipher operation on data
key index	required	uint32	Index of cipher key within internal storage
algorithm	required	Algorithm	Cipher algorithm
input data	required	bytes	Cipher input data
cipher mode	optional	CipherMode	Cipher mode

Table 4.215: CipherInput

Supported algorithms:

- AES
- Triple DES

Supported cipher modes:

- EBC
- CBC

Supported key management methods:

- Fixed key
- DUKPT

Response

Response contains `CipherOutput` message:

Field Name	Label	Type	Description
output data	required	bytes	Cipher output data
ksn	optional	bytes	Key Serial Number (KSN) in case of DUKPT key used

Table 4.216: CipherOutput

CBC ciphering mode

Supported parameters:

Field Name	Label	Type	Description
iv	optional	bytes	Initialization vector (IV), zeroes if not set
padding	optional	CipherPadding	Input data padding, PKCS7 if not set

Table 4.217: CBC

Supported paddings:

Value	Number	Description
NONE	0	Never pad (full blocks only)
PKCS7	1	PKCS7 padding
ONE AND ZEROS	2	ISO/IEC 7816-4 padding
ZEROS AND LEN	3	ANSI X.923 padding

ZEROS	4	Zero padding (not reversible)

Table 4.218: CipherPadding

Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.9.4 Compute HMAC

This commands allows to compute HMAC over arbitrary data on secret key from the PIN-pad's secure storage.

Available from version 1.40.

Request

Request contains `HmacInput` message:

Field Name	Label	Type	Description
key index	required	uint32	Index of key within internal storage
algorithm	required	Algorithm	HMAC algorithm
input data	required	bytes	HMAC input data

Table 4.220: HmacInput

Supported algorithms:

- HMAC-SHA1
- HMAC-SHA256

Response

Response contains `HmacOutput` message:

Field Name	Label	Type	Description
output data	required	bytes	HMAC output data

Table 4.221: HmacOutput

Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.9.5 Enter and encrypt PIN

This commands allows to enter a PIN (password) on the display and then encrypts it on a secret key from the Internal Secure Storage.

Available from version 1.44.

DUKPT is available from version 1.45.

Request

Request contains `EnterOnlinePin` message:

Field Name	Label	Type	Description
pan	optional	string	Primary Account Number (PAN), 12-19 ASCII decimal digits only
entry	optional	SecureEntry	Parameters used for PIN entry dialog
encryption key	required	EncryptionKey	Encryption Key

Table 4.223: EnterOnlinePin

If the `pan` field is not provided then will be used PAN of the last card.

Response

Response contains `EntryResult` message:

Field Name	Label	Type	Description
status	required	EntryStatus	Status of PIN entry
pin block	optional	bytes	Encrypted PIN-block (PIN ^PAN). Present when ‘status‘ is ‘PIN ENTERED‘
ksn	optional	bytes	Key Serial Number (KSN) of DUKPT. Present when ‘status‘ is ‘PIN ENTERED‘

Table 4.224: EntryResult

Possible entry statuses:

Value	Number	Description
PIN ENTERED	0	Online PIN entered by cardholder, terminal successfully built PIN-block
PIN ENTRY BYPASSED	1	Cardholder has rejected PIN entry (e.g. by Cancel button)
PIN ENTRY TIMEOUT	2	PIN has not entered during predefined timeout

Table 4.225: EntryStatus

Auxiliary messages

Message	Comments
Pending	Return
Notification	Return
Cancellation	Allow

Direct display hardware

IntelliReader doesn’t control LCD on such hardware.

To enter a PIN you need to display a PIN-entry dialog as usual and provide coordinates of the buttons to IntelliReader in the `external` field of the `SecureEntry` entry.

IntelliReader will send `Notification` message with the `secure_entry_status` field set. The `text_length` field of the `SecureEntryStatus` is set to the count of the entered digits.

Encryption key requirements

Master Key used for PIN-block encryption must have:

- `PIN_ENCRYPTION` usage
- `TRIPLE_DES` or `AES` algorithm
- Master/Session or DUKPT method

Supported PIN-block formats

- ISO 9564 Format 0 for Triple DES
- ISO 9564 Format 4 for AES

Output PIN-block format will be automatically chosen by the Master Key's algorithm.

PIN attack delay

IntelliReader implements the following PIN attack delay algorithm: If user enters different PINs for the same PAN then after fifth PIN entered IntelliReader adds the 50 seconds delay.

4.9.6 Compute MAC

This commands allows to compute MAC over arbitrary data on secret key from the PIN-pad's secure storage.

Available from version 1.53.

Request

Request contains `GenerateMac` message:

Field Name	Label	Type	Description
input data	required	bytes	MAC input data
cmac	optional	Cmac	CMAC
mac algo 1	optional	MacAlgorithm1	ISO 9797-1:2011 Algorithm 1
mac algo 2	optional	MacAlgorithm2	ISO 9797-1:2011 Algorithm 2
mac algo 3	optional	MacAlgorithm3	ISO 9797-1:2011 Algorithm 3
mac algo 4	optional	MacAlgorithm4	ISO 9797-1:2011 Algorithm 4
mac algo 6	optional	MacAlgorithm6	ISO 9797-1:2011 Algorithm 6

Table 4.227: GenerateMac

Supported algorithms:

- CMAC (ISO 9797-1:2011 Algorithm 5)
- ISO 9797-1:2011 Algorithm 1 *Available from version 1.54.*
- ISO 9797-1:2011 Algorithm 2 *Available from version 1.54.*
- ISO 9797-1:2011 Algorithm 3 *Available from version 1.54.*
- ISO 9797-1:2011 Algorithm 4 *Available from version 1.54.*
- ISO 9797-1:2011 Algorithm 6 *Available from version 1.54.*

Supported keys:

- Fixed Key (AES, 3DES)
- X9.17 Session Key (AES, 3DES)
- TR31 Session Key (AES, 3DES)
- DUKPT (AES)

Response

Response contains `MacOutput` message:

Field Name	Label	Type	Description
output data	required	bytes	MAC output data
ksn	optional	bytes	Key Serial Number (KSN) of DUKPT.
second ksn	optional	bytes	Key Serial Number (KSN) of DUKPT for the second key.

Table 4.228: MacOutput

Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

Supported key derivation methods

MAC algorithms 2, 4 and 6 use derived keys for MAC calculations.

To set a derivation method, use the `MacKeyDerivationMethod` parameter of the chosen algorithm:

Value	Number	Description
XORED WITH F0	0	Complementing alternate substrings of four bits commencing with the first four bits (ISO 9797-1:2011)
KEY DERIVATION METHOD 1	1	Key derivation method 1 (ISO 9797-1:2011)

Table 4.230: MacKeyDerivationMethod

For `XORED_WITH_F0` a single derived key is calculated by complementing alternate substrings of four bits commencing with the first four bits of the original key (ISO 9797-1:2011). A pair of `(original key, derived key)` is formed to be used in the MAC algorithm.

For `KEY_DERIVATION_METHOD_1` a pair of derived keys is calculated from a common master key using Key Derivation Method 1 (ISO 9797-1:2011). A pair of `(first derived key, second derived key)` is formed to be used in the MAC algorithm.

4.10 Service module

This module accepts requests for service operations, some of which are supported only certain hardware model:

Field Name	Label	Type	Description
prepare update	optional	Prepare	Prepare device to firmware update
update block	optional	UpdateBlock	Update block of firmware image
apply update	optional	Apply	Apply saved firmware update
rollback update	optional	Rollback	Rollback firmware update
get diagnostic	optional	GetDiagnostic	DEPRECATED in favor of srv.selftest.RunSelfTest
prepare for config	optional	Prepare	Prepare device to upload configuration
upload block of config	optional	UploadBlock	Upload one block of the configuration
check configuration	optional	CheckConfiguration	Check uploaded configuration
run selftest	optional	RunSelfTest	Run self-test

Table 4.231: Service

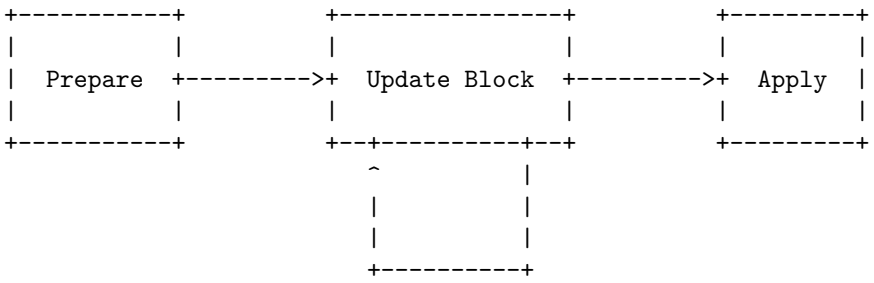
Module ID: **07h**.

4.10.1 Update Firmware

Firmware update procedure consists of three steps:

1. Prepare firmware update
2. Upload and save firmware image block (many times)
3. Apply uploaded firmware

Whole procedure flow is depicted on the following diagram:



These steps must be done **strongly** in described order and will be described in-depth in the following chapters.

This procedure is used in the Initialization phase.

4.10.1.1 Firmware rollback feature

Device contains two partitions for firmware image. Built-in boot application chooses partition from which firmware application will be started.

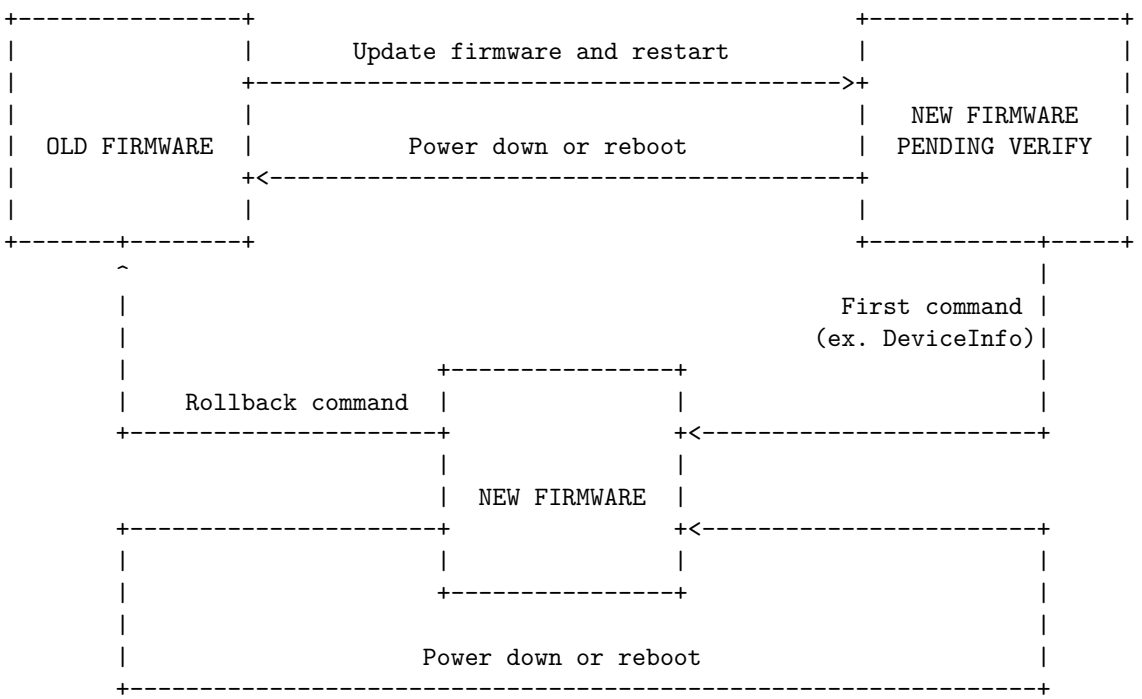
Built-in boot application has 'firmware rollback feature' that allows return to previously working partition if partition chosen by boot application is 'broken' somehow.

Firmware is 'broken' if it is not able to process request from the controller. On other hand, firmware will be marked as 'correct' when it have been processed first command.

So, to deploy new firmware after successful update host controller must:

1. restart device
2. and then send any command (ex. `DeviceInfo`) to device

Possible states are depicted on the following diagram:



4.10.1.2 IntelliReader Firmware (IRF) file format

All numbers are encoded in the big-endian byte order. All sizes are in bytes.

Field	Size	Description
Magic	4	Magic bytes “IRF1”
Size	4	Size of the Firmware Manifest field
Firmware Manifest	var	Firmware manifest encoded in the Protocol Buffers binary representation
Padding	var	Padding with zeroes up to 124 bytes from the start
Checksum	4	CRC32 starting from offset 0 till the Checksum field
Images	var	Concatenation of firmware images

Firmware manifest is defined in the `FirmwareManifest` message:

Field Name	Label	Type	Description
version	required	string	Version of the firmware
images	repeated	FirmwareImage	List of firmware images

Table 4.233: FirmwareManifest

Field Name	Label	Type	Description
kind	required	FirmwareImageKind	Kind of the image
offset	required	uint32	Offset of the image in bytes. Note that the value of the offset depends on FirmwareImage origin: - offset that is returned as part of the ‘srv.firmware update.Prepare‘ command’s response is from the start of the file; - offset that is inside an IRF package is from the start of the images (i.e. it does not include 128 bytes of the header).
size	required	uint32	Size of the image in bytes

Table 4.234: FirmwareImage

Note that `offset` here is from the start of images (i.e. it does not include 128 bytes of the header)

4.10.1.3 Prepare firmware update

This command starts firmware update procedure, device performs some prerequisite actions.

This command was changed in the backward-compatible way in the version 1.46.

Request

Request contains `Prepare` message:

Field Name	Label	Type	Description
firmware image size	optional	uint32	DEPRECATED
first 128 bytes	optional	bytes	First 128 bytes of the firmware image

Table 4.235: Prepare

The `first_128_bytes` field must contain the first 128 bytes of the firmware file.

Response

Response contains `FirmwareImage` message:

Field Name	Label	Type	Description
kind	required	FirmwareImageKind	Kind of the image
offset	required	uint32	Offset of the image in bytes. Note that the value of the offset depends on FirmwareImage origin: - offset that is returned as part of the 'srv.firmware update.Prepare' command's response is from the start of the file; - offset that is inside an IRF package is from the start of the images (i.e. it does not include 128 bytes of the header).
size	required	uint32	Size of the image in bytes

Table 4.236: FirmwareImage

The `offset` & `size` fields describe the location of the firmware image within the firmware file. Note that `offset` here is from the start of the file.

Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Ignore

4.10.1.4 Update block of firmware image

This command conveys one block of firmware image.

Controller should locate the firmware image within firmware file using `offset` and `size` fields from the `Prepare` response.

Controller should split firmware image into blocks and then transmit all blocks one by one using this command.

Request

Request contains `UpdateBlock` message:

Field Name	Label	Type	Description
data	required	bytes	Raw data block (max size 4096 bytes)

Table 4.238: UpdateBlock

Optimal size for `data` parameter is 4096 bytes.

Response

No data carried with successful response.

Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Ignore

Finalize firmware update

This command finalizes (applies) firmware update. This command must be sent after all data block have been transmitted. Newly updated firmware will be running after device restart.

Note that depending from the device model, the device may take some time to apply the firmware update.

Use the Initialization procedure to get into syncronization after firmware update.

Request

Request contains `Apply` message:

Field Name	Label	Type	Description
restart	optional	bool	Restart device if update have been successfully finished

Table 4.240: Apply

Response

No data carried with successful response.

Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Ignore

4.10.1.5 Rollback firmware update

This command rolls back previous firmware update if possible.

Request

Request contains `Rollback` message.

Response

No data carried with successful response.

Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Ignore

4.10.2 Upload reader configuration

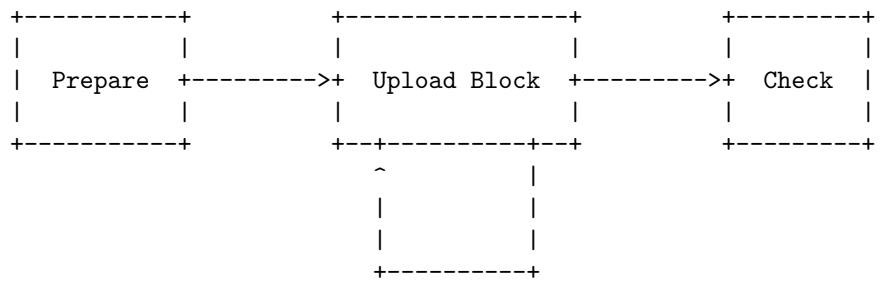
See Annex B for more information about `Reader Configuration`.

Devices with 4MBytes flash memory have the following limitation: the configuration will be uploaded into the inactive firmware partition (see Firmware rollback feature). Due to such limitation `Reader Configuration` will be lost after firmware update and it should be uploaded again. Devices with 16MBytes flash memory have not such limitation.

Configuration upload consists of three steps:

1. Prepare device to upload configuration
2. Upload block (chunk) of configuration file
3. Check uploaded configuration

Whole procedure flow is depicted on the following diagram:



These steps will be described in-depth in the following chapters.

Device reboot is required to apply the configuration.

This procedure is used in the Initialization phase.

4.10.2.1 Prepare device to upload configuration

This command starts upload configuration procedure, device performs some prerequisite actions.

Request

Request contains `Prepare` message:

Field Name	Label	Type	Description
config size	required	uint32	Size of the configuration file in bytes
region	optional	Region	Region of the internal storage

Table 4.243: Prepare

Response

No data carried with successful response.

Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Ignore

4.10.2.2 Upload block of configuration

This command conveys one block of configuration file. Controller should split configuration into blocks and then transmit all blocks one by one using this command.

Request

Request contains `UploadBlock` message:

Field Name	Label	Type	Description
data	required	bytes	Raw data block (max size 4096 bytes)

Table 4.245: UploadBlock

Optimal size for `data` parameter is 4096 bytes. All blocks length, except last, must be multiple to 16.

Response

Since version 1.50.7505 reader checks the uploaded configuration as soon as it received the last block.

If there are no errors in the uploaded configuration, the response contains the `Configuration` message that's described in the Check uploaded configuration command.

Otherwise (version $<$ 1.50.7505 or a block is not last) no data carried with successful response.

Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Ignore

Check uploaded configuration

This commands checks uploaded (or stored in the device) configuration and returns some information about it.

Request

Request contains `CheckConfiguration` message:

Field Name	Label	Type	Description
region	optional	Region	Region of the internal storage

Table 4.247: CheckConfiguration

Response

Response contains `Configuration` message:

Field Name	Label	Type	Description
files	repeated	File	Files withing configuration
crc32	optional	uint32	CRC32 of the configuration

Table 4.248: Configuration

`crc32` field contains CRC32 of the whole configuration file. Absence of the field for the protocol version >= 1.7 means that there is no configuration was uploaded.

`files` field contains information about each file in the configuration:

Field Name	Label	Type	Description
name	required	bytes	File name
size	required	uint32	File size
crc32	required	uint32	CRC32 value of the file

Table 4.249: File

Empty `files` field means that there is no configuration.

Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Ignore

4.10.2.3 Two regions within configuration area

Since version 1.41 IntelliReader supports two regions within the configuration area:

- Common: default, for any files
- EMV: special, small size region, only for EMV related files

Most of time you may just use “Common” region for all files.

But, when there are two separated parties that configure the reader it may be useful to split configuration into two parts: EMV and non-EMV files. In such case you can manage and upload them separatelly.

The target region is defined by the `region` field in `Prepare` and `CheckConfiguration` messages.

4.10.3 Self test

This command requires the reader to perform a self test. Reader actively tests the connected HW modules such as NFC, QR-scanner and etc so it may took some time to finish the command.

This command is used in the Initialization phase.

Available since v. 1.42.

4.10.3.1 Request

Request contains the `RunSelfTest` message.

4.10.3.2 Response

Response contains `SelfTestResult` message:

Field Name	Label	Type	Description
hardware	required	Hardware	Device's hardware status

Table 4.251: SelfTestResult

Response contains list of HW modules that have been tested:

Field Name	Label	Type	Description
nfc	required	HwComponent	The NFC hardware
sam	required	HwComponent	Secure Authentication Module (SAM) hardware
display	required	HwComponent	The display (LCD)
touchscreen	required	HwComponent	The touchscreen of the display
qr scanner	required	HwComponent	QR (barcode) scanner
dry contacts	required	HwComponent	Dry contacts (turnstile control lines) hardware

Table 4.252: Hardware

Result contains one of the following statuses for each HW module:

Value	Number	Description
HW COMPONENT OK	0	Hardware component works normally, 'info' field may contain HW module version
HW COMPONENT BROKEN	1	Hardware component is broken, 'info' field may contain additional information
HW COMPONENT ABSENT	2	Hardware component is absent

Table 4.253: HwComponentStatus

4.10.3.3 Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Ignore

4.11 Graphic User Interface module

This module accepts requests to show graphical information to cardholder (user) by the reader's screen. Display commands are only supported by hardware with LCD module.

Field Name	Label	Type	Description
show screen	optional	ShowScreen	Shows user-defined screen on the reader's display
input dialog	optional	InputDialog	Shows the input screen and interacts with user to enter text
menu dialog	optional	MenuDialog	Shows the menu and interacts with user to select an item
draw bitmap	optional	DrawBitmap	Draw bitmap on the LCD
slideshow	optional	Slideshow	Show pictures as slideshow

Table 4.255: Gui

Built-in font supports:

- Armenian
- Azerbaijani
- Belarusian
- Bulgarian
- English
- Greek
- Hungarian
- Kazakh
- Romanian
- Russian
- Ukrainian alphabets
- digits
- and special characters

Module ID: **0Ah**.

4.11.1 Show user-defined screen

This command allows to display text, picture or QR-code to any position on the reader's screen. But instead of specifying certain `(X, Y)` coordinates on the screen, user should "build" a screen by combine and nest various widgets. `Layout` widget does not appear on the screen, it need only to position other widgets on the screen.

Note this command updates the whole screen.

There are following widgets:

Field Name	Label	Type	Description
widgets	repeated	Widget	Widgets within layout

Table 4.256: VerticalLayout

Field Name	Label	Type	Description
widgets	repeated	Widget	Widgets within layout

Table 4.257: HorizontalLayout

Field Name	Label	Type	Description
text	required	string	The text to show
vertical alignment	optional	VerticalAlignment	How to vertically align the text
horizontal alignment	optional	HorizontalAlignment	How to horizontally align the text
background	optional	Background	Widget background, use root widget background by default
foreground	optional	Foreground	Widget foreground color, use root widget foreground by default
font	optional	Font	Font for rendering text
button id	optional	uint32	Makes widget "pressable" and assign ID to distinguish it
border	optional	Border	Optional border around the Text widget

Table 4.258: Text

Field Name	Label	Type	Description
picture id	required	PictureId	The picture ID to show
vertical alignment	optional	VerticalAlignment	How to vertically align the picture
horizontal alignment	optional	HorizontalAlignment	How to horizontally align the picture
background	optional	Background	Widget background, use root widget background by default
button id	optional	uint32	Makes widget "pressable" and assign ID to distinguish it

Table 4.259: Picture

Field Name	Label	Type	Description
name	required	string	The customer picture name to show
vertical alignment	optional	VerticalAlignment	How to vertically align the picture
horizontal alignment	optional	HorizontalAlignment	How to horizontally align the picture
background	optional	Background	Widget background, use root widget background by default
button id	optional	uint32	Makes widget "pressable" and assign ID to distinguish it

Table 4.260: CustomerPicture

Field Name	Label	Type	Description
text	required	bytes	Text to generate QR code
module dimension	optional	uint32	Fixed module (dot) dimension in pixels
vertical alignment	optional	VerticalAlignment	How to vertically align the QR-code
horizontal alignment	optional	HorizontalAlignment	How to horizontally align the QR-code
background	optional	Background	QR-code background, black by default
foreground	optional	Foreground	QR-code foreground, white by default
error correction level	optional	ErrorCorrectionLevel	QR-code error correction level (ECC)
quiet zone size	optional	uint32	QR-code Quiet Zone size in the modules (dots)

Table 4.261: QrCode

Field Name	Label	Type	Description
text id	required	TextId	The text ID to show
prefix	optional	string	Optional prefix before generated text
vertical alignment	optional	VerticalAlignment	How to vertically align the text
horizontal alignment	optional	HorizontalAlignment	How to horizontally align the text
background	optional	Background	Widget background, use root widget background by default
foreground	optional	Foreground	Widget foreground color, use root widget foreground by default
font	optional	Font	Font for rendering text
button id	optional	uint32	Makes widget ”pressable” and assign ID to distinguish it

Table 4.262: GeneratedText

4.11.1.1 Features availability

Some features are available only from the certain protocol version:

Feature	Available since version
The `border` field of the `Text` widget	1.23
The `GeneratedText` widget	1.28
The `module_dimension` field of the `QrCode` widget	1.31
The `vertical_alignment` field of the `QrCode` widget	1.31
The `horizontal_alignment` field of the `QrCode` widget	1.31
The `background` field of the `QrCode` widget	1.33
The `foreground` field of the `QrCode` widget	1.33
The `error_correction_level` field of the `QrCode` widget	1.56
The `quiet_zone_size` field of the `QrCode` widget	1.56

4.11.1.2 Alignment

There are following alignments:

Value	Number	Description
CENTER VERTICALLY	0	Centers the element vertically
BOTTOM	1	Aligns the element to the bottom
TOP	2	Aligns the element to the top
OPTIMAL VERTICALLY	3	Minimizes vertical space usage of the element

Table 4.264: VerticalAlignment

Value	Number	Description
CENTER HORIZONTALLY	0	Centers the element horizontally

LEFT	1	Aligns the element to the left
RIGHT	2	Aligns the element to the right
OPTIMAL HORIZONTALLY	3	Minimizes horizontal space usage of the element

Table 4.265: HorizontalAlignment

4.11.1.3 Caching

This command "caches" widgets that were displayed previously and draws only those that were changed from previous request.

This feature can be used to speed-up rendering: if controller has changed only one widget on the screen (ex. only text string), then IR redraws only part of the screen that contains this text.

4.11.1.4 Background and foreground

By default background filled by the black solid color and foreground (text) filled by the white color. This can be changed on:

- per widget basis for `Text`, `Picture`, `CustomerPicture` and `QrCode` widgets
- per screen basis by parameters of the `ShowScreen` message

4.11.1.5 Built-in fonts

There are following built-in fonts:

Value	Number	Description
REGULAR FONT	0	Regular font for normal text, height 30 pixels
LARGE FONT	1	Large font for important text, height 56 pixels
MONOSPACE FONT	2	Monospace font, all symbols have the same WxH: 15x29

Table 4.266: Font

Both fonts contain the same characters with the same appearance. See `Annex C Built-in Fonts Metrics` for dimensions of each character.

`MONOSPACE_FONT` is available since v. 1.32.

4.11.1.6 Built-in pictures

There are following built-in pictures:

Value	Number	Description
APPROVED	0	Operation has been approved (accepted), 200x148
REJECTED	1	Operation has been rejected (declined), 148x148
EMV CONTACTLESS SYMBOL	2	EMV symbol means "place contacless cards here", 160x96
AWAITING	3	Awaiting something, 82x129
APPROVED DISCOUNT	4	Operation has been approved (accepted) with discount, 200x148
MASTERCARD LOGO	10	MasterCard logotype, 62x39
VISA LOGO	11	VISA logotype, 76x39
MIR LOGO	12	MIR payment card logotype, 83x39
UNIONPAY LOGO	13	UNIONPAY payment card logotype, 55x39

Table 4.267: PictureId

4.11.1.7 Customer pictures

It is possible to display pictures provided by the customer. Pictures must be provided in the `Reader Configuration` in advance.

Use `CustomerPicture` widget with `name` set to picture's filename in the `Reader Configuration`.

4.11.1.8 QR-code

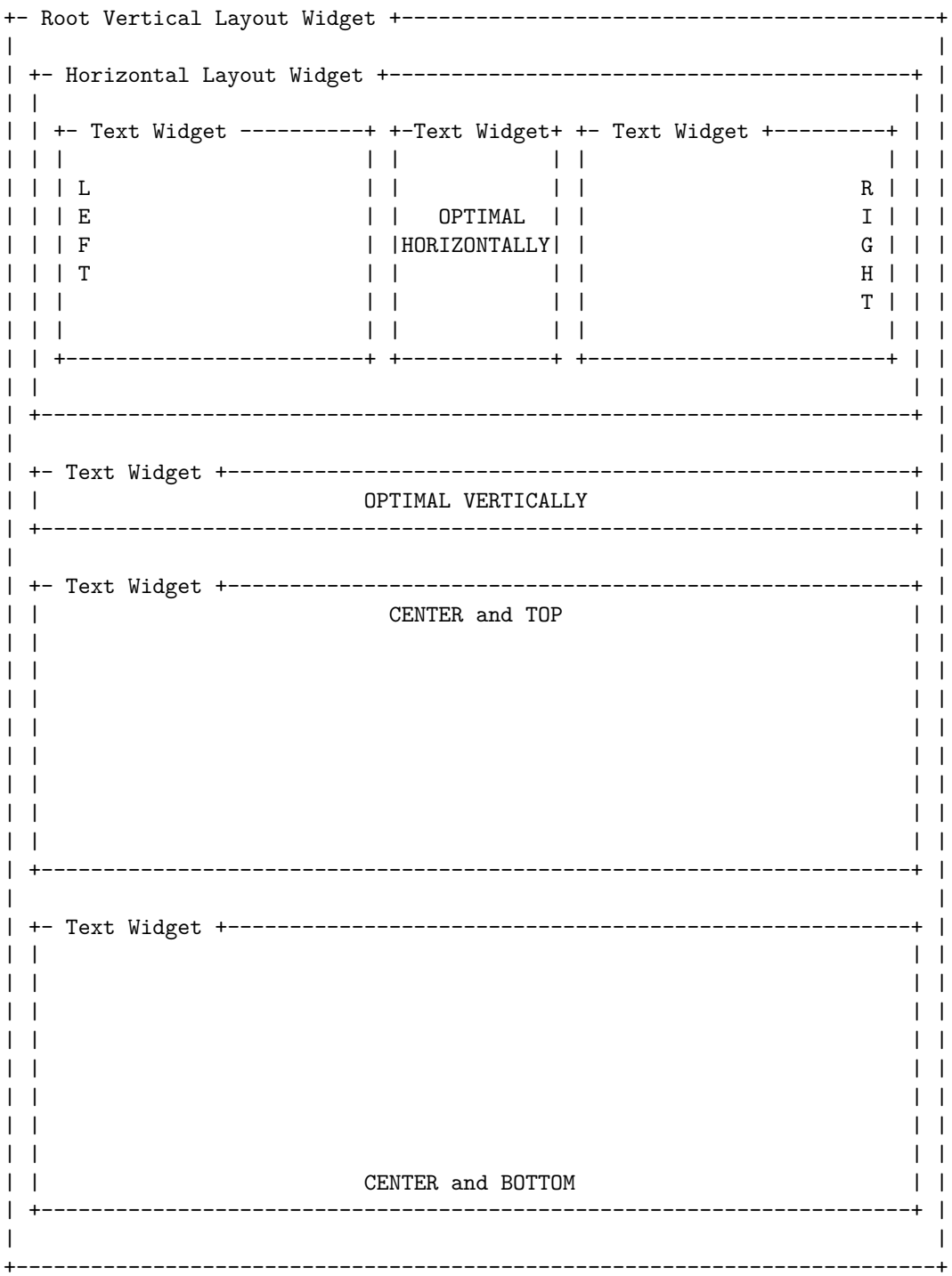
It is possible to display text information in QR-code format. Use `QrCode` widget with field `text` filled with information to be coded in QR and displayed.

Size and position of the widget can be configured by `module_dimension`, `vertical_alignment` and `horizontal_alignment` fields. Note that `module_dimension` field of the `QrCode` widget would be maximized if not provided.

4.11.1.9 Pressable widgets

It is possible to make `Text`, `Picture` and `CustomerPicture` widgets are "pressable" by setting the `button_id` field. Value of the `button_id` field may be arbitrary, but it should be meaningful for the controller.

4.11.1.10 Example Text positioning using vertical & horizontal Layouts:



4.11.1.11 Request

Request contains `ShowScreen` message:

Field Name	Label	Type	Description
root	required	Widget	Screen root widget
background	optional	Background	Screen background, SOLID FILL BLACK by default
foreground	optional	Foreground	Screen foreground color, SOLID FILL WHITE by default
border	optional	Border	Optional border around the display edge

Table 4.268: ShowScreen

The `border` field of the `ShowScreen` message is available since v. 1.29.

4.11.1.12 Response

No data carried with successful response.

4.11.1.13 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.11.1.14 Example

Show the screen with “TAP CARD” caption, contactless symbol at the center and payment systems line at the bottom:

```
show_screen {
    root {
        vertical_layout {
            widgets {
                text {
                    text: "TAP CARD"
                    font: LARGE_FONT
                }
            }
            widgets {
                picture {
                    picture_id: EMV_CONTACTLESS_SYMBOL
                }
            }
            widgets {
                horizontal_layout {
                    widgets {
                        picture {
                            picture_id: VISA_LOGO
                            vertical_alignment: BOTTOM
                            horizontal_alignment: LEFT
                        }
                    }
                    widgets {
                        picture {
                            picture_id: MASTERCARD_LOGO
                            vertical_alignment: BOTTOM
                        }
```

```
                    }
                    widgets {
                        picture {
                            picture_id: MIR_LOGO
                            vertical_alignment: BOTTOM
                            horizontal_alignment: RIGHT
                        }
                    }
                }
            }
        }
    }
}
```

4.11.2 Input screen dialog

Show the input screen and then interact with user to enter text.

The background color Hex code is `0xE8E8E8`.

This command is available from version 1.5.

4.11.2.1 Request

Request contains `InputDialog` message:

Field Name	Label	Type	Description
caption	required	string	Caption of the screen
placeholder	optional	string	Placeholder for text to be entered
timeout	optional	uint32	Inactivity timeout in seconds
min text length	optional	uint32	Minimal text length allowed to confirmed
max text length	optional	uint32	Maximum text length allowed to enter
layouts	repeated	KeypadLayout	Sequence of the layouts available for user. Default sequence: en -> EN -> 123 -> %;{} -> ru -> RU
hide input	optional	bool	Hide input text with '*' characters
large font	optional	bool	Use large font for the input text

Table 4.270: InputDialog

The `hide_input` and `large_font` fields as well as the `PINPAD` keypad layout are *available from version 1.44*.

4.11.2.2 Response

Reader returns `EnteredText` message when user press the 'OK' button:

Field Name	Label	Type	Description
text	required	string	Text entered by user

Table 4.271: EnteredText

If user does not touch the screen during time defined in `timeout` field reader returns failure response with `TIMEOUT_EXPIRED` error.

If user pressed "Cancel" key reader returns failure response with `CANCELLED_BY_CARDHOLDER` error.

4.11.2.3 Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Allow

4.11.2.4 Example

Show input dialog with caption "INPUT NAME":

```
input_dialog { caption: "INPUT NAME" }
```

4.11.3 Menu screen dialog

Show the menu screen and then interact with user to select an item.

Menu consists of leaf items and a tree of sub-menues with arbitrary nesting. When user selects a leaf item reader returns associated `id`. When user selects a sub-menu item reader shows new screen with sub-menu's items and waits for user action.

It's possible to create "Yes/No" like dialogue by providing only two leaf items with respective text.

The background color Hex code is `0xE8E8E8`.

This command is available from version 1.6.

Since version 1.34 it is possible to provide a `show_item_id` parameter to show the sub-menu that contains a widget with the `id`.

4.11.3.1 Request

Request contains `MenuDialog` message:

Field Name	Label	Type	Description
caption	required	string	The dialog caption
item list	required	ItemList	List of the items to show to the user
timeout	optional	uint32	Inactivity timeout in seconds
show item id	optional	uint32	Start from LeafItem with 'show item id' instead of root menu

Table 4.273: MenuDialog

The `caption` field may be multiline by separating lines by `\n`.

4.11.3.2 Response

Reader returns `SelectedItem` message when user selected a leaf item:

Field Name	Label	Type	Description
id	required	uint32	The ID of leaf item was selected

Table 4.274: SelectedItem

If user does not touch the screen during time defined in `timeout` field reader returns failure response with `TIMEOUT_EXPIRED` error.

If user pressed the "Back" key reader returns failure response with `CANCELLED_BY_CARDHOLDER` error.

4.11.3.3 Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Allow

4.11.3.4 Example

The menu tree:

```
item1 -> 1
item2 ->
         sub item1 -> 11
         sub item2 -> 12
         sub item3 ->
                      sub sub item1 -> 111
```

```
                    sub sub item2 -> 112
item3 -> 3
item4 -> 4
item5 -> 5
item6 -> 6
item7 -> 7
item8 -> 8
item9 -> 9
item10 -> 10
item11 -> 11
item12 -> 12
item13 -> 13
```

Legend: `menu item text -> item id`.

The command:

```
menu_dialog {
    caption: "Choose an item"
    item_list {
        items {
            text: "item1" leaf_item { id: 1 }
        }
        items {
            text: "item2"
            submenu {
                items { text: "sub item1" leaf_item { id: 11 } }
                items { text: "sub item2" leaf_item { id: 12 } }
                items {
                    text: "sub item3"
                    submenu {
                        items { text: "sub sub item1" leaf_item { id: 111 } }
                        items { text: "sub sub item2" leaf_item { id: 112 } }
                    }
                }
            }
        }
        items { text: "item3" leaf_item { id: 3 } }
        items { text: "item4" leaf_item { id: 4 } }
        items { text: "item5" leaf_item { id: 5 } }
        items { text: "item6" leaf_item { id: 6 } }
        items { text: "item7" leaf_item { id: 7 } }
        items { text: "item8" leaf_item { id: 8 } }
        items { text: "item9" leaf_item { id: 9 } }
        items { text: "item10" leaf_item { id: 10 } }
        items { text: "item11" leaf_item { id: 11 } }
        items { text: "item12" leaf_item { id: 12 } }
        items { text: "item13" leaf_item { id: 13 } }
    }
}
```

4.11.4 Draw bitmap

This command allows to draw arbitrary bitmap on the reader screen.

It is possible to draw your own GUI in a virtual framebuffer and send frames as bitmaps to the reader for displaying. To get touch events from the user use `Poll touchscreen for an event` command. The most perfomance can be achieved when using GUI library such as LVGL that re-draws only changed area of the whole screen. Ask the support to get integration demo that used LVGL GUI library.

If data to draw exceedes 4096 bytes then split it into the chunks and send multiple commands:

1. Set the `area` field to desired area and send the first chunk of the data
2. Clear the `area` field and send the second chunk of the data
3. Send remained chunks of the data

This command is available from version 1.8.

4.11.4.1 Request

Request contains `DrawBitmap` message:

Field Name	Label	Type	Description
data	required	bytes	Compressed data to draw
area	optional	Area	Area on the LCD where to draw data

Table 4.276: DrawBitmap

`data` field must contains compressed `RGB888` data. Compression algorithm is `DEFLATE` with default compression rate (`6`).

4.11.4.2 Response

No data carried with successful response.

4.11.4.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.11.5 Slideshow

This command allows to show specially prepared image as separated slides in a 'slideshow'. It's possible to show simple animated picture on the reader's screen.

Frames within picture should be vertically "glued":

```
+-------+
|Frame 1|
+-------+
|Frame 2|
+-------+
|  ...  |
+-------+
|Frame N|
+-------+
```

Picture must be prepared and loaded as a regular customer picture, see How to Convert GIF image.

Count of frames within picture should set in `frames_count` field of the message.

If necessary a delay between frames may be defined using `delay_ms` field of the message.

This command is available from version 1.16.

4.11.5.1 Request

Request contains `Slideshow` message:

Field Name	Label	Type	Description
name	required	string	The customer picture name to show
background	required	Background	Border background
frames count	required	uint32	How many frames are within picture
delay ms	optional	uint32	Delay in milliseconds between frames
vertical alignment	optional	VerticalAlignment	How to vertically align the picture
horizontal alignment	optional	HorizontalAlignment	How to horizontally align the picture

Table 4.278: Slideshow

4.11.5.2 Response

No data carried with successful response.

4.11.5.3 Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Ignore

4.11.5.4 Example

Show "Visa Sensory Branding" slideshow that loaded within example's configuration:

```
slideshow {
    name: "visa.bmp"
    background { solid_fill: SOLID_FILL_WHITE }
    frames_count: 33
    delay_ms: 40
}
```

4.12 Touchscreen module

Each message should contains *one of* the following message types:

Field Name	Label	Type	Description
poll touchscreen	optional	PollTouchscreen	Poll touchscreen for an event

Table 4.280: Touchscreen

Module ID: **0Bh**.

4.12.1 Poll touchscreen for an event

This command has two modes defined by the `event_type` field.

4.12.1.1 Polling for GUI widget to be pressed

In this mode the command works in conjunction with GUI module.

It returns value of `button_id` field of the GUI widget that has been "pressed" by the cardholder. See "Show user-defined screen" command for more information.

4.12.1.2 Polling for press in arbitrary point of the screen

In this mode the command returns `(x, y)` coordinates on the screen as soon as cardholder pressed the touchscreen surface.

4.12.1.3 Request

Request contains `PollTouchscreen` message:

Field Name	Label	Type	Description
event type	required	EventType	Polling for one of the event types
timeout	optional	uint32	Time to polling touchscreen

Table 4.281: PollTouchscreen

4.12.1.4 Response

Successful response contains `TouchscreenEvent` message:

Field Name	Label	Type	Description
widget id	optional	WidgetId	GUI widget has been pressed
coordinates	optional	Coordinates	(x,y) coordinates on the touchscreen

Table 4.282: TouchscreenEvent

Which event will be returned is determinated by the `event_type` field of the request.

4.12.1.5 Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Allow

4.12.1.6 Example

Poll the touchscreen for the `5000` milliseconds:

```
poll_touchscreen { event_type: WIDGET_ID timeout: 5000 }
```

4.13 Complex module

This module intended to combine requests to various reader interfaces into signle command.

Each message should contains *one of* the following message types:

Field Name	Label	Type	Description
poll for event	optional	PollForEvent	Poll multiple reader interfaces for an event

Table 4.284: Complex

Module ID: **0Ch**.

4.13.1 Poll for an event

This command combines requests to different modules into single command.

Since version 1.26 it is possible to `show the screen` and `set the LEDs`:

- On the start of the polling
- Upon an occurred event

Since version 1.28 it is possible to run a command if Mifare Classic card was presented during contactless card polling

The supported actions are:

Field Name	Label	Type	Description
troika av3 read ticket	optional	ReadTicket	Reads logical ticket out of the Troika card

Table 4.285: MifareCardAction

IntelliReader treats card as a Mifare Classic card if all conditions are true:

- the `type` field of the `Token` is any of:
  - `MIFARE_CLASSIC_1K`
  - `MIFARE_CLASSIC_2K`
  - `MIFARE_CLASSIC_4K`
  - `SMART_MX_WITH_MIFARE_1K`
  - `SMART_MX_WITH_MIFARE_4K`
- the `answer_to_select` field of the `Token` is absent

4.13.1.1 Request

Request contains `PollForEvent` message:

Field Name	Label	Type	Description
poll for token	optional	PollForToken	Poll for contactless token
perform txn	optional	PerformTransaction	Poll for contactless token and then perform transaction
poll touchscreen	optional	PollTouchscreen	Poll touchscreen for pressed GUI widget
poll for qrcode	optional	PollForQrCode	Poll for QR-code/Barcode
on start ui action	optional	UserInterface	Perform an user interface action on the start of the polling
on event ui action	optional	UserInterface	Perform an user interface action if an event is occured
on mifare card present	optional	MifareCardAction	Perform an action if a Mifare card has been presented

Table 4.286: PollForEvent

The `poll_for_qrcode` field is available from version 1.3.

4.13.1.2 Response

Successful response contains `Event` message with one of the following field:

Field Name	Label	Type	Description
contactless token	optional	Token	Contactless token has been detected
contactless txn result	optional	TransactionResult	Contactless Transaction has been performed
touchscreen event	optional	TouchscreenEvent	GUI widget has been pressed
qrcode	optional	QrCodeEvent	QR-code/Barcode has been detected
troika ticket	optional	Ticket	Logical ticket of the Troika card

Table 4.287: Event

4.13.1.3 Auxiliary messages

Message	Comments
Pending	Return
Notification	Depends on combined commands
Cancellation	Allow

4.13.1.4 Example

Poll for contactless token (EMV transaction will be occur if token detected) as well as touchscreen:

```
poll_for_event {
    perform_txn {
        poll_for_token {
            timeout: 10000
            prefer_mifare: false
            poll_stm_sri512: true
        }
        amount_authorized: 100
        transaction_date: "\x13\x06\x13"
        transaction_type: "\x00"
        terminal_country_code: "\x06\x43"
        transaction_time: "\x12\x00\x00"
        transaction_currency_code: "\x06\x43"
    }
    poll_touchscreen { event_type: WIDGET_ID }
}
```

4.14 QR-code module

Certain hardware models may be equipped by the QR-code/Barcode reader. This module provides commands to interact with them.

Each message should contains *one of* the following message types:

Field Name	Label	Type	Description
poll for qrcode	optional	PollForQrCode	Poll for QR-code event
send command	optional	SendCommand	Send a command to the scanner

Table 4.289: QrCode

Module ID: **0Dh**.

Available from version 1.3.

4.14.1 Poll for QR-code

This command enables QR-code reader to scan for QR-code and awaiting until code will be present or timeout occurs.

Since version 1.36 IntelliReader supports EMV QR code in Consumer Presented Mode. If reader detects EMV QR code, it validates QR code contents and returns EMV tags extracted from the code. To enable EMV QR processing set `EventType` field to `EMV_QR_CODE`.

4.14.1.1 Request

Request contains `PollForQrCode` message:

Field Name	Label	Type	Description
event type	optional	EventType	Polling for one of the event types
timeout	optional	uint32	Time to polling
read indication	optional	Indication	Indication type of successful read event

Table 4.290: PollForQrCode

The `event_type` field may be set to the following values:

Value	Number	Description
RAW DATA	0	QR-code/Barcode raw data as is
EMV QR CODE	1	EMV Consumer Presented QR Code

Table 4.291: EventType

The `read_indication` field may be set to the following values:

Value	Number	Description
ONETIME BEEP	0	One-time sound at 2700Hz for 200ms
NONE	1	Disable indication

Table 4.292: Indication

The `read_indication` field is available since v. 1.43.

4.14.1.2 Response

Successful response contains `QrCodeEvent` message with one of the following field:

Field Name	Label	Type	Description
raw data	optional	RawData	QR-code/Barcode raw data as it has been received from scanner
validation result	optional	ValidationResult	Result of EMV Code QR validation along with EMV tags extracted from QR Code

Table 4.293: QrCodeEvent

4.14.1.3 Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Allow

4.14.1.4 Example

Poll for QR-code with timeout 15 seconds:

```
poll_for_qrcode { timeout: 15000 }
```

4.14.2 Send command to the QR-scanner

This command allows to send a command to the QR-scanner.

Please note that Yoko E4000 may no answer on the first command if it was entered into the lower power mode.

Available since version 1.25.

4.14.2.1 Request

Request contains `SendCommand` message:

Field Name	Label	Type	Description
command	required	bytes	Scanner command
timeout	required	uint32	Timeout in milliseconds to receive scanner's answer

Table 4.295: SendCommand

4.14.2.2 Response

Successful response contains `ScannerReply` message:

Field Name	Label	Type	Description
reply	required	bytes	Reply received from the scanner

Table 4.296: ScannerReply

4.14.2.3 Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Ignore

4.14.2.4 Example

Read the firmware version of Yoko E4000:

```
send_command {
    command: "\x57\x00\x27\x20\x00\x00\x06\x99\x50\x00"
    timeout: 1000
}
```

4.15 CIPURSE cards module

This module contains commands to work with CIPURSE™V2-compliant PICCs. *CIPURSE* is an open security standard for transit fare collection systems. Specifications can be get for free from *OSPT Alliance* organisation.

Each message should contains *one of* the following message types:

Field Name	Label	Type	Description
authorize sam	optional	AuthorizeHwSam	Unlock CIPURSE HW SAM in the reader
select file	optional	SelectFile	Select file on the CIPURSE card
establish secure channel	optional	EstablishSecureChannel	Establish secure channel between CIPURSE card and the reader
read binary	optional	ReadBinary	Read binary record content
update binary	optional	UpdateBinary	Update binary record content
read record	optional	ReadRecord	Read the content of the linear record(s)
update record	optional	UpdateRecord	Update the content of the linear record(s)
append record	optional	AppendRecord	Append new cyclic record
read value	optional	ReadValue	Read the value of the value-record(s)
modify value	optional	ModifyValue	Increment or decrement the value of the record-value

Table 4.298: Cipurse

Module ID: **0Eh**.

Available from version 1.4.

4.15.1 Authorize HW SAM

Unlocks CIPURSE hardware SAM by providing the authorization password. Internally this command selects SAM application and then performs `VERIFY_SAM_PASSWORD` command defined in CIPURSE™V2 SAM Specification.

Before execute this command CIPURSE SAM must be successfully powered by the `Power on` command.

4.15.1.1 Request

Request contains `AuthorizeHwSam` message:

Field Name	Label	Type	Description
slot	required	CardSlot	Card slot in the reader
aid	required	bytes	CIPURSE SAM Application ID (ADF name)
password	optional	bytes	SAM password (use default if absent)

Table 4.299: AuthorizeHwSam

4.15.1.2 Response

No data carried with successful response.

4.15.1.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.15.1.4 Example

Select SAM application (AID: `D276000004FF00000000000000000001`) on the CIPURSE SAM in the `SAM3_SLOT` and then unlock it using default password:

```
authorize_sam {
    slot: SAM3_SLOT
    aid: "\xd2\x76\x00\x00\x04\xff\x00\x00\x00\x00\x00\x00\x00\x00\x00\x01"
}
```

4.15.2 Select File

Select one of the following files on the CIPURSE-compatible PICC:

- MF, DF or EF by FileID
- ADF by DF Name (AID)

4.15.2.1 Request

Request contains `SelectFile` message:

Field Name	Label	Type	Description
aid	optional	bytes	Application ID
file id	optional	uint32	File ID

Table 4.301: SelectFile

4.15.2.2 Response

No data carried with successful response.

4.15.2.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.15.2.4 Example

Select application (ADF) by AID `A0000005070100`:

```
select_file { aid: "\xa0\x00\x00\x05\x07\x01\x01" }
```

4.15.3 Establish Secure Channel

Establish secure channel between CIPURSE card and the reader using CIPURSE hardware SAM. CIPURSE SAM must be authorized (unlocked) in advance using the `Authorize HW SAM` command.

4.15.3.1 Request

Request contains `EstablishSecureChannel` message:

Field Name	Label	Type	Description
key number	required	uint32	Number of key in the current file
sam key id	required	uint32	Either master key for key diversification or direct key if no key diversification
diversification data	optional	bytes	Diversification data (leave empty to use most recent div. data)
slot	optional	CardSlot	Card slot in the reader with CIPURSE SAM (use last authorized if absent)
security level	optional	SecurityLevel	Communication security level for transferring data between PCD and PICC

Table 4.303: EstablishSecureChannel

`security_level` field is available since v. 1.19.

4.15.3.2 Response

No data carried with successful response.

4.15.3.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.15.3.4 Example

Establish secure channel using:

- key #1 in the current file
- direct key #3 on the CIPURSE SAM
- use key diversification data

```
establish_secure_channel {
    key_number: 1
    sam_key_id: 3
    diversification_data: "\x05\x02\x05\x85\x55\xb2\x5c\x00\x00\xff\x5f\x7c\x69\x00\x00\x00"
}
```

4.15.4 Read Binary Record

Reads out content from the binary record.

4.15.4.1 Request

Request contains `ReadBinary` message:

Field Name	Label	Type	Description
short file id	required	uint32	Short File ID to be read
length	required	uint32	How many bytes read
offset	optional	uint32	File offset if any

Table 4.305: ReadBinary

4.15.4.2 Response

Successful response contains `Binary` message:

Field Name	Label	Type	Description
data	required	bytes	Binary record data

Table 4.306: Binary

4.15.4.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.15.4.4 Example

Read sixteen (`512`) bytes of the first record of SFID #1:

```
read_binary { short_file_id: 1 length: 512 }
```

4.15.5 Update Binary Record

Updates data within binary record.

4.15.5.1 Request

Request contains `UpdateBinary` message:

Field Name	Label	Type	Description
short file id	required	uint32	Short File ID to be read
data	required	bytes	Bytes to be write
offset	optional	uint32	File offset if any

Table 4.308: UpdateBinary

4.15.5.2 Response

No data carried with successful response.

4.15.5.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.15.5.4 Example

Update the first binary record of SFID #1:

```
update_binary {
    short_file_id: 1
    data: "
    \xde\xad\xbe\xef\x00\x00\x00\x00\xca\xfe\xfa\xde\x00\x00\x00\x00
```

```
    \xde\xad\xbe\xef\x00\x00\x00\x00\xca\xfe\xfa\xde\x00\x00\x00\x00
    \xde\xad\xbe\xef\x00\x00\x00\x00\xca\xfe\xfa\xde\x00\x00\x00\x00
    \xde\xad\xbe\xef\x00\x00\x00\x00\xca\xfe\xfa\xde\x00\x00\x00\x00
    \xde\xad\xbe\xef\x00\x00\x00\x00\xca\xfe\xfa\xde\x00\x00\x00\x00
    "
}
```

4.15.6 Read Linear Record

Reads one or multiple linear records.

4.15.6.1 Request

Request contains `ReadRecord` message:

Field Name	Label	Type	Description
short file id	required	uint32	Short File ID
length	required	uint32	How many bytes read
first record	optional	uint32	Number of the first record (in the range of 1 to 254)
read multiple records	optional	bool	Read multiple records at once

Table 4.310: ReadRecord

4.15.6.2 Response

Successful response contains `Record` message:

Field Name	Label	Type	Description
data	required	bytes	Linear record(s) data

Table 4.311: Record

4.15.6.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.15.6.4 Example

Read out 64 bytes starting from the first record of SFID #2:

```
read_record { short_file_id: 2 length: 64 first_record: 1 read_multiple_records: true }
```

4.15.7 Update Linear Record

Update one or multiple linear records.

4.15.7.1 Request

Request contains `UpdateRecord` message:

Field Name	Label	Type	Description
short file id	required	uint32	Short File ID
data	required	bytes	Data to be update
first record	optional	uint32	Number of the first record (in the range of 1 to 254)
update multiple records	optional	bool	Update multiple records at once

Table 4.313: UpdateRecord

4.15.7.2 Response

No data carried with successful response.

4.15.7.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.15.7.4 Example

Update the first linear record of the SFID #2:

```
update_record {
    short_file_id: 2
    data: "
    \xde\xad\xbe\xef\x00\x00\x00\x00\xca\xfe\xfa\xde\x00\x00\x00\x00
    \xde\xad\xbe\xef\x00\x00\x00\x00\xca\xfe\xfa\xde\x00\x00\x00\x00
    \xde\xad\xbe\xef\x00\x00\x00\x00\xca\xfe\xfa\xde\x00\x00\x00\x00
    \xde\xad\xbe\xef\x00\x00\x00\x00\xca\xfe\xfa\xde\x00\x00\x00\x00
    "
    first_record: 1
    update_multiple_records: true
}
```

4.15.8 Append New Cyclic Record

Appends new record to list of the cyclic records.

4.15.8.1 Request

Request contains `AppendRecord` message:

Field Name	Label	Type	Description
short file id	required	uint32	Short File ID
data	required	bytes	Record data

Table 4.315: AppendRecord

`data`'s length must match cyclic record's length.

4.15.8.2 Response

No data carried with successful response.

4.15.8.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.15.8.4 Example

Append new record to SFID #4:

```
append_record {
    short_file_id: 4
    data: "
    \xde\xad\xbe\xef\x00\x00\x00\x00\xca\xfe\xfa\xde\x00\x00\x00\x00
    \xde\xad\xbe\xef\x00\x00\x00\x00\xca\xfe\xfa\xde\x00\x00\x00\x00
    \xde\xad\xbe\xef\x00\x00\x00\x00\xca\xfe\xfa\xde\x00\x00\x00\x00
    "
}
```

4.15.9 Read Value-Record

Reads out one or multiple values of the value-record(s).

4.15.9.1 Request

Request contains `read_value` message:

Field Name	Label	Type	Description
short file id	required	uint32	Short File ID
count	required	uint32	How many values read
first record	optional	uint32	Number of the first record (in the range of 1 to 254)

Table 4.317: ReadValue

4.15.9.2 Response

Successful response contains `Values` message:

Field Name	Label	Type	Description
values	repeated	sint32	Value-record(s) value(s)

Table 4.318: Values

4.15.9.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.15.9.4 Example

Read single value from the first record of SFID #3:

```
read_value { short_file_id: 3 count: 1 first_record: 1 }
```

4.15.10 Modify Value-Record

Modifies (increment or decrement) value of the value-record.

4.15.10.1 Request

Request contains `ModifyValue` message:

Field Name	Label	Type	Description
short file id	required	uint32	Short File ID
operand	required	sint32	Modification operand. If value is positive then increment otherwise decrement original value
record	optional	uint32	Number of the record (in the range of 1 to 254)

Table 4.320: ModifyValue

4.15.10.2 Response

Successful response contains `NewValue` message:

Field Name	Label	Type	Description
value	required	sint32	

Table 4.321: NewValue

4.15.10.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.15.10.4 Example

Increment current value of the first record in the SFID #3 by one (`1`):

```
modify_value { short_file_id: 3 operand: 1 record: 1 }
```

4.16 Turnstile control module

This module contains commands to control a turnstile.

Each message should contains *one of* the following message types:

Field Name	Label	Type	Description
allow one pass	optional	AllowOnePass	Allows one pass through turnstile

Table 4.323: Turnstile

Module ID: **10h**.

Available from version 1.11.

4.16.1 Allow one pass

This command opens the turnstile door, waits for feedback from a turnstile that passenger has been passed, and then closes the door. If passanger did not passed during `timeout_ms` then reader closes the doors and returns `TIMEOUT_EXPIRED` error.

4.16.1.1 Request

Request contains `AllowOnePass` message:

Field Name	Label	Type	Description
timeout ms	required	uint32	Max time for passing through turnstile
hold level	optional	HoldLevel	Hold active level to keep turnstile openned (used by default)
one impulse	optional	OneImpulse	Send one short signal impulse to open turnstile

Table 4.324: AllowOnePass

4.16.1.2 Response

No data carried with successful response.

4.16.1.3 Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Ignore

4.17 Troika transport card module

This module contains commands specific to the Troika transit card.

Each message should contains *one of* the following message types:

Field Name	Label	Type	Description
troika av3 read ticket	optional	ReadTicket	Reads logical ticket out of the Troika card
troika av3 write ticket	optional	WriteTicket	Prepares bitmap from parameters and writes bitmap on the card

Table 4.326: TroikaCard

Module ID: **12h**.

Available from version 1.28.

4.17.1 Read logical ticket

The command gets list of the sectors on the card (along with authentication parameters), current date & time and uses the steps to find out a valid ticket on the card:

1. Choose the sector from the list in the following order
2. Authenticates to the chosen sector
3. If fail then return to step 1
4. Reads out the first three (3) blocks of the sector
5. If fail then return to step 1
6. Passes the blocks to the programmable logic in NXP SAM AV3
7. If fail then return to step 1
8. If a logical ticket has a success status then return it
9. Otherwise return to step 1

IntelliReader makes three (3) attempts to authenticate a card and to read data. If no ticket found then the error will be returned.

4.17.1.1 Request

Request contains `ReadTicket` message:

Field Name	Label	Type	Description
date	required	Date	Current date
time	required	Time	Current time
sectors	repeated	SamKey	Sectors to looking for ticket

Table 4.327: ReadTicket

4.17.1.2 Response

Successful response contains `Ticket` message:

Field Name	Label	Type	Description
sector number	required	uint32	Number of Mifare Classic sector
ticket data	required	bytes	Packed logical ticket (Troika proprietary format)

Table 4.328: Ticket

4.17.1.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.17.2 Validate ticket and write bitmap

The command validates the ticket against provided parameters, prepares and writes the sector bitmap to the card.

IntelliReader make three (3) attempts to authenticate a card and to write a sector bitmap to the card.

4.17.2.1 Request

Request contains `WriteTicket` message:

Field Name	Label	Type	Description
sector	required	SamKey	Sector where ticket should be written
ticket params	required	bytes	Ticket parameters (Troika proprietary format)
validate params	required	bytes	Validate parameters (Troika proprietary format)

Table 4.330: WriteTicket

4.17.2.2 Response

Successful response contains `WriteResult` message:

Field Name	Label	Type	Description
sector bitmap	required	bytes	Sector bitmap (48 bytes)
write ok	required	bool	Whether the sector bitmap is successfully written on the card

Table 4.331: WriteResult

IntelliReader returns response even in the case when the sector bitmap has not been successfully written to the card.

The `write_ok` field shows the status of the write.

Controller can repeat the write of the sector bitmap using the `Mifare Classic Write Multiple Sectors` command later.

4.17.2.3 Auxiliary messages

Message	Comments
Pending	Never return
Notification	Never return
Cancellation	Ignore

4.18 Speaker module

Each message should contains *one of* the following message types:

Field Name	Label	Type	Description
play file	optional	PlayFile	Plays audio from file on the speaker
speaker test	optional	SpeakerTest	Test the speaker

Table 4.333: Speaker

Module ID: **13h**.

Available from version 1.55.

4.18.1 Play audio file

This command plays audio file from the reader's internal storage. The file should be loaded to the reader in advance using the `Upload reader configuration` command.

Currently supported audio file format is MP3, see Customer MP3 audio files chapter for more information.

4.18.1.1 Request

Request contains `PlayFile` message:

Field Name	Label	Type	Description
filename	required	string	Filename of the file to play

Table 4.334: PlayFile

4.18.1.2 Response

No data carried with successful response.

4.18.1.3 Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Ignore

4.18.2 Speaker test

This command allows to test the speaker by playing a test sound.

4.18.2.1 Request

Request contains the `SpeakerTest` message:

Field Name	Label	Type	Description
sine wave test	optional	SineWaveTest	Test the speaker by playing a sine wave

Table 4.336: SpeakerTest

Field Name	Label	Type	Description
frequency hz	required	uint32	Frequency of the sine wave in Hz
duration secs	required	uint32	Duration of the sine wave in seconds

Table 4.337: SineWaveTest

4.18.2.2 Response

No data carried with successful response.

4.18.2.3 Auxiliary messages

Message	Comments
Pending	Return
Notification	Never return
Cancellation	Ignore

4.19 Failure response

When the reader can not successfully process incoming request, it replies with `failure` response:

Field Name	Label	Type	Description
error	required	Error	Error causes failure
error string	optional	string	Carries additional context to error as arbitrary text
mifare error at	optional	MifareErrorAt	Additional information in case of Mifare error

Table 4.339: FailureResponse

Optional `error_string` that carried along with `error` can be used to investigate the cause of the `error`.

Optional `mifare_error_at` field available since v. 1.49.

List of the possible errors:

Value	Number	Description
GENERAL FAIL	1	Any failure that does not described by certain error
NFC ERROR	2	NFC (contactless) module error
CHECKSUM MISMATCH	3	Incoming message have wrong checksum
PAYLOAD PARSING ERROR	4	Incoming message's payload parsing is failed
TRUNCATED MESSAGE	5	Incoming message truncated (due to intercharacter timeout or whole message timeout)
MESSAGE TOO BIG	6	Incoming message too big
UNEXPECTED HEADER	7	First two bytes of message are not equal to 'IR'
UNEXPECTED COMMAND	8	IR module receives unexpected command
CLESS CARDS COLLISION ERROR	9	Contactless cards collision occurred
TIMEOUT EXPIRED	10	Timeout defined for command is expired
PROCESSING STOPPED	11	Command processing has been stopped by 'Control' message
CONFIGURATION ERROR	12	Unable to process command due to configuration error
CARD TIMEOUT	13	No answer from the card
UNSUPPORTED COMMAND	14	IR module receives unsupported command
INVALID PARAMETER	15	One (or more) parameters in request invalid
NFC TRANSMISSION ERROR	16	An error occurred on transmitting or receiving NFC (Contactless) frame
MIFARE CARD NOT ACKNOWLEDGE COMMAND	17	Mifare card returns NAK to the reader's command
CANCELLED BY CARDHOLDER	18	Command cancelled by cardholder (end user of device)
SECURITY FUNCTIONS RESTRICTED	19	Security functions are restricted by the device's security monitor
MFP AUTHENTICATION ERROR	20	Mifare Plus card authentication error
ICC IS NOT PRESENT	21	Contact Card (ICC) is not present (inserted) in the reader
ICC IS NOT READY	22	Contact Card (ICC) is not ready to work (probably not powered)
ICC TIMEOUT	23	No answer from the Contact Card (ICC)
ABSENT HARDWARE MODULE	24	Command requires an absent hardware module
SAM AV2 CONDITIONS NOT SATISFIED	25	Mifare SAM AV2 conditions of use not satisfied
SAM AV2 AUTHENTICATION ERROR	26	Mifare SAM AV2 authentication error
MFC AUTHENTICATION ERROR	27	Mifare Classic authentication error
RETRY LIMIT IS OUT	28	Limit to retry operation is out
RETRY TIME IS OUT	29	Time to retry operation is out

INTELLIREADER IS BUSY	30	IntelliReader device is busy due to processing previous request
AT LEAST ONE ACTION REQUIRED	31	Poll For Event requires at least one action
UNSUPPORTED FEATURE	32	Command requires unsupported feature
ABSENT QRCODE HARDWARE	33	QR-code hardware is absent or broken
OPERATION IN PROGRESS	34	Background operation is in progress, try again later
QRCODE HARDWARE ERROR	35	QR-code hardware error

Table 4.340: Error

4.20 Notification message

Processing of some requests can be performed along with optional `notification` messages.

Each `notification` message always contains *one of* the following message types:

Field Name	Label	Type	Description
log message	optional	LogMessage	
user message	optional	UserMessage	
kernel message	optional	KernelMessage	
secure entry status	optional	SecureEntryStatus	

Table 4.341: Notification

4.20.1 Log message

Debug-like messages not intended to be present to the user.

Field Name	Label	Type	Description
level	required	Importance	
msg	required	string	

Table 4.342: LogMessage

`Log message` have few *severity* levels to distinguish various messages by it importance:

Value	Number	Description
EMERGENCY	1	
ALERT	2	
CRITICAL	3	
ERROR	4	
WARNING	5	
NOTICE	6	
INFO	7	
DEBUG	8	

Table 4.343: Importance

4.20.2 User message

Notifications that can be translated to the user if it is possible in the current Host and necessary by the current business logic.

Field Name	Label	Type	Description
message id	required	MessageId	

Table 4.344: UserMessage

Each message have distinct identification:

Value	Number	Description
PRESENT ONE CARD ONLY	0	Contactless collision detected, present one card only

Table 4.345: MessageId

Chapter 5

Best Practices

This chapter describes best practices for various typical scenarios.

5.1 Hardware diagnostic

This chapter describes how to check *IR* hardware against errors.

All hardware errors can be splitted on two parts:

1. Peripheral hardware used in the *IR* device
2. Physical connection between *IR* and *Controller*

There are two approaches to diagnose hardware. *Controller* can implement only one or both simultaneously.

5.1.1 Periodical diagnostic

Controller may periodically (ex. once per hour) request diagnostic information from the *IR*.

Use the `Self test` command to retrieve information about hardware errors.

Use `Get device statistic` command to retrieve information about errors detected on the physical interface used for `TCP/IP` connection.

Controller should collects data that it got by these commands and upload it to the back-office (aka management) service for the following analysis by the technical staff.

5.1.2 On demand diagnostic

Controller may provide some way for technical staff to manually run diagnostic actions and display the result.

To check physical connection used for `TCP/IP` *Controller* can transmit 1Mbyte random data through existing connection and then check *IR*'s `TCP/IP` stack statistic regarding errors.

Steps:

1. Send `Echo` command 256 times. The `data` field of the command should contain 4096 random bytes. Optionally *Controller* may check round trip time (RTT) of the `Echo` request. For Ethernet RTT is about 50 milliseconds.
2. Send `Get device statistic` command and then checkout the `lwip` field of the response message.

To check peripheral hardware use the `Self test` command.

5.2 TCP-keepalive and re-connection

Note: this chapter is only applicable when `TCP/IP` connection is used.

IR utilizes `TCP` feature named `TCP-keepalive` (see Wikipedia for more information) to detect physical connection break. If *IR* detects that connection is broken then it closes `TCP` port from it's side, shows "AWAITING CONNECTION" screen, turns the red LED and awaiting new connection.

Controller can use following approaches to detect that connection has been closed:

- Implement `TCP-keepalive` feature for its side
- OR continuously polling *IR* by some command, ex. `Echo`

Anyway, if *Controller* detected that `TCP` connection has been closed then it must close existed connection and then establish new one.

5.3 Broken connection diagnostic for USB/RS232 port

When hardware have only USB/RS232 connection *IR* enables a timer for incoming commands.

If *IR* doesn't get any command from the USB/RS232 port during 20 seconds it shows "AWAITING CONNECTION" screen and turns the red LED. That timer starts running after *IR* returned response to the *Host*.

5.4 How to customize "AWAITING CONNECTION" screen

It possible to customize the "AWAITING CONNECTION" screen by one of two approaches. Files should be uploaded within `Reader Configuration`.

5.4.1 Custom picture

To show a customer picture instead of default "AWAITING CONNECTION" screen you need to upload your picture under the `awaiting_connection.bmp` name.

5.4.2 Build custom screen

It is possible to show arbitrary screen instead of default "AWAITING CONNECTION" screen. You need to upload a binary file named `awaiting_connection.bin` that contains a binary encoded protobuf data for the `ShowScreen` message.

5.5 How to prevent display flickering

When terminal is polling for contactless card, RF field from the NFC antenna may affect the display picture.

That is observed like a little display flickering and doesn't mean any failure. Flickering is less visible when GUI styled in dark colors / soft tint.

To prevent flickering at all, use special low power polling mode. It can be enabled by setting `polling_mode` parameter to the `LOW_POWER_POLLING`.

Please note, low power polling may slightly decrease card detection range.

Chapter 6

Annex A Protobuf Scalar Value Types

ProtoType	C++ type	C# type	Go type	Java type	Python type	Notes
double	double	double	float64	double	float	
float	float	float	float32	float	float	
int32	int32	int	int32	int	int	Uses variable-length encoding. Inefficient for encoding negative numbers – if your field is likely to have negative values, use sint32 instead.
int64	int64	long	int64	long	int/long	Uses variable-length encoding. Inefficient for encoding negative numbers – if your field is likely to have negative values, use sint64 instead.
uint32	uint32	uint	uint32	int	int/long	Uses variable-length encoding.
uint64	uint64	ulong	uint64	long	int/long	Uses variable-length encoding.
sint32	int32	int	int32	int	int	Uses variable-length encoding. Signed int value. These more efficiently encode negative numbers than regular int32s.
sint64	int64	long	int64	long	int/long	Uses variable-length encoding. Signed int value. These more efficiently encode negative numbers than regular int64s.
fixed32	uint32	uint	uint32	int	int	Always four bytes. More efficient than uint32 if values are often greater than 2^28.
fixed64	uint64	ulong	uint64	long	int/long	Always eight bytes. More efficient than uint64 if values are often greater than 2^56.
sfixed32	int32	int	int32	int	int	Always four bytes.
sfixed64	int64	long	int64	long	int/long	Always eight bytes.
bool	bool	bool	bool	boolean	boolean	
string	string	string	string	String	str/unicode	A string must always contain UTF-8 encoded or 7-bit ASCII text.
bytes	string	ByteString	[]byte	ByteString	str	May contain any arbitrary sequence of bytes.

Table 6.1: Protobuf Scalar Value Types

Chapter 7

Annex B Reader Configuration

7.1 Overview

The reader configuration is the ZIP-archive with some files, max size is 1920 KiBytes.

There are following restrictions applied to the ZIP-archive:

1. ZIP64 format is not supported
2. Only “Deflate” compression method is supported
3. “Local File Headers” of the ZIP-archive must contains actual information
4. All “Local File Headers” must not use “Data Descriptor” feature
5. Subfolders are not supported
6. Filenames must be UTF8 encoded

Example content of the ZIP-archive:

- `awaiting_connection.bmp`
- `luggage.bmp`

7.2 Customer pictures

The reader configuration may contains pictures in the BMP format.

There are following restrictions applied to the pictures:

1. No compression method must be used
2. Pixels must be encoded in the “Red8, Green8, Blue8” (RGB888 or RGB24) format
3. Image must be flipped upside down
4. Image width should be a multiple of 4 pixels
5. Height and width fields of the BMP header (DIB header) must be set to actual image dimensions
6. Image must not contain “ICC color profile”
7. Image must not contain alpha channel

To convert picture in the required format you can use Free Open Source tool ImageMagick:

- `convert <source_image> -type truecolor -separate -swap 0,2 -combine -flip -strip BMP3:<destination`

7.2.1 Convert GIF image to BMP sprite for the Slideshow command

Following commands are used ImageMagick and FFmpeg tools.

- Convert GIF into picture list:

  `ffmpeg -i input.gif -vsync 0 out%03d.png`

- Combine pictures into BMP:

  `convert out*.png -append [options from the previous chapter] sprite.bmp`

- Number of `out*.png` files defines value of `frames_count` field of the command

7.3 Customer MP3 audio files

There are following restrictions applied to the MP3 audio files:

- Only MPEG-1 & MPEG-2 Layer III formats are supported
- Supported sample rates: 22.05, 24, 32, 44.1, 48 kHz
- Only mono audio is supported

7.4 Contactless

IntelliReader provides default (built-in) contactless kernel parameters. In most cases you do not need to change the parameters because they already set to suitable values.

If you think you *need* to change the parameters then contact the support.

7.4.1 Built-in parameters

Supported Application IDs (AID):

- `A0000000031010`
- `A0000000032010`
- `A0000000041010`
- `A0000000043060`
- `A0000006581010`
- `A0000006582010`
- `A0000006581099`

Certificate Authority (CA) keys:

- `A000000003`: indexes `07, 08, 09, 10`
- `A000000004`: indexes `04, 05, 06`
- `A000000658`: indexes `31, 32, 33, 34, 35, 36`

Certificate Revocation List (CRL):

- `A000000658`:
  - index `32`: serial numbers `0004E0 00057F 000580 0005DA 000854 0008C0 0008C1 000961 000B36 000C2E`
  - index `35`: serial numbers `000B83 000B84 000D9D 000DA6 000F2A 000FB1 001181`
  - index `36`: serial numbers `000FCD`

Transaction Amount limits:

- Visa/PayWave:
  - Transaction Limit: `1000.00`
  - CVM Required Limit: `1000.00`
  - Floor Limit: `1000.00`
- MasterCard/PayPass:
  - Transaction Limit (On-device CVM): `1000.00`
  - Transaction Limit (No On-device CVM): `1000.00`
  - CVM Required Limit: `1000.00`
  - Floor Limit: `0.00`
- MIR:
  - Transaction Limit (CD-CVM): `3000.00`
  - Transaction Limit (Non CD-CVM): `3000.00`
  - CVM Required Limit: `3000.00`
  - Floor Limit: `0.00`

Kernel common options:

- Mandatory Offline Data Authentication (ODA): enabled
- Expired cards: accept cards expired in February 2022 or later

Kernel specific options:

- Visa/PayWave:
  - MagStripe emulation (MSD): unsupported

  - qVSDC: supported
  - ODA for Online Authorizations: supported
  - Online PIN & Signature: supported
- MasterCard/PayPass:
  - MagStripe emulation (MSD): disabled
- MIR:
  - Delayed Authorization Mode: enabled
  - Allow Expired Card (AEC): enabled
  - Threshold Application Expiration Date (TAED): `220228`

7.5 Yoko QR-scanners

There are different versions of the Yoko QR-scanner that use different protocols. IntelliReader automatically detects the protocol at the first interaction with the scanner.

It's possible to change default configuration of the scanner by upload a configuration JSON-file within standard reader configuration archive.

Upload `e4000.json` file to configure E4000 and `e10.json` for E10.

The configuration file format:

- `settings_{,v2}`: list of commands (settings) to send to the scanner, may be empty
- `comm`: command of the scanner, the number in hexdecimal format
- `data`: data to be passed along with command, an array of bytes in hexdecimal format
- `mask`: optional parameter used to change one or more bits in the configuration without modifying others, format the same as for `data`

In `e4000.json` the `settings` key is used for E4000 version 1 and the `settings_v2` key for E4000 version 2.

It's recommended to fill all `settings*` objects.

IntelliReader uses the configuration at the first attempt to work with the scanner, for example first `Poll for QR-code` command.

IntelliReader processes the configuration file in the following manner:

1. Parse the file and check that it's well-formed
2. For each command in the respective settings section from the top to bottom:
   1. Read out current value of the data from the scanner
   2. If current value is not equal to configured one then write the new value to the scanner

Please, note that IntelliReader forces the following settings to the "STX" value:

- "Start Character" for E4000
- "Prefix" for E10

There is no way to change this because IntelliReader's internal logic is built upon that setting.

7.5.1 E4000 default (built-in) configuration

```
{
  "settings": [
    { "comm": "0x0028", "data": [ "0x00" ] },
    { "comm": "0x0095", "data": [ "0x01" ] }
  ],
  "settings_v2": [
    { "comm": "0x030201", "data": ["0x02", "0x00", "0x00", "0x00"] },
    { "comm": "0x061802", "data": ["0x01", "0x00", "0x00", "0x00"] }
  ]
}
```

7.5.2 E10 default (built-in) configuration

Since v. 1.44:

```
{
  "settings": [
    { "comm": "0x0000", "data": [ "0xD4" ] },
    { "comm": "0x0006", "data": [ "0x0A" ] }
  ]
}
```

7.6 Hardware configuration

The `hwconfig.json` file within the configuration can be used to change some parameters defined at the factory.

7.6.1 RGB LEDs brightness

Use the `lcd_brightness` parameter to change the max RGB LEDs brightness.

Example: set the max brightness to 50% of the full brightness:

```
{ "lcd_brightness": "50,50,50" }
```

Each number is corresponding to the red, green and blue color component accordingly.

7.7 Apple Enhanced Contactless Polling (ECP) configuration

The `nfc.json` file within the configuration can be used to configure the Apple ECP VASUP-A frame value.

Example:

```
{"ecp": { "frame": "6a02c8010003ffffff010000f0" } }
```

Type A CRC bytes will be added automatically.

Reader checks the file at startup and automatically adds the Apple ECP VASUP-A frame into the polling cycle.

Possibility to change the Apple ECP VASUP-A frame value is available since v. 1.50.

Chapter 8

Annex C Built-in Fonts Metrics

8.1 Regular Font Symbols

Char	Code	WxH	Char	Code	WxH	Char	Code	WxH	Char	Code	WxH
0	U+0030	14x30	1	U+0031	14x30	2	U+0032	14x30	3	U+0033	14x30
4	U+0034	14x30	5	U+0035	14x30	6	U+0036	14x30	7	U+0037	14x30
8	U+0038	14x30	9	U+0039	14x30		U+0020	7x30	!	U+0021	8x30
”	U+0022	12x30	#	U+0023	14x30	$	U+0024	14x30	%	U+0025	22x30
&	U+0026	18x30	’	U+0027	6x30	(	U+0028	8x30	)	U+0029	8x30
*	U+002A	10x30	+	U+002B	15x30	,	U+002C	6x30	-	U+002D	8x30
.	U+002E	6x30	/	U+002F	7x30	:	U+003A	6x30	;	U+003B	6x30
<	U+003C	15x30	=	U+003D	15x30	>	U+003E	15x30	?	U+003F	15x30
@	U+0040	24x30	[	U+005B	8x30	\	U+005C	7x30	]	U+005D	8x30
^	U+005E	15x30	_	U+005F	14x30	‘	U+0060	6x30	{	U+007B	10x30
|	U+007C	7x30	}	U+007D	10x30	~	U+007E	13x30	№	U+2116	25x30
…	U+2026	19x30	A	U+0041	17x30	B	U+0042	18x30	C	U+0043	18x30
D	U+0044	18x30	E	U+0045	17x30	F	U+0046	16x30	G	U+0047	19x30
H	U+0048	19x30	I	U+0049	8x30	J	U+004A	14x30	K	U+004B	18x30
L	U+004C	15x30	M	U+004D	22x30	N	U+004E	19x30	O	U+004F	20x30
P	U+0050	17x30	Q	U+0051	20x30	R	U+0052	18x30	S	U+0053	17x30
T	U+0054	16x30	U	U+0055	18x30	V	U+0056	17x30	W	U+0057	24x30
X	U+0058	17x30	Y	U+0059	16x30	Z	U+005A	15x30	a	U+0061	14x30
b	U+0062	16x30	c	U+0063	14x30	d	U+0064	16x30	e	U+0065	15x30
f	U+0066	9x30	g	U+0067	15x30	h	U+0068	15x30	i	U+0069	7x30
j	U+006A	7x30	k	U+006B	14x30	l	U+006C	7x30	m	U+006D	22x30
n	U+006E	15x30	o	U+006F	15x30	p	U+0070	16x30	q	U+0071	16x30
r	U+0072	10x30	s	U+0073	14x30	t	U+0074	8x30	u	U+0075	15x30
v	U+0076	14x30	w	U+0077	20x30	x	U+0078	14x30	y	U+0079	14x30
z	U+007A	13x30	А	U+0410	17x30	Б	U+0411	18x30	В	U+0412	18x30
Г	U+0413	15x30	Д	U+0414	22x30	Е	U+0415	16x30	Ё	U+0401	16x30
Ж	U+0416	27x30	З	U+0417	17x30	И	U+0418	19x30	Й	U+0419	19x30
К	U+041A	18x30	Л	U+041B	18x30	М	U+041C	22x30	Н	U+041D	19x30
О	U+041E	20x30	П	U+041F	19x30	Р	U+0420	17x30	С	U+0421	18x30
Т	U+0422	16x30	У	U+0423	18x30	Ф	U+0424	23x30	Х	U+0425	16x30
Ц	U+0426	20x30	Ч	U+0427	18x30	Ш	U+0428	26x30	Щ	U+0429	28x30
Ъ	U+042A	21x30	Ы	U+042B	24x30	Ь	U+042C	17x30	Э	U+042D	18x30
Ю	U+042E	27x30	Я	U+042F	18x30	а	U+0430	14x30	б	U+0431	15x30
в	U+0432	14x30	г	U+0433	11x30	д	U+0434	17x30	е	U+0435	15x30
ё	U+0451	15x30	ж	U+0436	20x30	з	U+0437	14x30	и	U+0438	16x30
й	U+0439	16x30	к	U+043A	14x30	л	U+043B	14x30	м	U+043C	17x30
н	U+043D	15x30	о	U+043E	15x30	п	U+043F	15x30	р	U+0440	16x30
с	U+0441	14x30	т	U+0442	11x30	у	U+0443	14x30	ф	U+0444	24x30
х	U+0445	13x30	ц	U+0446	16x30	ч	U+0447	14x30	ш	U+0448	22x30

щ	U+0449	24x30	ъ	U+044A	18x30	ы	U+044B	20x30	ь	U+044C	14x30
э	U+044D	14x30	ю	U+044E	21x30	я	U+044F	15x30	Ç	U+00C7	18x30
Ə	U+018F	20x30	Ğ	U+011E	19x30	İ	U+0130	8x30	Ö	U+00D6	20x30
Ş	U+015E	17x30	Ü	U+00DC	18x30	ç	U+00E7	14x30	ə	U+0259	15x30
ğ	U+011F	15x30	ı	U+0131	7x30	ö	U+00F6	15x30	ş	U+015F	14x30
ü	U+00FC	15x30	Ї	U+0407	8x30	ї	U+0457	8x30	Ґ	U+0490	16x30
ґ	U+0491	11x30	Є	U+0404	18x30	є	U+0454	14x30	І	U+0406	8x30
і	U+0456	7x30	Ա	U+0531	18x30	Բ	U+0532	18x30	Գ	U+0533	19x30
Դ	U+0534	19x30	Ե	U+0535	18x30	Զ	U+0536	20x30	Է	U+0537	12x30
Ը	U+0538	18x30	Թ	U+0539	21x30	Ժ	U+053A	17x30	Ի	U+053B	18x30
Լ	U+053C	10x30	Խ	U+053D	22x30	Ծ	U+053E	21x30	Կ	U+053F	18x30
Հ	U+0540	16x30	Ձ	U+0541	17x30	Ղ	U+0542	19x30	Ճ	U+0543	17x30
Մ	U+0544	19x30	Յ	U+0545	16x30	Ն	U+0546	19x30	Շ	U+0547	16x30
Ո	U+0548	19x30	Չ	U+0549	16x30	Պ	U+054A	18x30	Ջ	U+054B	20x30
Ռ	U+054C	19x30	Ս	U+054D	19x30	Վ	U+054E	19x30	Տ	U+054F	17x30
Ր	U+0550	18x30	Ց	U+0551	16x30	Ւ	U+0552	11x30	Փ	U+0553	20x30
Ք	U+0554	17x30	Օ	U+0555	20x30	Ֆ	U+0556	19x30	ա	U+0561	22x30
բ	U+0562	15x30	գ	U+0563	16x30	դ	U+0564	16x30	ե	U+0565	15x30
զ	U+0566	16x30	է	U+0567	10x30	ը	U+0568	15x30	թ	U+0569	16x30
ժ	U+056A	18x30	ի	U+056B	15x30	լ	U+056C	8x30	խ	U+056D	21x30
ծ	U+056E	17x30	կ	U+056F	15x30	հ	U+0570	15x30	ձ	U+0571	15x30
ղ	U+0572	16x30	ճ	U+0573	15x30	մ	U+0574	15x30	յ	U+0575	8x30
ն	U+0576	15x30	շ	U+0577	14x30	ո	U+0578	15x30	չ	U+0579	10x30
պ	U+057A	22x30	ջ	U+057B	14x30	ռ	U+057C	15x30	ս	U+057D	15x30
վ	U+057E	16x30	տ	U+057F	24x30	ր	U+0580	15x30	g	U+0581	15x30
ւ	U+0582	9x30	փ	U+0583	24x30	ք	U+0584	15x30	o	U+0585	15x30
ֆ	U+0586	19x30	և	U+0587	16x30	´	U+055B	6x30	á	U+00E1	14x30
é	U+00E9	15x30	í	U+00ED	7x30	ó	U+00F3	15x30	ő	U+0151	15x30
ú	U+00FA	15x30	ű	U+0171	15x30	Á	U+00C1	17x30	É	U+00C9	17x30
Í	U+00CD	8x30	Ó	U+00D3	20x30	Ő	U+0150	20x30	Ú	U+00DA	18x30
Ű	U+0170	18x30	Ə	U+04D8	18x30	Ғ	U+0492	16x30	Қ	U+049A	19x30
Ң	U+04A2	21x30	Ө	U+04E8	20x30	Ұ	U+04B0	16x30	Ү	U+04AE	16x30
һ	U+04BA	17x30	ə	U+04D9	15x30	ғ	U+0493	11x30	қ	U+049B	14x30
ң	U+04A3	16x30	ө	U+04E9	15x30	ұ	U+04B1	15x30	ү	U+04AF	14x30
ҳ	U+04BB	15x30	₽	U+20BD	18x30	֏	U+058F	19x30	A	U+0391	17x30
α	U+03B1	16x30	ά	U+03AC	16x30	B	U+0392	17x30	β	U+03B2	15x30
Γ	U+0393	16x30	γ	U+03B3	14x30	Δ	U+0394	19x30	δ	U+03B4	16x30
E	U+0395	16x30	ε	U+03B5	14x30	έ	U+03AD	14x30	Z	U+0396	15x30
ζ	U+03B6	13x30	H	U+0397	19x30	η	U+03B7	15x30	ή	U+03AE	15x30
Θ	U+0398	20x30	θ	U+03B8	15x30	I	U+0399	8x30	ι	U+03B9	9x30
ί	U+03AF	9x30	ϊ	U+03CA	9x30	K	U+039A	18x30	κ	U+03BA	14x30
Λ	U+039B	18x30	λ	U+03BB	15x30	M	U+039C	23x30	μ	U+03BC	15x30
N	U+039D	19x30	ν	U+03BD	14x30	Ξ	U+039E	17x30	ξ	U+03BE	13x30
O	U+039F	20x30	o	U+03BF	15x30	ό	U+03CC	15x30	Π	U+03A0	19x30
π	U+03C0	16x30	P	U+03A1	18x30	ρ	U+03C1	16x30	Σ	U+03A3	17x30
σ	U+03C3	17x30	ς	U+03C2	15x30	T	U+03A4	16x30	τ	U+03C4	14x30
Y	U+03A5	19x30	υ	U+03C5	15x30	ύ	U+03CD	15x30	ü	U+03CB	15x30
Φ	U+03A6	20x30	φ	U+03C6	20x30	X	U+03A7	20x30	χ	U+03C7	16x30
Ψ	U+03A8	21x30	ψ	U+03C8	18x30	Ω	U+03A9	20x30	ω	U+03C9	20x30
ώ	U+03CE	20x30	Ў	U+040E	18x30	ў	U+045E	14x30	Ă	U+0102	17x30
ă	U+0103	14x30	Â	U+00C2	17x30	â	U+00E2	14x30	Î	U+00CE	8x30
î	U+00EE	8x30	Ș	U+0218	17x30	ș	U+0219	14x30	Ț	U+021A	16x30
ț	U+021B	8x30									

Table 8.1: Regular Font

8.2 Large Font Symbols

Char	Code	WxH	Char	Code	WxH	Char	Code	WxH	Char	Code	WxH
0	U+0030	26x56	1	U+0031	26x56	2	U+0032	26x56	3	U+0033	26x56
4	U+0034	26x56	5	U+0035	26x56	6	U+0036	26x56	7	U+0037	26x56
8	U+0038	26x56	9	U+0039	26x56		U+0020	13x56	!	U+0021	16x56
”	U+0022	22x56	#	U+0023	26x56	$	U+0024	26x56	%	U+0025	42x56
&	U+0026	34x56	’	U+0027	12x56	(	U+0028	16x56	)	U+0029	16x56
*	U+002A	18x56	+	U+002B	27x56	,	U+002C	12x56	-	U+002D	16x56
.	U+002E	12x56	/	U+002F	13x56	:	U+003A	12x56	;	U+003B	12x56
<	U+003C	27x56	=	U+003D	27x56	>	U+003E	27x56	?	U+003F	29x56
@	U+0040	46x56	[	U+005B	16x56	\	U+005C	13x56	]	U+005D	16x56
^	U+005E	27x56	_	U+005F	26x56	‘	U+0060	12x56	{	U+007B	18x56
|	U+007C	13x56	}	U+007D	18x56	~	U+007E	23x56	№	U+2116	47x56
…	U+2026	35x56	A	U+0041	33x56	B	U+0042	33x56	C	U+0043	34x56
D	U+0044	34x56	E	U+0045	31x56	F	U+0046	30x56	G	U+0047	36x56
H	U+0048	35x56	I	U+0049	15x56	J	U+004A	27x56	K	U+004B	34x56
L	U+004C	29x56	M	U+004D	41x56	N	U+004E	35x56	O	U+004F	37x56
P	U+0050	32x56	Q	U+0051	37x56	R	U+0052	34x56	S	U+0053	32x56
T	U+0054	30x56	U	U+0055	35x56	V	U+0056	31x56	W	U+0057	45x56
X	U+0058	32x56	Y	U+0059	30x56	Z	U+005A	29x56	a	U+0061	27x56
b	U+0062	29x56	c	U+0063	26x56	d	U+0064	29x56	e	U+0065	27x56
f	U+0066	16x56	g	U+0067	29x56	h	U+0068	28x56	i	U+0069	13x56
j	U+006A	13x56	k	U+006B	27x56	l	U+006C	13x56	m	U+006D	42x56
n	U+006E	29x56	o	U+006F	29x56	p	U+0070	29x56	q	U+0071	29x56
r	U+0072	18x56	s	U+0073	26x56	t	U+0074	16x56	u	U+0075	29x56
v	U+0076	25x56	w	U+0077	37x56	x	U+0078	26x56	y	U+0079	26x56
z	U+007A	24x56	А	U+0410	33x56	Б	U+0411	33x56	В	U+0412	33x56
Г	U+0413	29x56	Д	U+0414	42x56	Е	U+0415	31x56	Ё	U+0401	31x56
Ж	U+0416	51x56	З	U+0417	32x56	И	U+0418	35x56	Й	U+0419	35x56
К	U+041A	34x56	Л	U+041B	33x56	М	U+041C	41x56	Н	U+041D	35x56
О	U+041E	37x56	П	U+041F	35x56	Р	U+0420	32x56	С	U+0421	34x56
Т	U+0422	30x56	У	U+0423	33x56	Ф	U+0424	43x56	Х	U+0425	31x56
Ц	U+0426	38x56	Ч	U+0427	33x56	Ш	U+0428	50x56	Щ	U+0429	54x56
Ъ	U+042A	40x56	Ы	U+042B	46x56	Ь	U+042C	32x56	Э	U+042D	34x56
Ю	U+042E	51x56	Я	U+042F	33x56	а	U+0430	27x56	б	U+0431	28x56
в	U+0432	26x56	г	U+0433	20x56	д	U+0434	32x56	е	U+0435	27x56
ё	U+0451	27x56	ж	U+0436	37x56	з	U+0437	27x56	и	U+0438	29x56
й	U+0439	29x56	к	U+043A	26x56	л	U+043B	27x56	м	U+043C	32x56
н	U+043D	29x56	о	U+043E	29x56	п	U+043F	29x56	р	U+0440	29x56
с	U+0441	26x56	т	U+0442	21x56	у	U+0443	26x56	ф	U+0444	46x56
х	U+0445	25x56	ц	U+0446	31x56	ч	U+0447	27x56	ш	U+0448	42x56
щ	U+0449	46x56	ъ	U+044A	33x56	ы	U+044B	37x56	ь	U+044C	27x56
э	U+044D	27x56	ю	U+044E	40x56	я	U+044F	27x56	Ç	U+00C7	34x56
Ə	U+018F	37x56	Ğ	U+011E	36x56	İ	U+0130	15x56	Ö	U+00D6	37x56
Ş	U+015E	32x56	Ü	U+00DC	35x56	ç	U+00E7	26x56	ə	U+0259	27x56
ğ	U+011F	29x56	ı	U+0131	13x56	ö	U+00F6	29x56	ş	U+015F	26x56
ü	U+00FC	29x56	Ї	U+0407	15x56	ї	U+0457	14x56	Ґ	U+0490	30x56
ґ	U+0491	21x56	Є	U+0404	34x56	є	U+0454	27x56	І	U+0406	15x56
і	U+0456	13x56	Ա	U+0531	34x56	Բ	U+0532	34x56	Գ	U+0533	36x56
Դ	U+0534	35x56	Ե	U+0535	33x56	Զ	U+0536	38x56	Է	U+0537	23x56
Ը	U+0538	34x56	Թ	U+0539	40x56	Ժ	U+053A	32x56	Ի	U+053B	34x56
Լ	U+053C	20x56	Խ	U+053D	42x56	Ծ	U+053E	40x56	Կ	U+053F	33x56
Հ	U+0540	31x56	Ձ	U+0541	31x56	Ղ	U+0542	35x56	Ճ	U+0543	33x56
Մ	U+0544	36x56	Յ	U+0545	30x56	Ն	U+0546	35x56	Շ	U+0547	31x56
Ո	U+0548	35x56	Չ	U+0549	31x56	Պ	U+054A	34x56	Ջ	U+054B	38x56
Ռ	U+054C	35x56	Ս	U+054D	35x56	Վ	U+054E	36x56	Տ	U+054F	32x56
Ր	U+0550	34x56	Ց	U+0551	30x56	Ւ	U+0552	20x56	Փ	U+0553	38x56
Ք	U+0554	32x56	Օ	U+0555	38x56	Ֆ	U+0556	35x56	ш	U+0561	41x56
բ	U+0562	29x56	գ	U+0563	30x56	դ	U+0564	30x56	ե	U+0565	28x56

ɋ	U+0566	30x56	ɬ	U+0567	20x56	ɒ	U+0568	29x56	ɸ	U+0569	31x56
ժ	U+056A	33x56	ɦ	U+056B	29x56	ɭ	U+056C	15x56	ɮ	U+056D	39x56
ծ	U+056E	31x56	ɥ	U+056F	29x56	ɧ	U+0570	29x56	ð	U+0571	29x56
ɳ	U+0572	29x56	ɓ	U+0573	29x56	ʊ	U+0574	29x56	ɟ	U+0575	15x56
ʉ	U+0576	29x56	ƨ	U+0577	25x56	ɲ	U+0578	28x56	ɾ	U+0579	18x56
ɯ	U+057A	41x56	ɋ	U+057B	27x56	ɾ	U+057C	28x56	ɯ	U+057D	28x56
ɥ	U+057E	29x56	ɯ	U+057F	45x56	ɲ	U+0580	29x56	ɡ	U+0581	29x56
ɪ	U+0582	16x56	ɸ	U+0583	45x56	ɓ	U+0584	29x56	ο	U+0585	28x56
ֆ	U+0586	36x56	ɯ	U+0587	29x56	՛	U+055B	12x56	á	U+00E1	27x56
é	U+00E9	27x56	í	U+00ED	13x56	ó	U+00F3	29x56	ő	U+0151	29x56
ú	U+00FA	29x56	ű	U+0171	29x56	Á	U+00C1	33x56	É	U+00C9	31x56
Í	U+00CD	15x56	Ó	U+00D3	37x56	Ő	U+0150	37x56	Ú	U+00DA	35x56
Ű	U+0170	35x56	Ə	U+04D8	34x56	Ғ	U+0492	29x56	Ҟ	U+049A	36x56
Ҥ	U+04A2	39x56	Ө	U+04E8	37x56	Ұ	U+04B0	30x56	Ү	U+04AE	30x56
ҳ	U+04BA	32x56	ə	U+04D9	27x56	ғ	U+0493	21x56	ҟ	U+049B	27x56
ҥ	U+04A3	31x56	ө	U+04E9	28x56	ұ	U+04B1	28x56	ү	U+04AF	27x56
ҳ	U+04BB	28x56	₽	U+20BD	35x56	Ֆ	U+058F	35x56	Α	U+0391	33x56
α	U+03B1	30x56	ά	U+03AC	30x56	Β	U+0392	32x56	β	U+03B2	28x56
Γ	U+0393	30x56	γ	U+03B3	27x56	Δ	U+0394	35x56	δ	U+03B4	29x56
Ε	U+0395	31x56	ε	U+03B5	27x56	έ	U+03AD	27x56	Ζ	U+0396	29x56
ζ	U+03B6	24x56	Η	U+0397	35x56	η	U+03B7	28x56	ή	U+03AE	28x56
Θ	U+0398	37x56	θ	U+03B8	28x56	Ι	U+0399	15x56	ι	U+03B9	16x56
ί	U+03AF	17x56	ϊ	U+03CA	16x56	Κ	U+039A	34x56	κ	U+03BA	27x56
Λ	U+039B	34x56	λ	U+03BB	28x56	Μ	U+039C	43x56	μ	U+03BC	29x56
Ν	U+039D	36x56	ν	U+03BD	27x56	Ξ	U+039E	32x56	ξ	U+03BE	25x56
Ο	U+039F	37x56	ο	U+03BF	29x56	ό	U+03CC	29x56	Π	U+03A0	36x56
π	U+03C0	31x56	Ρ	U+03A1	34x56	ρ	U+03C1	30x56	Σ	U+03A3	32x56
σ	U+03C3	31x56	ς	U+03C2	28x56	Τ	U+03A4	31x56	τ	U+03C4	26x56
Υ	U+03A5	36x56	υ	U+03C5	29x56	ύ	U+03CD	29x56	ϋ	U+03CB	29x56
Φ	U+03A6	37x56	φ	U+03C6	37x56	Χ	U+03A7	37x56	χ	U+03C7	30x56
Ψ	U+03A8	39x56	ψ	U+03C8	34x56	Ω	U+03A9	38x56	ω	U+03C9	38x56
ώ	U+03CE	38x56	Ў	U+040E	33x56	ў	U+045E	26x56	Ă	U+0102	33x56
ă	U+0103	27x56	Â	U+00C2	33x56	â	U+00E2	27x56	Î	U+00CE	15x56
î	U+00EE	14x56	Ș	U+0218	32x56	ș	U+0219	26x56	Ț	U+021A	30x56
ț	U+021B	16x56									

Table 8.2: Large Font

8.3 Monospace Font Symbols

The monospace font contains the same character set as the regular font with the following dimensions:

- Width: 15
- Height: 29